

Università di Pisa

Dipartimento di Informatica

Generative and Deep Learning

Risposte alle domande d'esame

Volume di riscontro per *Dispensa_GDL.pdf*

*Le risposte seguono capitolo per capitolo e domanda per domanda
la numerazione della dispensa. Non sostituiscono la dispensa:
servono a verificare quello che hai già provato a dire.*

August 24, 2026

Contents

Come usare questo file	5
1 Parte I — Fondamenti di modelli probabilistici e causalità	6
Capitolo 1 — Introduzione al corso	6
Capitolo 2 — Probabilità e modelli grafici	7
Capitolo 3 — Reti Bayesiane I	8
Capitolo 4 — Reti Bayesiane II	9
Capitolo 5 — Causalità	11
Capitolo 6 — Structure learning	13
2 Parte II — Learning nei modelli probabilistici	16
Capitolo 7 — Apprendimento con variabili osservate	16
Capitolo 8 — EM e mixture of Gaussians	18
Capitolo 9 — Modelli Markov nascosti	19
Capitolo 10 — Inferenza variazionale	21
Capitolo 11 — Latent Dirichlet Allocation	23
Capitolo 12 — Metodi di campionamento	24
Capitolo 13 — Campi casuali Markoviani	26
3 Parte III — Deep learning fondamentale	29
Capitolo 14 — Convolutional Neural Networks	29
Capitolo 15 — Propagazione dell’informazione	32
Capitolo 16 — Modelli ricorrenti con porte	34
Capitolo 17 — Encoder-decoder e cross-attention	36
Capitolo 18 — Transformers	39
4 Parte IV — Generative deep learning	43
Capitolo 19 — Autoencoders e manifold learning	43
Capitolo 20 — Variational Autoencoders	45
Capitolo 21 — Generative Adversarial Networks	48
Capitolo 22 — Normalizing Flows	51
Capitolo 23 — Diffusion Models	55
Capitolo 24 — Score matching e flow matching	58
Capitolo 25 — Causal Representation Learning	62
5 Parte V — Graph Neural Networks	66
Capitolo 26 — Fondamenti di deep learning per grafi	66

Capitolo 27 — Message passing e GNN convoluzionali	68
Capitolo 28 — Graph attention e graph transformer	72
6 Parte VI — Reinforcement Learning	78
Capitolo 29 — Fondamenti di RL	78
Capitolo 30 — Algoritmi RL	81
7 Parte VII — GrIDDD	87
Capitolo 31 — GrIDDD (L1)	87
Capitolo 31 — GrIDDD (L2)	88
Capitolo 31 — GrIDDD (L3)	90

Come usare questo file

Questo volume contiene una risposta per ognuna delle domande d'esame della dispensa, nello stesso ordine e con la stessa numerazione: la domanda 3 del capitolo 14 della dispensa è la D3 della sezione "Capitolo 14" qui dentro.

Leggi questo prima di usarlo

Un file di risposte è lo strumento di studio più pericoloso che esista, per una ragione precisa: leggere una risposta corretta produce la sensazione di saperla. È l'effetto di *fluency*, e sistematicamente sopravvaluta quanto si è imparato.

Il formato dell'orale rende questa illusione costosa. Le regole (slide GDL35) sono tre domande di complessità crescente e, se non rispondi a quella corrente in cinque-dieci minuti, l'orale finisce lì con il voto delle domande già affrontate. Non si viene valutati sul riconoscere: si viene valutati sul *produrre*, a voce, sotto tempo.

Quindi l'unico modo corretto di usare questo file è: rispondi prima, ad alta voce e senza guardare; solo dopo apri la risposta e confronta. Se ti accorgi di stare leggendo la risposta prima di aver provato, stai facendo un'attività diversa dal prepararti — piacevole, ma inutile.

Un secondo criterio, più severo del "mi sembra di saperlo": dopo aver letto la risposta, richiudila e ridilla. Se la seconda volta non esce fluida, non era acquisita.

Che cosa contengono le risposte. Sono scritte come *traccia di ciò che una buona risposta deve contenere*, non come testo da imparare a memoria: i passaggi obbligati delle derivazioni, i numeri quando servono, e i punti in cui una risposta plausibile è sbagliata. Dove la dispensa contiene già una derivazione completa o un esempio numerico, la risposta rimanda al capitolo invece di ricopiarlo.

Copertura: tutte e 235 le domande dei 31 capitoli della dispensa.

Che cosa non c'è, e perché. L'Appendice A della dispensa (p.174–180) è già in formato domanda-risposta: ogni voce ha la traccia della risposta accanto. Duplicarla qui l'avrebbe solo allungata. Anche le domande L2/L3 di `domande_orale_GDL.md` restano dove sono, per la stessa ragione: quel file è costruito per l'interrogazione a voce e porta già le risposte tra parentesi.

Chapter 1

Parte I — Fondamenti di modelli probabilistici e causalità

Capitolo 1 — Introduzione al corso

D1. Qual è la progressione logica dei cinque moduli del corso? Perché si inizia da modelli probabilistici e non direttamente dal deep learning?

La progressione è: fondamenti probabilistici e causalità → learning nei modelli probabilistici → deep learning fondamentale → generative deep learning → modelli avanzati (grafi, RL). Non è un ordine cronologico né di difficoltà crescente: è un ordine *concettuale*.

La ragione per cui si parte dai PGM è che tutti gli oggetti che poi ricompaiono nel deep learning generativo nascono lì, in una forma in cui sono ancora leggibili. L'ELBO si incontra in EM e in inferenza variazionale prima di ricomparire nel VAE e nella diffusion; il message passing si incontra in belief propagation prima di ricomparire nelle GNN e nella self-attention; le variabili latenti si incontrano in GMM e HMM, dove il posterior si calcola esattamente, prima di incontrarle dove va approssimato con una rete.

Detto altrimenti: nella prima metà del corso la probabilità è *esplicita* e si vede cosa si sta facendo; nella seconda è nascosta dentro reti neurali. Chi arriva alla seconda senza la prima impara le formule ma non riconosce che sono le stesse.

D2. Come si relazionano i modelli probabilistici grafici e le reti neurali profonde nell'ambito del generative deep learning?

Tre relazioni, in ordine di profondità.

Il PGM definisce il modello, la rete parametrizza i fattori. In un VAE il grafo è banale ($z \rightarrow x$), ma è un PGM a tutti gli effetti: quello che cambia è che $p(x | z)$ non è una CPT ma una rete. Lo stesso vale per la diffusion, che è una catena $z_T \rightarrow \dots \rightarrow z_0$ con transizioni parametrizzate.

La rete sostituisce l'inferenza, non solo il modello. Nella VI classica si ottimizzano parametri variazionali per ogni esempio; nel VAE si impara una funzione (l'encoder) che li produce in un forward pass. È l'inferenza *ammortizzata*, ed è il vero salto tecnico.

Il grafo diventa il dato. Nelle GNN il grafo non codifica più indipendenze fra variabili ma è l'oggetto da rappresentare, e il message passing viene riusato come schema computazionale con funzioni apprese al posto dei potenziali.

Il senso della sintesi: i PGM danno la semantica (che cosa significa il modello, quali indipendenze assume), le reti danno la capacità espressiva e la scalabilità.

Capitolo 2 — Probabilità e modelli grafici

D1. Dimostrare la regola della catena. Quando è utile sfruttare le indipendenze condizionali?

Dimostrazione per induzione applicando ripetutamente la definizione di condizionata $P(A, B) = P(A | B)P(B)$:

$$P(x_1, \dots, x_N) = P(x_N | x_1, \dots, x_{N-1}) P(x_1, \dots, x_{N-1}) = \dots = \prod_{i=1}^N P(x_i | x_1, \dots, x_{i-1}).$$

Vale per *qualunque* ordinamento delle variabili.

Il punto da dire subito: da sola la chain rule non risparmia nulla. È un'identità, e il numero di parametri resta $k^N - 1$. Il guadagno arriva quando si cancella parte dei condizionanti perché irrilevanti, $P(x_i | x_1, \dots, x_{i-1}) = P(x_i | \text{pa}(x_i))$: la chain rule apre gli slot, le indipendenze condizionali li svuotano.

Da qui segue anche perché l'ordinamento conta: un ordinamento coerente con la struttura causale produce pochi genitori per nodo, uno arbitrario può produrre famiglie grandi che nessuna indipendenza riesce poi a ridurre. Lo stesso insieme di indipendenze può quindi dare grafi di costo molto diverso.

D2. Enunciare il Teorema di Bayes e spiegare prior, likelihood, posterior e evidence.

$$P(h | \mathbf{d}) = \frac{P(\mathbf{d} | h) P(h)}{P(\mathbf{d})}, \quad P(\mathbf{d}) = \sum_{h'} P(\mathbf{d} | h') P(h').$$

Il *prior* $P(h)$ è ciò che si crede prima dei dati. La *likelihood* $P(\mathbf{d} | h)$ è quanto l'ipotesi rende plausibili i dati osservati: attenzione, è una funzione di h , non una distribuzione su h — non integra a uno in h . L'*evidence* $P(\mathbf{d})$ è la costante di normalizzazione, e in pratica è il termine intrattabile che genera metà del corso (VI, sampling, ELBO). Il *posterior* è la credenza aggiornata.

L'uso corretto è quello del test medico (Cap. 2 della dispensa, esempio svolto): con prevalenza 1%, sensibilità 90% e falsi positivi 5%, un test positivo dà $P(\text{malato} | +) \approx 0.154$, non 0.9. Il prior domina perché i sani sono molti di più, e l'evidence a denominatore lo registra.

D3. Dare un esempio dove $X \perp Y$ ma $X \not\perp Y | Z$ e viceversa.

Prima direzione (collider). X, Y due monete eque indipendenti, $Z = X \oplus Y$. Marginalmente $P(X=1, Y=1) = 0.25 = P(X=1)P(Y=1)$. Ma dato $Z = 0$ si ha $X = Y$ con certezza: da indipendenza a dipendenza deterministica, per solo effetto del condizionamento.

Seconda direzione (causa comune). $Z \sim \text{Bern}(0.5)$, con $P(X=1 | Z) = 0.1/0.9$ e $P(Y=1 | Z) = 0.2/0.8$, e $X \perp Y | Z$ per costruzione. Allora $P(X=1) = P(Y=1) = 0.5$ ma $P(X=1, Y=1) = 0.5(0.1)(0.2) + 0.5(0.9)(0.8) = 0.37 \neq 0.25$: marginalmente dipendenti, condizionatamente indipendenti.

La morale che l'esaminatore vuole sentire: sono i due casi opposti — collider e confounder — e la d-separazione serve esattamente a distinguerli guardando solo il grafo.

D4. Scrivere la PDF della Gaussiana multivariata. Cosa rappresenta la matrice Σ ?

$$\mathcal{N}(\mathbf{x} | \boldsymbol{\mu}, \Sigma) = \frac{1}{(2\pi)^{D/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})\right).$$

Σ è la matrice di covarianza, simmetrica e semidefinita positiva. Tre letture, da dare in quest'ordine.

Statistica: $\Sigma_{ij} = \text{Cov}(x_i, x_j)$; la diagonale sono le varianze.

Geometrica: le superfici di livello sono ellissoidi $(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}) = c$; gli autovettori di $\boldsymbol{\Sigma}$ danno gli assi principali e le radici degli autovalori le lunghezze dei semiassi. Il termine all'esponente è la distanza di Mahalanobis.

Strutturale, ed è quella che collega al resto del corso: gli zeri della *precisione* $\boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1}$, non di $\boldsymbol{\Sigma}$, codificano le indipendenze condizionali. $\Lambda_{ij} = 0 \iff x_i \perp x_j \mid \text{resto}$, mentre $\Sigma_{ij} = 0$ dà solo l'indipendenza marginale. È il ponte con i Gaussian graphical model.

D5. *Confrontare stima ML, MAP e Bayesiana. Pro e contro di ciascun approccio.*

Non sono tre algoritmi in competizione ma tre livelli della stessa risposta. Il bayesiano restituisce l'intera $P(\theta \mid D)$; MAP ne restituisce la moda; ML la moda con prior uniforme implicito.

ML: nessun iperparametro da scegliere, invariante per riparametrizzazione, ma con pochi dati sovra-adatta e produce probabilità zero (tre teste su tre danno $\hat{\lambda} = 1$).

MAP: il prior regolarizza ed elimina lo zero (con Beta(2, 2) si ottiene 0.8), è economico quanto ML. Difetti: il prior va scelto, e soprattutto **MAP non è invariante per riparametrizzazione** mentre ML lo è, perché il posterior è una densità su θ e sotto cambio di variabile acquista un Jacobiano che sposta la moda.

Bayesiano: quantifica l'incertezza e dà la predittiva $P(x' \mid D) = \int P(x' \mid \theta)P(\theta \mid D)d\theta$ (nell'esempio: $5/7 \approx 0.714$). Costo: integrali intrattabili fuori dai casi coniugati, ed è da lì che nascono VI e MCMC.

Con molti dati la likelihood domina e le tre risposte convergono: la differenza si vede solo nel regime di pochi dati.

D6. *Qual è il ruolo della distribuzione Dirichlet come prior per distribuzioni categoriche?*

La Dirichlet è la coniugata della categorica: se $\boldsymbol{\mu} \sim \text{Dir}(\boldsymbol{\alpha})$ e si osservano conteggi $\mathbf{n}$, allora $\boldsymbol{\mu} \mid \text{dati} \sim \text{Dir}(\boldsymbol{\alpha} + \mathbf{n})$. La forma funzionale si conserva e l'aggiornamento è una somma di conteggi.

Gli α_k si leggono come *pseudo-conteggi*: il Laplace smoothing è esattamente il MAP con $\alpha_k = 2$, ed è il motivo per cui è una scelta principata e non una toppa. La media a posteriori è $(\alpha_k + n_k) / \sum_j (\alpha_j + n_j)$.

Il parametro controlla anche la *sparsità* sul simpleso: $\alpha < 1$ concentra la massa vicino ai vertici (poche categorie dominanti), $\alpha > 1$ verso il centro (distribuzioni quasi uniformi), $\alpha = 1$ dà l'uniforme sul simpleso. È la leva usata in LDA per decidere se un documento parla di pochi temi o di tutti.

Capitolo 3 — Reti Bayesiane I

D1. *Illustrare come una BN permette di ridurre il numero di parametri. Esempio numerico.*

Ogni nodo costa $(k-1)k^{|\text{pa}(x_i)|}$ parametri, quindi il totale è $\sum_i (k-1)k^{|\text{pa}(x_i)|}$ contro $k^N - 1$ della congiunta piena.

Esempio: cinque variabili binarie con $A \rightarrow C$, $B \rightarrow C$, $C \rightarrow D$, $C \rightarrow E$. Congiunta piena $2^5 - 1 = 31$; con la struttura $1 + 1 + 4 + 2 + 2 = 10$. Per la catena $X_1 \rightarrow \dots \rightarrow X_{100}$ binaria: $1 + 99 \cdot 2 = 199$ contro $2^{100} - 1 \approx 10^{30}$.

Il punto qualitativo che vale il voto: il costo è esponenziale nel *numero di genitori*, non nel numero di variabili. Un DAG con cento nodi e un genitore ciascuno è banale; dieci nodi di cui uno con nove genitori ha già 512 parametri in quella sola famiglia. La sparsità che serve è locale.

D2. *Enunciare la Local Markov Property e mostrare come segue dalla fattorizzazione.*

Enunciato: ogni variabile è indipendente dai suoi non-discendenti, dati i suoi genitori:

$$X_i \perp \text{ND}(X_i) \mid \text{pa}(X_i).$$

Da fattorizzazione a LMP: si ordini topologicamente. Data $P(\mathbf{x}) = \prod_i P(x_i \mid \text{pa}(x_i))$, si marginalizzino i discendenti di X_i : le loro CPT sommano a uno una alla volta partendo dalle foglie, quindi spariscono. Resta un prodotto sui soli non-discendenti e su X_i , in cui l'unico fattore che contiene x_i è $P(x_i \mid \text{pa}(x_i))$. Normalizzando si ottiene $P(x_i \mid \text{nd}) = P(x_i \mid \text{pa}(x_i))$.

L'implicazione vale anche al contrario (LMP $\Rightarrow$ fattorizzazione), quindi le due caratterizzazioni sono equivalenti. E la d-separazione è la versione globale della stessa proprietà: implica la LMP ed è più forte, perché copre insiemi condizionanti arbitrari.

D3. *Descrivere l'Ancestral Sampling. Perché produce campioni corretti dalla congiunta?*

Si ordinano i nodi topologicamente e si campiona ciascuno dalla propria CPT, condizionando sui valori *già estratti* dei genitori.

Correttezza: la probabilità della tupla prodotta è per costruzione $\prod_i P(x_i \mid \text{pa}(x_i))$, che per il teorema di fattorizzazione è esattamente $P(\mathbf{x})$. L'ordinamento topologico non è un dettaglio implementativo ma la condizione di correttezza: garantisce che al turno di ogni nodo i genitori abbiano già un valore, quindi che si stia leggendo la riga giusta della CPT.

Limite da dichiarare sempre: campiona dalla congiunta, ma non sa condizionare su evidenza sui *discendenti*. Per $P(A \mid C=1)$ bisogna generare e scartare i campioni con $C \neq 1$, cioè fare rejection sampling, con un tasso di accettazione che crolla al crescere dell'evidenza.

D4. *Cosa è la notazione plate? Esempio di utilizzo.*

Un rettangolo (*plate*) attorno a un sottografo indica ripetizione: il contenuto va replicato N volte, con l'indice scritto in un angolo. Serve a rappresentare in modo finito modelli con un numero di variabili che dipende dai dati, e rende visibile quali parametri sono *condivisi* (stanno fuori dal plate) e quali sono per esempio (dentro).

Esempio minimo: $\mu \rightarrow x_n$ con x_n dentro un plate N , cioè N osservazioni i.i.d. da un parametro comune. Esempio realistico: LDA, con un plate esterno sui documenti D contenente θ_d , un plate interno sulle posizioni N_d contenente z_{dn} e w_{dn} , e β_k in un plate separato sui topic. Da quel disegno si legge subito che θ è per documento e β è per corpus — che è precisamente la differenza fra LDA e pLSA.

D5. *Due DAG con strutture diverse possono rappresentare le stesse distribuzioni? Quando?*

Sì: sono *Markov-equivalenti* quando implicano le stesse indipendenze condizionali. Il criterio (Verma e Pearl) è: stesso scheletro non orientato e stesse v-structure, cioè stesse triple $X \rightarrow Z \leftarrow Y$ con X e Y non adiacenti.

Esempio: $X \rightarrow Z \rightarrow Y$, $X \leftarrow Z \leftarrow Y$ e $X \leftarrow Z \rightarrow Y$ sono equivalenti (tutti implicano solo $X \perp Y \mid Z$), mentre $X \rightarrow Z \leftarrow Y$ non lo è perché è l'unico con una v-structure, e implica $X \perp Y$ invece.

La conseguenza è pesante e va detta: da dati puramente osservativi si può identificare al massimo la *classe di equivalenza*, rappresentata da una CPDAG. La direzione degli archi non orientati non è recuperabile senza interventi o assunzioni parametriche aggiuntive.

Capitolo 4 — Reti Bayesiane II

D1. *Definire d-separazione e descrivere il criterio per catene, fork e collider.*

X e Y sono d-separati da Z se *ogni* percorso non orientato fra loro è bloccato. Un percorso

è bloccato se contiene un nodo W tale che: (a) W è chain ($\cdot \rightarrow W \rightarrow \cdot$) o fork ($\cdot \leftarrow W \rightarrow \cdot$) e $W \in Z$; oppure (b) W è collider ($\cdot \rightarrow W \leftarrow \cdot$) e né W né alcun suo discendente è in Z .

struttura	senza condizionare	condizionando su Z
chain $X \rightarrow Z \rightarrow Y$	dipendenti	indipendenti
fork $X \leftarrow Z \rightarrow Y$	dipendenti	indipendenti
collider $X \rightarrow Z \leftarrow Y$	indipendenti	dipendenti

Due precisazioni che distinguono la risposta buona: i percorsi si guardano *non orientati* (si può risalire una freccia contromano), e la direzione delle frecce serve solo a stabilire *che tipo di valvola* sia ciascun nodo. Il collider si apre anche osservando un suo discendente, ed è l'errore più frequente.

Vale infine la *soundness*: d-separazione nel grafo implica indipendenza nella distribuzione. Il viceversa richiede la *faithfulness*.

D2. Dato il DAG $A \rightarrow B \leftarrow C \rightarrow D$, verificare se $A \perp D$ e $A \perp D \mid C$.

L'unico percorso fra A e D è $A \rightarrow B \leftarrow C \rightarrow D$, che contiene due nodi intermedi: B (collider) e C (fork).

$A \perp D$? B è un collider non osservato e non ha discendenti osservati, quindi blocca. Il percorso è bloccato $\Rightarrow$ **sì**, $A \perp D$.

$A \perp D \mid C$? C è un fork ed è ora osservato, quindi blocca a sua volta. Inoltre B resta un collider non osservato e continua a bloccare. Il percorso è bloccato due volte $\Rightarrow$ **sì**, $A \perp D \mid C$.

Basta un solo nodo bloccante per chiudere un percorso: non serve che blocchino tutti. Da notare, per mostrare di aver capito il meccanismo: condizionando invece su B (o su un suo discendente) il collider si aprirebbe, ma il percorso resterebbe comunque chiuso da C se anche C fosse osservato — mentre con $\{B\}$ solo, A e D diventerebbero *dipendenti*.

D3. Spiegare il fenomeno di *explaining away* con un esempio concreto.

Due cause indipendenti di un effetto comune diventano dipendenti quando l'effetto è osservato: scoprire che una causa era presente *abbassa* la probabilità dell'altra, perché l'effetto è già spiegato.

Con i numeri della rete allarme (Cap. 4 della dispensa): $P(F=1) = 0.01$, $P(T=1) = 0.02$, $P(A=1 \mid F, T)$ pari a 0.95/0.94/0.29/0.001. Allora

$$P(F=1) = 0.010 \rightarrow P(F=1 \mid A=1) \approx 0.584 \rightarrow P(F=1 \mid A=1, T=1) \approx 0.032.$$

Il primo salto è l'evidenza che risale la freccia; il secondo è l'*explaining away*, con il furto che torna quasi al valore a priori.

Il punto concettuale da nominare: F e T non si influenzano causalmente in nessun momento. Ciò che è cambiato è che, avendo fissato il loro effetto comune, le due cause si contendono la stessa spiegazione. È un effetto della *selezione*, non della causalità — ed è la stessa struttura del bias di selezione (o di Berkson) in statistica.

D4. Definire il Markov Blanket. Perché è importante per *feature selection*?

$$MB(X_i) = \text{pa}(X_i) \cup \text{ch}(X_i) \cup \{\text{co-genitori dei figli}\}, \quad X_i \perp \mathcal{V} \setminus (\{X_i\} \cup MB(X_i)) \mid MB(X_i).$$

Genitori e figli sono attesi; i co-genitori sorprendono, e la ragione è l'*explaining away*: osservare un figlio lo rende un collider e mette X_i in competizione con gli altri genitori di quel figlio, quindi per neutralizzare l'ambiguità serve sapere anche loro.

Per la feature selection: il MB è l'insieme *minimo e sufficiente* per predire X_i . Nulla al di fuori aggiunge informazione una volta noto il blanket, e nulla al suo interno è superfluo. È quindi la soluzione teorica ottima al problema della selezione di variabili, e algoritmi come IAMB e HITON cercano di stimarlo direttamente dai dati.

Attenzione a un limite: “ottimo” vale rispetto alla distribuzione vera e sotto faithfulness. In pratica il MB va stimato, e in campioni piccoli i test di indipendenza sbagliano.

D5. Cosa afferma la *faithfulness assumption*? Perché è necessaria per *structure learning*?

Afferma che le indipendenze della distribuzione sono *tutte e sole* quelle leggibili come d-separazioni nel grafo:

$$X \perp Y \mid Z \text{ in } P \iff X \perp_d Y \mid Z \text{ in } G.$$

La direzione “ $\Leftarrow$ ” è la soundness e vale sempre per costruzione. La faithfulness aggiunge “ $\Rightarrow$ ”: non esistono indipendenze *accidentali*, prodotte da cancellazioni numeriche fra parametri e non dalla struttura.

Serve per lo *structure learning* perché quegli algoritmi *leggono* la struttura dai test di indipendenza: se una indipendenza osservata non corrispondesse a una d-separazione, l'algoritmo cancellerebbe un arco che esiste davvero.

Il controesempio classico: due percorsi causali da X a Y i cui effetti si annullano esattamente (uno positivo e uno negativo di pari intensità). Si osserva $X \perp Y$ pur essendoci un legame causale. Le violazioni esatte hanno misura nulla nello spazio dei parametri — di qui la difesa usuale — ma le *quasi*-violazioni hanno misura positiva e in campioni finiti fanno danno.

Capitolo 5 — Causalità

D1. Perché correlazione non implica causalità? Esempio con tre strutture causali diverse compatibili.

Perché la stessa correlazione osservata fra X e Y è compatibile con strutture causali diverse, che implicano le stesse indipendenze ma predizioni diverse sotto intervento:

1. $X \rightarrow Y$: causalità diretta;
2. $X \leftarrow Y$: causalità inversa;
3. $X \leftarrow C \rightarrow Y$: confondimento, nessun legame causale.

Tutte e tre danno $X \not\perp Y$ e appartengono alla stessa classe di equivalenza di Markov. C'è un quarto meccanismo che non è nemmeno strutturale: il *selection bias*, cioè condizionare implicitamente su un collider per come i dati sono stati raccolti.

La differenza si vede solo intervenendo: in (1) forzare X cambia Y , in (2) e (3) no. Ed è precisamente questo che il do-operator formalizza.

D2. Definire l'operatore $do(\cdot)$. Come differisce da condizionare su un'osservazione?

$P(y \mid do(x))$ è la distribuzione di Y nel modello *mutilato* $G_{\bar{X}}$, ottenuto cancellando tutte le frecce entranti in X e fissando $X = x$.

$P(y \mid x)$ significa: ho osservato $X = x$ e restringo l'attenzione a quella sottopopolazione, che però è selezionata dalle cause di X , le quali influenzano anche Y . $P(y \mid do(x))$ significa: ho imposto $X = x$ dall'esterno, quindi nessuno lo ha più scelto in base alle proprie caratteristiche e le frecce entranti vanno tagliate.

La differenza è tutta nei percorsi *backdoor*. Se non ce ne sono, i due coincidono. La randomizzazione funziona perché taglia le frecce entranti fisicamente invece che sulla lavagna.

D3. Data una struttura con confondimento $C \rightarrow X$, $C \rightarrow Y$, $X \rightarrow Y$, derivare $P(Y \mid do(X))$.

Nel grafo mutilato $G_{\bar{X}}$ si cancella $C \rightarrow X$. Lì X non ha genitori, quindi $C \perp X$, e:

$$P(y \mid do(x)) = P_m(y \mid x) = \sum_c P_m(y \mid x, c) P_m(c \mid x) = \sum_c P(y \mid x, c) P(c).$$

I due passaggi che rendono valida la derivazione: $P_m(y \mid x, c) = P(y \mid x, c)$ perché il meccanismo di Y non è stato toccato (invarianza modulare), e $P_m(c \mid x) = P_m(c) = P(c)$ perché nel grafo mutilato C e X sono indipendenti e il meccanismo di C è invariato.

Questa è la *formula di aggiustamento*. Il contrasto con l'osservazionale è netto: $P(y \mid x) = \sum_c P(y \mid x, c) P(c \mid x)$, che pesa gli strati con $P(c \mid x)$ invece che con $P(c)$. La differenza fra i due pesi è tutto il confondimento — ed è quella che nell'esempio dei calcoli renali ribalta la conclusione da B migliore a A migliore.

D4. Cos'è uno SCM? Come si rappresenta un intervento in uno SCM?

Uno Structural Causal Model è una tripla $(\mathbf{U}, \mathbf{V}, \mathcal{F})$: variabili esogene $\mathbf{U}$ con distribuzione $P(\mathbf{u})$, variabili endogene $\mathbf{V}$, e un insieme di equazioni strutturali

$$V_i := f_i(\text{pa}(V_i), U_i).$$

Il “:=” è asimmetrico apposta: è un'assegnazione, non un'uguaglianza, e non si può invertire come si farebbe in algebra. È lì che sta la causalità, e un sistema di equazioni ordinarie non la contiene.

Un intervento $do(X = x)$ *sostituisce l'equazione* di X con la costante x , lasciando tutte le altre invariate. Questo è il principio di invarianza modulare: le altre equazioni continuano a valere perché descrivono meccanismi fisici distinti. Graficamente corrisponde a cancellare le frecce entranti in X .

Rispetto alle reti Bayesiane causali, lo SCM dice qualcosa in più: fissando anche i valori di $\mathbf{U}$ si ottengono i *controfattuali*, che una BN causale da sola non permette di calcolare.

D5. Descrivere la gerarchia causale. Perché i controfattuali sono al livello più alto?

Tre livelli (Pearl):

1. **Associazione**, $P(y \mid x)$: “che cosa vedo?”. Basta la distribuzione congiunta.
2. **Intervento**, $P(y \mid do(x))$: “che cosa succede se faccio?”. Serve il grafo causale.
3. **Controfattuali**, $P(y_x \mid x', y')$: “che cosa *sarebbe* successo se avessi fatto x , dato che invece ho fatto x' e ho visto y' ?”. Serve lo SCM completo.

La gerarchia è *stretta*: esistono modelli che coincidono a un livello e differiscono al successivo, quindi nessuna quantità di dati del livello i identifica in generale il livello $i + 1$.

I controfattuali stanno in cima perché richiedono di ragionare su due mondi contemporaneamente — quello osservato e quello ipotetico — sullo *stesso individuo*, cioè con gli stessi valori delle esogene $\mathbf{U}$. La procedura è in tre passi: *abduzione* (aggiornare $P(\mathbf{u})$ con l'evidenza osservata), *azione* (mutilare il modello con $do(x)$), *predizione* (calcolare Y nel modello modificato con le $\mathbf{u}$ aggiornate). Il passo di abduzione è quello che i livelli inferiori non hanno.

D6. Cos'è il backdoor criterion? Darne un'applicazione pratica.

Z soddisfa il criterio backdoor rispetto a (X, Y) se: (i) nessun nodo di Z è discendente di X ; (ii) Z blocca tutti i percorsi da X a Y che partono con una freccia *entrante* in X . In tal caso

$$P(y \mid do(x)) = \sum_z P(y \mid x, z) P(z).$$

È il criterio grafico che dice *su che cosa* si può aggiustare, e le due condizioni corrispondono ai due modi di sbagliare: la (i) impedisce di condizionare su mediatori o su collider a valle, la (ii) chiede che il confondimento sia effettivamente chiuso.

Applicazione: calcoli renali, con $Z = \text{gravità del caso}$. Z non è discendente del trattamento e blocca l'unico backdoor $T \leftarrow Z \rightarrow R$. Aggiustando, $P(R \mid \text{do}(A)) = 0.833$ contro $P(R \mid \text{do}(B)) = 0.779$, mentre il confronto grezzo dava 0.78 contro 0.83: la conclusione si ribalta.

Da dire per chiudere bene: il criterio è *sufficiente*, non necessario. Esistono casi identificabili senza un insieme backdoor valido, per esempio tramite il *front-door* o, in generale, tramite do-calculus.

Capitolo 6 — Structure learning

D1. *Descrivere le tre fasi dell'algoritmo PC. Perché produce una CPDAG e non un DAG unico?*

Fase 1, scheletro: si parte dal grafo completo non orientato e si rimuove l'arco $X - Y$ appena si trova un insieme $S \subseteq \text{adj}(X) \setminus \{Y\}$ con $X \perp Y \mid S$, memorizzando $\text{sepset}(X, Y) = S$. Si procede per cardinalità crescente di S ($0, 1, 2, \dots$), il che tiene basso il numero di test.

Fase 2, v-structure: per ogni tripla non adiacente $X - Z - Y$ (con X, Y non collegati), si orienta $X \rightarrow Z \leftarrow Y$ se e solo se $Z \notin \text{sepset}(X, Y)$.

Fase 3, regole di Meek: si propagano gli orientamenti forzati, cioè quelli che eviterebbero di creare nuove v-structure o cicli diretti.

Il risultato è una CPDAG perché i dati osservativi identificano solo la *classe di equivalenza di Markov*: il collider è l'unica struttura con una firma osservazionale propria, mentre chain e fork sono indistinguibili fra loro. Gli archi rimasti non orientati sono quelli che cambiano direzione all'interno della classe.

D2. *Definire il BIC score. Perché include un termine di penalità?*

$$\text{BIC}(G, D) = \log P(D \mid \hat{\theta}_G, G) - \frac{d_G}{2} \log N,$$

con d_G numero di parametri liberi e N la dimensione del campione.

La penalità serve perché la sola log-verosimiglianza è *monotona* nella complessità: aggiungere un arco non può mai peggiorarla, quindi massimizzarla sceglierebbe sempre il grafo completo, che si adatta al rumore. Il termine $-\frac{d}{2} \log N$ è il prezzo per parametro.

Due proprietà da citare. È *consistente*: per $N \rightarrow \infty$ seleziona la vera classe di equivalenza (a differenza di AIC, che penalizza con $-d$ e tende a sovra-selezionare). Ed è *decomponibile*, $\text{BIC}(G) = \sum_i \text{BIC}(X_i \mid \text{pa}(X_i))$, il che permette di valutare l'effetto di un singolo arco senza ricalcolare tutto — proprietà da cui dipende l'efficienza della ricerca greedy.

Origine: è l'approssimazione di Laplace della log-evidenza marginale $\log P(D \mid G)$, quindi non è una penalità arbitraria ma un'approssimazione bayesiana.

D3. *Descrivere GES (fasi forward e backward). Quale proprietà di convergenza ha?*

GES è una ricerca greedy nello spazio delle CPDAG, non dei DAG, il che evita di esplorare separatamente grafi equivalenti.

Forward: partendo dal grafo vuoto, si aggiunge a ogni passo l'arco che massimizza l'incremento di score, finché nessuna aggiunta migliora.

Backward: dal grafo raggiunto, si rimuove a ogni passo l'arco la cui eliminazione massimizza lo score, finché nessuna rimozione migliora.

Proprietà (Chickering 2002): con uno score *consistente* e decomponibile, sotto faithfulness e nel limite di campione infinito, GES converge alla CPDAG corretta. È notevole perché una ricerca greedy in uno spazio super-esponenziale normalmente non ha garanzie: qui la struttura dello spazio delle classi di equivalenza rende ottimale una strategia locale in due fasi.

Confronto con PC: PC è constraint-based e dipende dall'affidabilità dei singoli test (un test sbagliato presto si propaga); GES è score-based e valuta il fit globale, quindi è più robusto al rumore ma più costoso.

D4. *Cos'è un Additive Noise Model? Perché consente di identificare la direzione causale?*

$$Y := f(X) + N_Y, \quad N_Y \perp X.$$

L'identificabilità nasce da un'asimmetria: in generale, se i dati sono generati da un ANM in direzione $X \rightarrow Y$, non esiste un ANM $X := g(Y) + N_X$ con $N_X \perp Y$ che generi la stessa distribuzione congiunta. L'eccezione nota è il caso lineare-gaussiano, che è simmetrico e resta non identificabile — di qui LiNGAM, che recupera l'identificabilità richiedendo rumore *non* gaussiano.

Il test pratico è diretto: si regredisce Y su X , si calcolano i residui e si testa $\hat{N}_Y \perp X$; poi si ripete nella direzione opposta. La direzione in cui i residui risultano indipendenti dal regressore è quella causale.

Perché è importante: rompe la classe di equivalenza di Markov usando *assunzioni parametriche* sulla forma del meccanismo, cioè informazione che PC e GES non usano. Con due sole variabili PC non può dire nulla, un ANM sì. Il prezzo è che la conclusione è valida solo se l'assunzione di additività regge.

D5. *Qual è la differenza tra PC e FCI? Quando si usa FCI?*

PC assume *sufficienza causale*: nessuna variabile latente confonde due variabili osservate. FCI rimuove quell'assunzione.

FCI restituisce un PAG (Partial Ancestral Graph) invece di una CPDAG, con un alfabeto di estremi più ricco: la freccia, il trattino, e il cerchio “o” che significa “non determinato dai dati”. Un arco bidirezionale $X \leftrightarrow Y$ nel PAG ha un significato preciso e prezioso: esiste un **confondente latente** fra X e Y .

Si usa FCI quando la sufficienza causale non è credibile, che è quasi sempre nei dati osservativi reali — in biologia, economia, epidemiologia è normale che qualcosa di rilevante non sia stato misurato. Il costo è duplice: computazionalmente è più pesante (deve considerare anche insiemi di separazione fuori dal vicinato), e statisticamente conclude meno, perché più archi restano indeterminati. È una scelta fra un risultato preciso sotto un'ipotesi forte e uno più debole ma onesto.

D6. *Implementare lo pseudocodice per la skeleton learning del PC.*

```

G ← grafo completo non orientato su V
sepsset[, ·] ← ∅
ℓ ← 0
repeat
  for each arco (X, Y) ∈ G do
    for each S ⊆ adj_G(X) \ {Y} con |S| = ℓ do
      if X ⊥ Y | S (test di indipendenza) then
        rimuovi l'arco (X, Y) da G
        sepsset[X, Y] ← sepsset[Y, X] ← S
        break
    ℓ ← ℓ + 1
until ℓ > max_X |adj_G(X)| - 1
return G, sepsset

```

Tre dettagli che spesso si perdono e che vale la pena nominare. Primo: il vicinato $\text{adj}_G(X)$ si aggiorna *durante* l'esecuzione, quindi ogni rimozione riduce lo spazio dei test successivi — è ciò che rende PC praticabile. Secondo: si procede per ℓ crescente perché così si trovano prima le separazioni più semplici, e i sepset minimali sono quelli che poi servono alla fase di orientamento. Terzo: l'algoritmo è *order-dependent*, cioè l'ordine in cui si esaminano gli archi può cambiare il risultato con campioni finiti; PC-stable corregge questo difetto congelando i vicini all'inizio di ogni livello.

Test tipici: χ^2 o G-test nel caso discreto, correlazione parziale di Fisher nel caso gaussiano.

Chapter 2

Parte II — Learning nei modelli probabilistici

Capitolo 7 — Apprendimento con variabili osservate

D1. Distinguere tra Maximum Likelihood, MAP e approccio Bayesiano. Quali sono i vantaggi e svantaggi di ciascuno?

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} P(D | \theta), \quad \hat{\theta}_{\text{MAP}} = \arg \max_{\theta} P(D | \theta)P(\theta), \quad P(\theta | D) = \frac{P(D | \theta)P(\theta)}{P(D)}.$$

ML e MAP restituiscono un punto, il bayesiano una distribuzione. ML è MAP con prior uniforme.

Con tre teste su tre lanci: $\hat{\lambda}_{\text{ML}} = 1$ (la croce diventa impossibile), $\hat{\lambda}_{\text{MAP}} = 4/5$ con Beta(2, 2), predittiva bayesiana $5/7 \approx 0.714$.

ML: nessun iperparametro, invariante per riparametrizzazione, asintoticamente efficiente. Contro: sovra-adatta con pochi dati, produce probabilità zero.

MAP: regolarizza a costo computazionale nullo, e molti regolarizzatori noti sono MAP travestiti (L2 = prior gaussiano, L1 = prior Laplace). Contro: dipende dal prior e **non è invariante per riparametrizzazione**, perché il posterior è una densità e sotto cambio di variabile acquista un Jacobiano che sposta la moda.

Bayesiano: propaga l'incertezza nelle predizioni, si comporta bene con pochi dati, permette model comparison via evidence. Contro: integrali intrattabili fuori dai coniugati — ed è precisamente il motivo per cui esistono i capitoli su VI e sampling.

D2. Derivare lo stimatore ML per i parametri di una distribuzione categorica.

Dati N campioni i.i.d. con conteggi n_k , la log-verosimiglianza è $\ell(\boldsymbol{\lambda}) = \sum_k n_k \log \lambda_k$ sotto il vincolo $\sum_k \lambda_k = 1$. Con un moltiplicatore di Lagrange:

$$\mathcal{L} = \sum_k n_k \log \lambda_k + \eta \left(1 - \sum_k \lambda_k\right), \quad \frac{\partial \mathcal{L}}{\partial \lambda_k} = \frac{n_k}{\lambda_k} - \eta = 0 \Rightarrow \lambda_k = \frac{n_k}{\eta}.$$

Imponendo la normalizzazione, $\sum_k n_k / \eta = 1 \Rightarrow \eta = N$, quindi

$$\boxed{\hat{\lambda}_k = \frac{n_k}{N}}.$$

Cioè la frequenza empirica. Il vincolo va gestito esplicitamente: derivare senza Lagrange è l'errore tipico.

Per una rete Bayesiana la stessa derivazione si applica *per famiglia*, dando $\hat{\theta}_{ijk} = \#(x_i=k, \text{pa}(x_i)=j) / \#(\text{pa}(x_i)=j)$, cioè un conteggio condizionale per ogni riga di ogni CPT.

D3. *Spiegare come il Laplace smoothing affronta il problema delle probabilità zero nei classificatori Naive Bayes.*

Il problema: se una feature-valore non compare mai con una certa classe nel training set, $\hat{P}(x_d | y) = 0$, e poiché Naive Bayes *moltiplica* i fattori, un solo zero azzerà l'intero prodotto. Una classe viene esclusa per una singola parola mai vista, indipendentemente da quanto tutte le altre la sostengano.

Il rimedio:

$$\hat{P}(x_d = v | y) = \frac{\#(x_d = v, y) + \alpha}{\#(y) + \alpha |V_d|},$$

con $\alpha = 1$ per Laplace. Il denominatore deve compensare i $|V_d|$ pseudo-conteggi aggiunti, altrimenti la distribuzione non normalizza — è l'errore che si vede più spesso.

L'interpretazione principata: è la stima MAP con prior $\text{Dir}(\alpha + 1)$, quindi non è una toppa ma il caso coniugato della domanda precedente. α interpola fra ML ($\alpha \rightarrow 0$) e uniforme ($\alpha \rightarrow \infty$), e il suo effetto svanisce al crescere di N .

D4. *Perché nei classificatori Naive Bayes il denominatore $P(\mathbf{x})$ può essere omissso nella fase di predizione?*

Perché la predizione è un $\arg \max$ su y , e $P(\mathbf{x})$ non dipende da y :

$$\hat{y} = \arg \max_y \frac{P(y) \prod_d P(x_d | y)}{P(\mathbf{x})} = \arg \max_y P(y) \prod_d P(x_d | y).$$

Un fattore positivo costante rispetto alla variabile di ottimizzazione non cambia il punto di massimo. In pratica si lavora in log per evitare underflow: $\hat{y} = \arg \max_y [\log P(y) + \sum_d \log P(x_d | y)]$.

Due precisazioni che completano la risposta. Primo: si può omettere solo per la *classificazione*; se servono probabilità calibrate va ripristinato, e si ottiene con un softmax sui punteggi non normalizzati, $P(\mathbf{x}) = \sum_{y'} P(y') \prod_d P(x_d | y')$. Secondo: le probabilità di Naive Bayes sono comunque notoriamente mal calibrate, perché l'assunzione di indipendenza fa contare più volte l'evidenza correlata e spinge il posterior verso 0 o 1. La classe scelta resta spesso corretta, il numero associato no.

D5. *Come si generalizza il learning MLE a reti Bayesiane arbitrarie?*

La log-verosimiglianza si *decompone* lungo la fattorizzazione:

$$\ell(\theta) = \sum_{n=1}^N \sum_i \log P(x_i^{(n)} | \text{pa}(x_i)^{(n)}, \theta_i) = \sum_i \underbrace{\left[\sum_n \log P(x_i^{(n)} | \text{pa}(x_i)^{(n)}, \theta_i) \right]}_{\text{dipende solo da } \theta_i}.$$

Poiché i parametri di famiglie diverse non compaiono negli stessi termini, il problema si spezza in N problemi indipendenti, uno per nodo, ciascuno risolvibile in forma chiusa come nella D2. Questa è la ragione strutturale per cui il learning con dati completi è facile.

Tre condizioni perché valga: dati completamente osservati, parametri delle diverse famiglie *disgiunti* (niente parameter tying, che altrimenti accoppia i termini), e struttura nota.

Appena una di queste cade, la decomposizione si rompe. In particolare con variabili nascoste compare $\log \sum_z$, il logaritmo non entra più nella somma e la forma chiusa sparisce — che è esattamente il problema del capitolo successivo.

Capitolo 8 — EM e mixture of Gaussians

D1. Derivare l'algoritmo EM da principi variazionali. Come si relaziona al lower bound ELBO?

Per ogni distribuzione $q(\mathbf{z})$, moltiplicando e dividendo dentro il logaritmo e applicando Jensen:

$$\log p(\mathbf{x} | \theta) = \log \sum_{\mathbf{z}} q(\mathbf{z}) \frac{p(\mathbf{x}, \mathbf{z} | \theta)}{q(\mathbf{z})} \geq \underbrace{\mathbb{E}_q \left[\log \frac{p(\mathbf{x}, \mathbf{z} | \theta)}{q(\mathbf{z})} \right]}_{\mathcal{L}(q, \theta)}.$$

E vale l'identità esatta

$$\log p(\mathbf{x} | \theta) = \mathcal{L}(q, \theta) + D_{\text{KL}}(q(\mathbf{z}) \| p(\mathbf{z} | \mathbf{x}, \theta)).$$

EM è coordinate ascent su $\mathcal{L}(q, \theta)$:

- **E-step:** massimizza rispetto a q con θ fisso. Poiché $\log p(\mathbf{x} | \theta)$ non dipende da q , il massimo si ha annullando la KL, cioè $q^*(\mathbf{z}) = p(\mathbf{z} | \mathbf{x}, \theta^{\text{old}})$. Il bound diventa *tight*.
- **M-step:** massimizza rispetto a θ con q fisso. I termini $-\mathbb{E}_q[\log q]$ non dipendono da θ , quindi resta $\theta^{\text{new}} = \arg \max_{\theta} \mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z} | \theta)] = \arg \max_{\theta} Q(\theta, \theta^{\text{old}})$.

La relazione con l'ELBO è quindi di identità: $\mathcal{L}(q, \theta)$ è l'ELBO, e EM è il caso fortunato in cui il posterior esatto è calcolabile e la famiglia variazionale lo contiene. Quando non lo è, si resta con una q approssimata e si ottiene l'inferenza variazionale.

D2. Spiegare come EM garantisce l'aumento monotono della log-verosimiglianza.

Catena in tre disuguaglianze:

$$\log p(\mathbf{x} | \theta^{\text{old}}) \stackrel{(a)}{=} \mathcal{L}(q^*, \theta^{\text{old}}) \stackrel{(b)}{\leq} \mathcal{L}(q^*, \theta^{\text{new}}) \stackrel{(c)}{\leq} \log p(\mathbf{x} | \theta^{\text{new}}).$$

(a) vale perché l'E-step azzerava la KL rendendo il bound tight; (b) perché l'M-step massimizza in θ , quindi non può peggiorare; (c) perché il bound resta un bound per qualunque q .

Il punto sottile è (a): senza il bound tight la catena si spezzerebbe, perché salire su un bound generico non dice nulla sulla funzione. È l'E-step esatto a rendere valido l'argomento — e infatti nella VI, dove il bound non è tight, si garantisce la monotonia dell'ELBO ma non della log-verosimiglianza.

Monotono non significa globale: EM converge a un punto stazionario, tipicamente un massimo locale, e il risultato dipende dall'inizializzazione. Da qui ripartenze multiple o inizializzazione con k-means. Va anche detto che nei GMM la log-verosimiglianza è *illimitata*: una componente che collassa su un punto con $\sigma \rightarrow 0$ la fa divergere, quindi servono vincoli o regolarizzazione.

D3. Derivare le equazioni di aggiornamento E-step e M-step per un GMM.

E-step, responsabilità via Bayes:

$$r_{nk} = P(z_n = k | \mathbf{x}_n, \theta^{\text{old}}) = \frac{\pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}.$$

M-step: si massimizza $Q = \sum_n \sum_k r_{nk} [\log \pi_k + \log \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)]$. Ponendo $N_k = \sum_n r_{nk}$, e annullando le derivate:

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_n r_{nk} \mathbf{x}_n, \quad \boldsymbol{\Sigma}_k = \frac{1}{N_k} \sum_n r_{nk} (\mathbf{x}_n - \boldsymbol{\mu}_k)(\mathbf{x}_n - \boldsymbol{\mu}_k)^\top, \quad \pi_k = \frac{N_k}{N}.$$

Per π_k serve il vincolo $\sum_k \pi_k = 1$, quindi di nuovo un moltiplicatore di Lagrange.

La lettura da dare a voce: sono le formule della stima ML del caso completamente osservato, con i conteggi interi sostituiti da *conteggi frazionari* r_{nk} . Sull'esempio del Cap. 8 (punti 1, 2, 3, 6), $x = 3$ riceve $r = (0.608, 0.392)$ e la log-verosimiglianza passa da -8.140 a -7.287 in un solo giro.

D4. *Quale è la differenza tra EM e k-means? Come si relazionano?*

k-means è il limite di EM per GMM con tre restrizioni: covarianze isotrope $\Sigma_k = \sigma^2 \mathbf{I}$ con $\sigma^2 \rightarrow 0$, pesi π_k uguali e fissi, e responsabilità che diventano *hard* ($r_{nk} \in \{0, 1\}$).

Il meccanismo del limite: nel calcolo di r_{nk} il softmax delle distanze scalate per $1/2\sigma^2$ diventa, per $\sigma^2 \rightarrow 0$, un arg min della distanza euclidea. L'E-step diventa l'assegnazione al centroide più vicino e l'M-step la media dei punti assegnati.

Le conseguenze pratiche di quelle restrizioni: k-means non può rappresentare cluster ellittici o con orientamenti diversi (covarianza isotropa), né cluster di dimensioni molto diverse (pesi fissi), né l'incertezza sui punti di confine (assegnazione hard). In compenso è più veloce e ha meno parametri, ed è per questo che si usa spesso per *inizializzare* EM.

D5. *Come si estende EM a reti Bayesiane arbitrarie? Quali sono i costi computazionali?*

Lo schema è invariato: l'E-step calcola $P(\mathbf{z} \mid \mathbf{x}, \theta^{\text{old}})$ per le variabili nascoste, l'M-step ristima i parametri come nel caso completo ma con *expected sufficient statistics*, cioè conteggi attesi:

$$\hat{\theta}_{ijk} = \frac{\mathbb{E}[\#(x_i=k, \text{pa}(x_i)=j)]}{\mathbb{E}[\#(\text{pa}(x_i)=j)]}, \quad \mathbb{E}[\#(\cdot)] = \sum_n P(x_i=k, \text{pa}(x_i)=j \mid \mathbf{x}^{(n)}, \theta^{\text{old}}).$$

Servono quindi solo i posterior *di famiglia*, non la congiunta completa sulle latenti.

Il costo si sposta tutto sull'E-step, che è un problema di inferenza. Con variable elimination o junction tree, il costo è esponenziale nella *treewidth* del grafo (più precisamente nella dimensione della clique più grande), non nel numero di nodi: alberi e catene sono trattabili, grafi densi no. Baum-Welch per gli HMM è esattamente questo schema, con l'E-step risolto da forward-backward in $\mathcal{O}(K^2T)$.

Quando l'inferenza esatta è impraticabile si passa a EM approssimato: E-step con loopy BP, con inferenza variazionale (variational EM, dove il bound non è più tight) o con campionamento (Monte Carlo EM).

Capitolo 9 — Modelli Markov nascosti

D1. *Derivare l'algoritmo forward per gli HMM e spiegare la complessità computazionale.*

Si definisce $\alpha_t(i) = P(x_{1:t}, z_t = i)$. Marginalizzando sullo stato precedente e usando le due assunzioni di markovianità e indipendenza delle emissioni:

$$\alpha_t(i) = \sum_j P(x_{1:t-1}, z_{t-1} = j) P(z_t = i \mid z_{t-1} = j) P(x_t \mid z_t = i) = \left[\sum_j \alpha_{t-1}(j) A_{ji} \right] B_{ix_t},$$

con inizializzazione $\alpha_1(i) = \pi_i B_{ix_1}$ e terminazione $P(x_{1:T}) = \sum_i \alpha_T(i)$.

Complessità: $\mathcal{O}(K^2T)$ in tempo e $\mathcal{O}(KT)$ in memoria (o $\mathcal{O}(K)$ se serve solo la verosimiglianza). Il confronto da fare è con l'enumerazione ingenua di tutti i cammini, che costerebbe $\mathcal{O}(K^T)$: la programmazione dinamica sfrutta il fatto che il futuro dipende dal passato solo attraverso z_t , quindi tutti i cammini che passano per lo stesso stato allo stesso istante possono essere sommati una volta per tutte.

Nota implementativa da menzionare: le α vanno a underflow per T grande, quindi si normalizza a ogni passo tenendo i fattori di scala, oppure si lavora in log.

D2. *Spiegare la differenza tra forward e Viterbi algorithm. Quale problema risolve ciascuno?*

Hanno la stessa struttura e la stessa complessità $\mathcal{O}(K^2T)$; differiscono per un solo operatore.

Forward usa $\sum$: $\alpha_t(i)$ somma su tutti i modi di arrivare in i . Risolve il problema di **valutazione**, $P(x_{1:T} | \lambda)$, cioè una marginalizzazione. Serve per confrontare modelli o come passo dell'apprendimento.

Viterbi usa max e memorizza i backpointer: $\delta_t(i) = \max_{z_{1:t-1}} P(x_{1:t}, z_{1:t-1}, z_t = i)$. Risolve il problema di **decodifica**, $\arg \max_{z_{1:T}} P(z_{1:T} | x_{1:T})$, cioè un'ottimizzazione, e richiede un backtracking finale per ricostruire il cammino.

Il corollario che vale il voto: la sequenza di Viterbi *non* coincide con la sequenza degli stati singolarmente più probabili $\arg \max_i \gamma_t(i)$. La prima massimizza la congiunta ed è per costruzione un cammino percorribile; la seconda massimizza T marginali separate e può contenere una transizione a probabilità zero, cioè essere una sequenza **impossibile**. Sull'esempio del Cap. 9, $P(x) = 0.146$ contro il contributo 0.0437 del solo cammino ottimo: gli altri sette cammini pesano insieme più di lui.

D3. *Derivare le equazioni di Baum-Welch da EM applicato ai HMM.*

Baum-Welch è EM con $\mathbf{z} = z_{1:T}$ come variabili latenti.

E-step: servono due posterior, calcolabili da forward-backward:

$$\gamma_t(i) = P(z_t = i | x_{1:T}) = \frac{\alpha_t(i)\beta_t(i)}{P(x_{1:T})},$$

$$\xi_t(i, j) = P(z_t = i, z_{t+1} = j | x_{1:T}) = \frac{\alpha_t(i)A_{ij}B_{jx_{t+1}}\beta_{t+1}(j)}{P(x_{1:T})}.$$

M-step: sono i conteggi attesi normalizzati, cioè le formule del caso completamente osservato con i conteggi sostituiti dalle loro attese:

$$\hat{\pi}_i = \gamma_1(i), \quad \hat{A}_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}, \quad \hat{B}_{ik} = \frac{\sum_{t: x_t=k} \gamma_t(i)}{\sum_t \gamma_t(i)}.$$

Da leggere così: $\hat{A}_{ij}$ è il numero atteso di transizioni $i \rightarrow j$ diviso il numero atteso di volte in cui si è stati in i (escluso l'ultimo istante, da cui non si transita). Vale la garanzia di monotonia di EM, quindi anche i limiti: massimi locali e dipendenza dall'inizializzazione, che negli HMM è severa.

D4. *Come si combina l'algoritmo forward e backward per il learning?*

$\alpha_t(i) = P(x_{1:t}, z_t = i)$ raccoglie l'evidenza dal *passato*; $\beta_t(i) = P(x_{t+1:T} | z_t = i)$ raccoglie quella dal *futuro*, con ricorsione all'indietro $\beta_t(i) = \sum_j A_{ij}B_{jx_{t+1}}\beta_{t+1}(j)$ e $\beta_T(i) = 1$.

Il loro prodotto dà la congiunta con tutta la sequenza osservata:

$$\alpha_t(i)\beta_t(i) = P(x_{1:T}, z_t = i) \Rightarrow \gamma_t(i) = \frac{\alpha_t(i)\beta_t(i)}{\sum_j \alpha_t(j)\beta_t(j)}.$$

Serve al learning perché l'E-step ha bisogno del posterior condizionato su *tutta* la sequenza, non solo sul prefisso: usare α da sola darebbe il *filtering* $P(z_t | x_{1:t})$, che è una quantità diversa e sarebbe sbagliata qui. Con α e β si ottiene lo *smoothing* $P(z_t | x_{1:T})$.

Costo complessivo $\mathcal{O}(K^2T)$, cioè lo stesso ordine di un singolo forward. Vale la pena notare che forward-backward è un caso particolare di belief propagation su un albero (qui una catena), dove α e β sono i messaggi nelle due direzioni: è il ponte con il message passing della Parte V.

D5. *Discutere l'estensione degli HMM a reti Bayesiane dinamiche più generali.*

Un HMM è la DBN minima: una sola variabile di stato discreta e una sola osservazione per istante. Una DBN generalizza permettendo un *insieme* di variabili per slice, con una struttura interna arbitraria, e una rete di transizione fra slice consecutive (modello 2-TBN).

Vantaggi: lo stato si fattorizza. Con M variabili binarie una DBN rappresenta 2^M stati con $\mathcal{O}(M \cdot 2^{\text{pa}})$ parametri anziché $\mathcal{O}(4^M)$ della matrice di transizione di un HMM equivalente. Si guadagna interpretabilità e si possono modellare dipendenze fra sottoprocessi.

Costo: l'inferenza esatta si degrada rapidamente. Anche partendo da slice sparse, il filtraggio in avanti *accoppia* le variabili (entanglement) e la belief state diventa densa, quindi la treewidth effettiva cresce e si finisce a inferenza approssimata (Boyer-Koller, particle filter, VI).

Casi particolari da citare: Kalman filter (stato continuo, transizioni ed emissioni lineari-gaussiane; forward-backward diventa filtro e smoother di Kalman), switching state-space model, e HMM fattoriali. E il collegamento in avanti: una RNN è l'analogo deterministico e a stato continuo di questa famiglia, che rinuncia alla semantica probabilistica in cambio di espressività e addestrabilità con la discesa del gradiente.

Capitolo 10 — Inferenza variazionale

D1. *Derivare l'ELBO da principi primi. Spiegare il ruolo della divergenza KL.*

Due derivazioni, entrambe da saper fare.

Via *Jensen*: $\log p(x) = \log \int q(z) \frac{p(x,z)}{q(z)} dz \geq \int q(z) \log \frac{p(x,z)}{q(z)} dz = \mathcal{L}(q)$, per concavità del logaritmo.

Via *identità esatta*, più informativa:

$$D_{\text{KL}}(q(z) \| p(z | x)) = \mathbb{E}_q \left[\log \frac{q(z)}{p(z | x)} \right] = \mathbb{E}_q \left[\log \frac{q(z)p(x)}{p(x,z)} \right] = \log p(x) - \mathcal{L}(q),$$

da cui

$$\boxed{\log p(x) = \mathcal{L}(q) + D_{\text{KL}}(q(z) \| p(z | x))}.$$

Il ruolo della KL è triplice. Essendo non negativa, garantisce che $\mathcal{L}$ sia un lower bound. Essendo $\log p(x)$ indipendente da q , massimizzare $\mathcal{L}$ è minimizzare la KL: si ottimizza una quantità calcolabile e si ottiene gratis l'avvicinamento al posterior, che non è calcolabile. Infine, il gap fra ELBO e log-evidenza è l'errore di approssimazione, quindi l'ELBO è anche una stima (per difetto) della log-verosimiglianza, usabile per model selection.

Verifica numerica nel Cap. 10: con $p(z) = (0.5, 0.5)$ e likelihood $(0.8, 0.2)$, per tre diverse q la somma ELBO + KL resta costante a $\log 0.5 = -0.693$.

D2. *Come si relaziona l'inferenza variazionale a EM?*

Sono lo stesso schema di coordinate ascent sullo stesso funzionale $\mathcal{L}(q, \theta)$; cambia solo quanto è ricca la famiglia in cui vive q .

In *EM* il posterior esatto $p(z | x, \theta)$ è calcolabile e appartiene alla famiglia ammessa: l'E-step lo sceglie, la KL si annulla, il bound diventa tight e la monotonia della log-verosimiglianza è garantita.

In *variational EM* il posterior esatto non è calcolabile, quindi si restringe q a una famiglia $\mathcal{Q}$ (tipicamente mean-field). L'E-step trova la migliore $q \in \mathcal{Q}$, ma la KL resta positiva: il bound non è tight, e si garantisce l'aumento monotono dell'*ELBO*, non della log-verosimiglianza.

In *VI pura* non c'è un θ da stimare: tutto è latente e si ottimizza solo q .

La formula riassuntiva: EM = VI con famiglia variazionale che contiene il posterior vero. E il passo successivo della genealogia è il VAE, dove q è prodotta da una rete (inferenza *ammortizzata*) invece che ottimizzata per esempio.

D3. Derivare la distribuzione ottimale q_j^* nel mean-field.

Con $q(\mathbf{z}) = \prod_j q_j(z_j)$, si isola la dipendenza da q_j nell'ELBO:

$$\mathcal{L}(q) = \int q_j \underbrace{\left[\mathbb{E}_{-j} [\log p(\mathbf{x}, \mathbf{z})] \right]}_{\text{chiamiamolo } \log \tilde{p}(\mathbf{x}, z_j)} dz_j - \int q_j \log q_j dz_j + \text{cost},$$

dove $\mathbb{E}_{-j}$ è l'attesa rispetto a tutti i fattori tranne il j -esimo. I due termini formano $-D_{\text{KL}}(q_j \| \tilde{p})$ a meno di costanti, e la KL è minimizzata quando i due argomenti coincidono:

$$\log q_j^*(z_j) = \mathbb{E}_{q_{-j}} [\log p(\mathbf{x}, \mathbf{z})] + \text{cost}.$$

Due letture. Nel calcolo pratico basta tenere i termini della log-congiunta che *contengono* z_j : tutto il resto è la costante di normalizzazione. E se il modello è coniugato, q_j^* risulta della stessa famiglia del prior, quindi l'aggiornamento si riduce a formule per i parametri.

Le equazioni sono *accoppiate*, perché q_j^* dipende dalle q_{-j} : da qui la necessità di iterare, cioè CAVI.

D4. Spiegare l'algoritmo CAVI. Qual è la complessità?

Coordinate Ascent Variational Inference: si inizializzano i q_j , poi si cicla aggiornando un fattore alla volta con la formula della D3, tenendo fissi gli altri, e si monitora l'ELBO fino a convergenza.

Proprietà: ogni aggiornamento è la soluzione *esatta* del sottoproblema in q_j , quindi l'ELBO cresce monotonamente a ogni passo; essendo superiormente limitato da $\log p(\mathbf{x})$, la successione converge. Converge a un ottimo *locale*, e la superficie dell'ELBO ha tipicamente molti massimi locali: l'inizializzazione conta, e si usano più restart.

Complessità: $\mathcal{O}(\text{iterazioni} \times m \times c)$ con m numero di fattori e c il costo di un singolo aggiornamento, che nei modelli coniugati è quello di calcolare attese di statistiche sufficienti. Il criterio di stop è la variazione relativa dell'ELBO.

Limite pratico: CAVI passa su *tutto* il dataset a ogni iterazione, quindi non scala. Le risposte sono SVI (stochastic VI, aggiornamenti su mini-batch con gradiente naturale) e l'inferenza ammortizzata dei VAE.

D5. Come scegliamo la famiglia variazionale Q ? Che tradeoff c'è tra flessibilità e trattabilità?

Q va scelta abbastanza ricca da avvicinare il posterior e abbastanza semplice da rendere calcolabili le attese dell'ELBO. Sono requisiti opposti, ed è tutto il compromesso.

Il *mean field* $q(\mathbf{z}) = \prod_j q_j(z_j)$ è l'estremo trattabile: aggiornamenti in forma chiusa nei modelli coniugati, ma per costruzione non può rappresentare correlazioni a posteriori. Se il posterior vero le ha, resta un gap incompressibile e — poiché si minimizza $D_{\text{KL}}(q \| p)$, che è *mode-seeking* — l'approssimazione tipicamente si stringe su una moda e **sottostima la varianza**. È il difetto sistematico da nominare: le incertezze prodotte dalla VI mean-field sono troppo piccole.

Alternative in ordine di flessibilità: structured mean field (si tengono alcune dipendenze, per esempio la catena in un HMM), famiglie parametriche più ricche (gaussiana a covarianza piena), normalizing flow come q , e inferenza ammortizzata con reti.

Il criterio pratico: si aumenta la flessibilità dove ci si aspetta che il posterior sia fortemente correlato, e si accetta il mean field dove le latenti sono quasi indipendenti.

Capitolo 11 — Latent Dirichlet Allocation

D1. *Descrivere il processo generativo di LDA. Come si relaziona ai dati osservati?*

Per ogni topic k : $\beta_k \sim \text{Dir}(\eta)$, distribuzione sulle parole del vocabolario. Per ogni documento d : $\theta_d \sim \text{Dir}(\alpha)$, distribuzione sui topic; poi, per ogni posizione n : $z_{dn} \sim \text{Cat}(\theta_d)$ e $w_{dn} \sim \text{Cat}(\beta_{z_{dn}})$.

L'unica cosa osservata sono le parole w_{dn} . Tutto il resto — θ , z , β — è latente, e l'inferenza consiste nell'invertire il processo: dato il testo, ricostruire quali miscele di topic e quali assegnazioni lo rendono probabile.

Due osservazioni che qualificano la risposta. Il modello è *bag of words*: l'ordine delle parole non compare da nessuna parte, quindi sintassi e negazione sono perse. E la scelta di mettere un prior su θ invece di trattarlo come parametro è precisamente ciò che rende il modello generativo a livello di corpus — è la differenza con pLSA.

D2. *Derivare l'ELBO per LDA sotto mean-field approximation.*

Famiglia variazionale fattorizzata:

$$q(\theta, z, \beta) = \prod_k q(\beta_k | \lambda_k) \prod_d \left[q(\theta_d | \gamma_d) \prod_n q(z_{dn} | \phi_{dn}) \right],$$

con $q(\beta_k)$ e $q(\theta_d)$ Dirichlet e $q(z_{dn})$ categorica. L'ELBO è

$$\mathcal{L} = \mathbb{E}_q[\log p(\mathbf{w}, \mathbf{z}, \theta, \beta)] - \mathbb{E}_q[\log q],$$

che espandendo la congiunta diventa

$$\mathcal{L} = \sum_{d,n} \mathbb{E}_q[\log p(w_{dn} | z_{dn}, \beta)] + \sum_{d,n} \mathbb{E}_q[\log p(z_{dn} | \theta_d)] + \sum_d \mathbb{E}_q[\log p(\theta_d | \alpha)] + \sum_k \mathbb{E}_q[\log p(\beta_k | \eta)] + \mathbb{H}[q].$$

Il punto tecnico che rende tutto calcolabile: per la Dirichlet vale

$$\mathbb{E}_q[\log \theta_{dk}] = \Psi(\gamma_{dk}) - \Psi\left(\sum_j \gamma_{dj}\right),$$

con Ψ la funzione digamma. È da lì che compaiono le digamma nelle formule di LDA, e saperlo dire mostra che si è seguita la derivazione e non solo il risultato.

D3. *Spiegare come funziona il CAVI per document-topic inference in LDA.*

Si alternano due aggiornamenti a livello di documento (E-step locale) e uno a livello di corpus (M-step globale).

$$\phi_{dnk} \propto \exp\left(\mathbb{E}_q[\log \theta_{dk}] + \mathbb{E}_q[\log \beta_{k,w_{dn}}]\right), \quad \gamma_{dk} = \alpha_k + \sum_n \phi_{dnk},$$

e a livello globale $\lambda_{kv} = \eta + \sum_{d,n} \phi_{dnk} \mathbb{I}[w_{dn} = v]$.

Si iterano ϕ e γ fino a convergenza per il singolo documento, poi si aggiorna λ . La lettura di ϕ_{dnk} è la stessa del collapsed Gibbs: un token è attratto da un topic se il documento già ne parla e se il topic ama quella parola. La differenza è che qui si mantengono distribuzioni (soft) invece di campionare un'assegnazione.

Complessità per iterazione: $\mathcal{O}(\sum_d N_d \cdot K)$. La versione stocastica (SVI) sostituisce la somma su tutti i documenti con un mini-batch e rende il metodo applicabile a corpora enormi.

D4. *Qual è il ruolo del prior Dirichlet su θ_d ? Come influenza learning?*

Controlla la *sparsità* della miscela di topic per documento, cioè quanto un documento può “parlare di tutto”.

$\alpha < 1$ concentra la massa vicino ai vertici del simpleso: ogni documento parla di uno o due topic. È di solito ciò che si vuole, perché produce topic distinti e interpretabili. $\alpha > 1$ concentra al centro: ogni documento è una miscela quasi uniforme e i topic si sovrappongono fino a diventare indistinguibili. $\alpha = 1$ dà l'uniforme sul simpleso.

Sul learning agisce come regolarizzatore via pseudo-conteggi: $\gamma_{dk} = \alpha_k + \sum_n \phi_{dnk}$, quindi α impedisce che un topic mai usato in un documento riceva probabilità esattamente zero, e stabilizza i documenti corti (dove i conteggi sono pochi e il prior pesa di più).

Da nominare anche: α può essere asimmetrico, e stimarlo dai dati anziché fissarlo migliora sensibilmente i risultati; il prior η su β svolge il ruolo analogo sulle distribuzioni parola-topic.

D5. Confrontare LDA e pLSA. Quali sono i vantaggi bayesiani?

In pLSA le θ_d sono *parametri liberi*, uno per documento. In LDA sono *variabili latenti* estratte da un prior Dirichlet condiviso.

Da questa differenza discende tutto il resto. Il numero di parametri di pLSA cresce con il corpus ($\mathcal{O}(DK + KV)$), quindi il modello sovra-adatta e ha bisogno di tempering per essere usabile; in LDA i parametri restano α e β , e la complessità non cresce con D .

Soprattutto, pLSA **non è un modello generativo per documenti nuovi**: non c'è modo di ottenere il θ di un documento mai visto, se non rieseguendo l'inferenza con i parametri degli altri congelati (folding-in), che è una procedura ad hoc e senza fondamento nel modello. LDA lo fa nativamente, perché θ ha un prior da cui si può inferire.

Infine, il prior regolarizza e permette di controllare la sparsità. Il collegamento generale da fare: è la stessa mossa che distingue la stima puntuale dall'approccio bayesiano nel Cap. 7 — invece di fissare un parametro, gli si mette sopra una distribuzione e la si integra.

Capitolo 12 — Metodi di campionamento

D1. Spiegare il principio di importance sampling e quando è utile.

Si vuole $\mathbb{E}_p[f]$ senza saper campionare da p . Si campiona da una proposta q e si correggono i pesi:

$$\mathbb{E}_p[f(x)] = \int f(x) \frac{p(x)}{q(x)} q(x) dx = \mathbb{E}_q[f(x)w(x)], \quad w(x) = \frac{p(x)}{q(x)}.$$

Se p è nota solo a meno di normalizzazione si usa la versione *self-normalized*, $\hat{\mathbb{E}} = \sum_i \tilde{w}_i f(x_i) / \sum_i \tilde{w}_i$, che è consistente ma leggermente distorta.

Requisito di validità: $q(x) > 0$ ovunque $p(x)f(x) \neq 0$. Se la proposta ha code più leggere del target, i pesi hanno varianza infinita e la stima è inaffidabile pur restando formalmente corretta.

È utile quando si vuole riusare gli stessi campioni per più target, quando si vogliono stimare eventi rari (scegliendo q che li visita spesso), e come componente di algoritmi più grandi (particle filter, IWAE). La diagnostica minima è la *effective sample size* $\text{ESS} = (\sum w_i)^2 / \sum w_i^2$: nell'esempio del Cap. 12, tre campioni danno $\text{ESS} \approx 1.85$, cioè uno domina.

D2. Derivare il Gibbs sampler da Metropolis-Hastings.

Si prenda MH con proposta “aggiorna la sola coordinata i dalla sua condizionale completa”:

$$q(\mathbf{x}' | \mathbf{x}) = p(x'_i | \mathbf{x}_{-i}), \quad \mathbf{x}'_{-i} = \mathbf{x}_{-i}.$$

Il rapporto di accettazione è

$$A = \min \left(1, \frac{p(\mathbf{x}')q(\mathbf{x} | \mathbf{x}')}{p(\mathbf{x})q(\mathbf{x}' | \mathbf{x})} \right) = \min \left(1, \frac{p(x'_i | \mathbf{x}_{-i})p(\mathbf{x}_{-i}) p(x_i | \mathbf{x}_{-i})}{p(x_i | \mathbf{x}_{-i})p(\mathbf{x}_{-i}) p(x'_i | \mathbf{x}_{-i})} \right) = \min(1, 1) = 1.$$

Tutti i fattori si cancellano, quindi **Gibbs è MH con accettazione sempre pari a 1**. È un caso particolare, non un algoritmo diverso.

Le conseguenze da nominare: non si spreca mai un campione (a differenza di rejection sampling), ma non c'è nemmeno un parametro di step size da tarare, quindi non si può controllare quanto la catena si muove. Se le variabili sono fortemente correlate, Gibbs si muove per piccoli passi lungo gli assi e mescola lentissimamente — il rimedio è il *blocked* Gibbs, che aggiorna insieme gruppi di variabili correlate. E serve saper campionare dalle condizionali complete, il che vincola la scelta del modello (coniugatezza) o costringe a un passo MH innestato.

D3. *Quale è la complessità computazionale di rejection sampling in alta dimensione?*

Si campiona $x \sim q$ e si accetta con probabilità $\tilde{p}(x)/(Mq(x))$, dove M è tale che $Mq(x) \geq \tilde{p}(x)$ ovunque. Il tasso di accettazione è $1/M$ (per densità normalizzate), quindi il numero atteso di proposte per un campione accettato è M .

Il problema è che M cresce tipicamente in modo *esponenziale* nella dimensione. Esempio classico: $p = \mathcal{N}(0, \sigma_p^2 \mathbf{I})$ e $q = \mathcal{N}(0, \sigma_q^2 \mathbf{I})$ in d dimensioni con $\sigma_q > \sigma_p$; il rapporto ottimo è $M = (\sigma_q/\sigma_p)^d$. Con $\sigma_q/\sigma_p = 1.01$ e $d = 1000$ si ha già $M \approx 21\,000$.

La ragione profonda è la concentrazione della misura: in alta dimensione la massa di probabilità sta in un guscio sottile, e due gussi anche leggermente diversi hanno sovrapposizione trascurabile. Nessuna proposta “ragionevolmente vicina” basta.

È il motivo per cui rejection sampling è utile solo in poche dimensioni, e per cui MCMC — che non richiede di indovinare globalmente il target ma esplora localmente — è l'unica opzione praticabile in alta dimensione.

D4. *Come si determina il burn-in period? Quali diagnostici di convergenza si possono usare?*

Il burn-in è il tratto iniziale della catena, ancora dipendente dall'inizializzazione, che si scarta. Non esiste un criterio esatto — la convergenza di una catena di Markov si può solo diagnosticare, mai dimostrare dai campioni — quindi si usano più indicatori concordanti.

Trace plot: si osserva la traiettoria dei parametri; un tratto iniziale con deriva sistematica seguito da oscillazione stazionaria indica dove tagliare.

$\hat{R}$ di *Gelman-Rubin*: si lanciano più catene da inizializzazioni diverse (sovradisperse) e si confronta la varianza *fra* catene con quella *entro* catena. $\hat{R} \rightarrow 1$ indica che le catene hanno dimenticato l'inizializzazione; la soglia d'uso corrente è $\hat{R} < 1.01$.

Autocorrelazione ed ESS: l'autocorrelazione a lag crescente indica quanto la catena mescola; l'ESS stima quanti campioni indipendenti equivalenti si hanno. Un ESS molto minore del numero di iterazioni segnala mixing lento.

Due note pratiche: il *thinning* (tenere un campione ogni k) riduce l'autocorrelazione ma butta informazione, e in generale non conviene se non per problemi di memoria; e nessuna diagnostica rileva una moda che la catena non ha mai visitato.

D5. *Derivare il collapsed Gibbs sampler per LDA.*

“Collapsed” significa che θ e β vengono *integrati analiticamente* sfruttando la coniugazione Dirichlet-Categorica, e si campionano solo le assegnazioni discrete z_{dn} . Integrando si ottengono rapporti di funzioni Gamma che si semplificano in rapporti di conteggi, e la condizionale completa risulta

$$P(z_{dn} = k \mid \mathbf{z}_{-dn}, \mathbf{w}) \propto \underbrace{(n_{d,k}^{-dn} + \alpha_k)}_{\text{quanto } d \text{ ama } k} \cdot \underbrace{\frac{n_{k,w_{dn}}^{-dn} + \eta}{n_{k,\cdot}^{-dn} + V\eta}}_{\text{quanto } k \text{ ama la parola}},$$

dove l'apice $-dn$ indica che il token corrente è escluso dai conteggi.

Procedura: si rimuove il token dai conteggi, si valuta la formula per ogni k , si normalizza, si campiona, si reinseriscono i conteggi aggiornati.

Perché conviene rispetto al Gibbs non collassato: lo spazio di campionamento è molto più piccolo (solo le z , non i semplici continui), la catena è meno autocorrelata, e la varianza dell'estimatore è ridotta — è un'istanza di Rao-Blackwellizzazione. Il prezzo è che gli aggiornamenti non sono più parallelizzabili in modo esatto, perché i conteggi globali cambiano a ogni token.

Esempio numerico nel Cap. 11: con $\alpha = 0.5$, $\eta = 0.1$, $V = 4$ e i conteggi dati, si ottiene $P(z = 1) = 0.793$ contro $P(z = 2) = 0.207$.

Capitolo 13 — Campi casuali Markoviani

D1. Enunciare il Teorema di Hammersley-Clifford. Perché è importante?

Per una distribuzione *strettamente positiva* $P(\mathbf{x}) > 0$, P soddisfa le proprietà di Markov rispetto al grafo non orientato G se e solo se fattorizza sulle clique massimali:

$$P(\mathbf{x}) = \frac{1}{Z} \prod_{c \in \mathcal{C}} \psi_c(\mathbf{x}_c), \quad Z = \sum_{\mathbf{x}} \prod_c \psi_c(\mathbf{x}_c).$$

È importante perché stabilisce l'equivalenza fra una proprietà *semantica* (le indipendenze condizionali leggibili dal grafo) e una *sintattica* (la forma del prodotto). È l'analogo, per i modelli non orientati, del teorema di fattorizzazione delle reti Bayesiane, e giustifica il fatto che si possa progettare un modello scegliendo potenziali sulle clique e leggerne le indipendenze dal grafo.

L'ipotesi di positività stretta non è tecnica: esistono controesempi (il classico è dovuto a Moussouris) con distribuzioni che hanno zeri, soddisfano le proprietà di Markov e non ammettono quella fattorizzazione. Va nominata.

Conseguenza operativa: si scrive $\psi_c = \exp(-E_c)$ e si ottiene la forma di Gibbs $P(\mathbf{x}) = \exp(-\sum_c E_c(\mathbf{x}_c))/Z$, che è il ponte con la meccanica statistica e con i modelli energy-based.

D2. Come si relazionano MRF e modelli grafici diretti? Quali indipendenze preservano?

Nessuna delle due classi contiene l'altra: esistono indipendenze rappresentabili solo da BN e altre solo da MRF.

Il caso che le BN catturano e gli MRF no è la **v-structure** $X \rightarrow Z \leftarrow Y$, che implica $X \perp Y$ ma $X \not\perp Y \mid Z$. Nessun grafo non orientato può codificare questo schema: se X e Y non sono adiacenti, condizionare su Z non può creare dipendenza.

Il caso opposto è il ciclo a quattro nodi $A - B - C - D - A$ senza corde, che implica $A \perp C \mid \{B, D\}$ e $B \perp D \mid \{A, C\}$ simultaneamente: nessun DAG le rappresenta entrambe senza aggiungerne altre.

Conversione: da BN a MRF si fa la *moralizzazione* — si sposano i genitori con lo stesso figlio e si rimuovono gli orientamenti. È una conversione con perdita, perché sposando i genitori si perde proprio l'informazione della v-structure. Ed è il primo passo della costruzione del junction tree, quindi non è un esercizio teorico.

D3. Derivare l'algoritmo di aggiornamento per Gibbs sampling in un RBM.

Energia e congiunta:

$$E(\mathbf{v}, \mathbf{h}) = -\mathbf{v}^\top \mathbf{W} \mathbf{h} - \mathbf{b}^\top \mathbf{v} - \mathbf{c}^\top \mathbf{h}, \quad P(\mathbf{v}, \mathbf{h}) = \frac{e^{-E}}{Z}.$$

Per la condizionale di una singola unità nascosta si isolano i termini che la contengono:

$$P(h_j = 1 | \mathbf{v}) = \frac{\exp(c_j + \sum_i v_i W_{ij})}{1 + \exp(c_j + \sum_i v_i W_{ij})} = \sigma\left(c_j + \sum_i v_i W_{ij}\right),$$

e simmetricamente $P(v_i = 1 | \mathbf{h}) = \sigma(b_i + \sum_j W_{ij} h_j)$.

Il fatto decisivo è che la struttura *bipartita* rende le unità di uno strato condizionalmente indipendenti dato l'altro:

$$P(\mathbf{h} | \mathbf{v}) = \prod_j P(h_j | \mathbf{v}), \quad P(\mathbf{v} | \mathbf{h}) = \prod_i P(v_i | \mathbf{h}).$$

Quindi si può fare *block Gibbs*: si campiona l'intero strato nascosto in parallelo, poi l'intero strato visibile, alternando. È molto più efficiente del Gibbs una-variabile-alla-volta, e in una Boltzmann machine generale (non ristretta) non sarebbe possibile — è tutta la ragione della restrizione.

D4. *Spiegare l'algoritmo Contrastive Divergence. Perché è un'approssimazione del gradiente?*

Il gradiente esatto è $\partial \ell / \partial W_{ij} = \langle v_i h_j \rangle_{\text{data}} - \langle v_i h_j \rangle_{\text{model}}$. Il primo termine è una media empirica; il secondo richiede campioni dalla distribuzione del modello, quindi una catena di Gibbs portata a convergenza, quindi in pratica è inaccessibile.

CD- k tronca la catena: parte *dal dato* invece che da uno stato casuale, esegue k passi di Gibbs (spesso $k = 1$) e usa lo stato risultante al posto di un campione dal modello. Aggiornamento: $\Delta W \propto \langle v h \rangle^{(0)} - \langle v h \rangle^{(k)}$, “alza il dato, abbassa il sogno”.

È un'approssimazione *distorta* per due ragioni: k passi non bastano a raggiungere la stazionaria, e partire dal dato tiene la catena in una regione già favorevole al dato stesso, sottostimando sistematicamente il termine negativo. Il fatto forte da dire: **CD-1 non è il gradiente di nessuna funzione obiettivo**, quindi non c'è una quantità che venga davvero minimizzata.

Funziona in pratica perché la direzione è quasi sempre corretta anche se il modulo non lo è, e il bias è tollerabile finché la distribuzione del modello resta vicina ai dati. Rimedi: CD- k con k maggiore, e soprattutto *Persistent CD*, che mantiene la catena fra un mini-batch e l'altro invece di ripartire dal dato, avvicinandosi molto di più alla vera fase negativa. Esempio numerico nel Cap. 13.

D5. *Come si generalizzano RBM a variabili continue? Quali sono i cambiamenti?*

La variante standard è la **Gaussian-Bernoulli RBM**: visibili continue, nascoste binarie. L'energia diventa

$$E(\mathbf{v}, \mathbf{h}) = \sum_i \frac{(v_i - b_i)^2}{2\sigma_i^2} - \sum_j c_j h_j - \sum_{i,j} \frac{v_i}{\sigma_i} W_{ij} h_j,$$

e la condizionale sui visibili passa da Bernoulli a gaussiana:

$$P(v_i | \mathbf{h}) = \mathcal{N}\left(b_i + \sigma_i \sum_j W_{ij} h_j, \sigma_i^2\right),$$

mentre $P(h_j = 1 | \mathbf{v}) = \sigma(c_j + \sum_i W_{ij} v_i / \sigma_i)$ resta logistica. Il termine quadratico è ciò che rende la distribuzione normalizzabile sui reali: senza, l'energia sarebbe lineare in v_i e l'integrale divergerebbe.

I cambiamenti pratici sono più fastidiosi della matematica. I dati vanno standardizzati; le σ_i sono difficili da apprendere insieme ai pesi (spesso si fissano a 1 dopo la normalizzazione); il training è più instabile e richiede learning rate molto più piccoli, perché il gradiente non è più limitato come nel caso binario.

Altre varianti da citare: ReLU/rectified units (Nair e Hinton), spike-and-slab RBM, e la mean-covariance RBM per modellare anche le correlazioni. Va detto per onestà storica che questa famiglia è stata largamente superata da VAE, flow e diffusion per la modellazione di dati continui.

Chapter 3

Parte III — Deep learning fondamentale

Capitolo 14 — Convolutional Neural Networks

D1. *Spiegare il concetto di parameter sharing nelle CNN e come contribuisce all'equivarianza traslazionale.*

Parameter sharing significa che lo *stesso* kernel viene applicato in ogni posizione spaziale, invece di avere pesi diversi per ogni punto come farebbe una fully connected. Il numero di parametri diventa indipendente dalla dimensione dell'input: un layer con 6 filtri 5×5 su input a 3 canali costa 456 parametri sia su 32×32 sia su 256×256 .

L'equivarianza segue direttamente: se lo stesso rilevatore agisce ovunque, traslando l'input la risposta si trasla identicamente, $f(T_v \mathbf{x}) = T_v f(\mathbf{x})$. Con pesi non condivisi non varrebbe, perché ogni posizione applicherebbe un rilevatore diverso.

Attenzione a non dire “invarianza”: la convoluzione è equivariante, non invariante. L'informazione sulla posizione non è persa, è trasportata — ed è per questo che una CNN serve anche per segmentazione e detection. L'invarianza la introducono il pooling (localmente) e il global pooling finale (globalmente): è una scelta architetturale successiva.

La giustificazione del vincolo è statistica: un bordo è un bordo ovunque si trovi, quindi la statistica dell'immagine è approssimativamente stazionaria per traslazione. Dove questo non vale (dati tabulari, feature non ordinate) il bias diventa un handicap.

D2. *Qual è la differenza tra convoluzione e cross-correlazione? Perché nelle reti neurali si implementa la cross-correlazione?*

$$(I * K)[i, j] = \sum_{m, n} I[i - m, j - n] K[m, n], \quad (I \star K)[i, j] = \sum_{m, n} I[i + m, j + n] K[m, n].$$

La convoluzione *ribalta* il kernel di 180° , la cross-correlazione no.

Nelle reti si implementa la cross-correlazione per un motivo semplice: il kernel è *appreso*, quindi ribaltarlo o no non cambia l'insieme delle funzioni rappresentabili — la rete impara direttamente il kernel già ribaltato se le serve. Il ribaltamento sarebbe puro costo computazionale senza guadagno espressivo.

La differenza conta invece quando il kernel è *dato* e non appreso, per esempio nel signal processing classico, dove la convoluzione ha la proprietà di commutatività e il teorema di convoluzione ($\mathcal{F}\{f * g\} = \mathcal{F}\{f\} \cdot \mathcal{F}\{g\}$), che la cross-correlazione non ha. Nelle librerie di deep

learning la funzione si chiama comunque **conv** per convenzione storica: è una imprecisione di nomenclatura consolidata.

D3. Derivare la formula per le dimensioni dell'output di una convoluzione: $O = \frac{I-K+2P}{S} + 1$.

Con padding P l'input effettivo ha lunghezza $I + 2P$. Il kernel di lunghezza K deve stare interamente dentro, quindi la sua posizione iniziale può andare da 0 a $(I + 2P) - K$: sono $(I + 2P - K) + 1$ posizioni con stride 1.

Con stride S si prendono solo le posizioni $0, S, 2S, \dots$, quindi il numero di posizioni valide è

$$O = \left\lfloor \frac{I + 2P - K}{S} \right\rfloor + 1.$$

Il “+1” conta la posizione iniziale; il floor serve perché con stride non divisore l'ultima finestra parziale viene scartata.

Casi utili da avere pronti: *same padding* con $S = 1$ richiede $P = (K - 1)/2$, che è intero solo per K dispari — ed è la ragione per cui i kernel sono quasi sempre dispari. Con $K = 3, P = 1, S = 1$ la dimensione si conserva; con $K = 3, P = 1, S = 2$ si dimezza. Il conto vale identico per ogni dimensione spaziale; la profondità dell'output è invece pari al numero di filtri, non a quello dei canali di input.

D4. Descrivere il *degradation problem* in reti molto profonde e come ResNet lo risolve.

Il degradation problem (He et al. 2015) è l'osservazione che aggiungendo layer l'errore di *training* peggiora, non solo quello di test. Non è overfitting — quello darebbe training migliore e test peggiore — ed è un fatto sorprendente perché una rete più profonda può sempre rappresentare quella meno profonda, aggiungendo layer identità.

Il problema è quindi di *ottimizzazione*: la rete non riesce a imparare la mappa identità partendo da pesi casuali. Approssimare l'identità con una pila di trasformazioni non lineari è, sorprendentemente, difficile.

ResNet cambia la parametrizzazione: invece di imparare $\mathcal{H}(\mathbf{x})$, il blocco impara il *residuo* $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$, e calcola $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$. Ora l'identità corrisponde a $\mathcal{F} = 0$, cioè a pesi nulli: è il punto più facile da raggiungere, non il più difficile. Se un layer non serve, la rete può disattivarlo senza sforzo.

Risultato: si è passati da reti di 20 layer a 152 e oltre, con miglioramenti monotoni.

D5. Spiegare il ruolo delle *skip connections* nel migliorare il flusso dei gradienti durante la *backpropagation*.

Con $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$, la derivata rispetto all'input è

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathcal{F}}{\partial \mathbf{x}} + \mathbf{I}.$$

Il termine $+\mathbf{I}$ è decisivo: attraverso L blocchi il gradiente si propaga come $\prod_{\ell} (\mathbf{I} + \partial \mathcal{F}_{\ell} / \partial \mathbf{x})$, che espanso contiene il termine $\mathbf{I}$ puro. Esiste quindi sempre un cammino lungo cui il gradiente arriva *non attenuato*, indipendentemente da quanto valgono i Jacobiani dei blocchi.

Senza skip, il gradiente è un prodotto puro di Jacobiani e svanisce (o esplode) esponenzialmente con la profondità — è lo stesso meccanismo del vanishing gradient nelle RNN, con la profondità al posto del tempo.

Il collegamento trasversale da fare: è la stessa idea del cell state additivo dell'LSTM ($\partial c_t / \partial c_{t-1} = f_t$) e delle connessioni residue dentro il blocco Transformer. Un cammino additivo che protegge il gradiente è un motivo architetturale ricorrente in tutto il corso, e riconoscerlo in tre contesti diversi è il tipo di risposta che vale la terza domanda.

Effetto collaterale documentato: le skip connection rendono la superficie di loss visibilmente più liscia, il che è un secondo motivo per cui l'ottimizzazione migliora.

D6. *Quale architettura tra LeNet, VGG, ResNet sarebbe più adatta per un'immagine 224×224 RGB? Giustificare.*

ResNet, senza esitazione, e la giustificazione va data in tre punti.

LeNet è fuori scala: progettata per 32×32 in scala di grigi, ha cinque layer e poche decine di migliaia di parametri. Su 224×224 RGB il suo campo recettivo non copre l'immagine e la capacità è insufficiente per la varietà di ImageNet.

VGG funzionerebbe — è nata proprio su 224×224 — ma è inefficiente: 138 milioni di parametri, di cui la grande maggioranza nei layer fully connected finali, e nessun meccanismo contro il degradation. Oltre i 19 layer non scala.

ResNet è la scelta corretta: le connessioni residue permettono profondità elevata senza degradazione, il global average pooling elimina i fully connected pesanti (ResNet-50 ha ~ 25 M parametri, cinque volte meno di VGG-16 pur essendo più profonda e più accurata), e i blocchi bottleneck $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ tengono basso il costo.

Precisazione onesta: se il vincolo fosse un dispositivo embedded, la risposta cambierebbe verso MobileNet o EfficientNet; se il dataset fosse piccolo, verso una ResNet pre-addestrata e fine-tuned. La domanda giusta è sempre “con quale budget e quanti dati”.

D7. *Quali sono i vantaggi computazionali delle convoluzioni depthwise separabili?*

Una convoluzione standard $D_K \times D_K$ da M a N canali su una mappa $D_F \times D_F$ costa $D_K^2 \cdot M \cdot N \cdot D_F^2$ moltiplicazioni. La versione separabile la spezza in due:

- *depthwise*: una convoluzione spaziale $D_K \times D_K$ per canale, senza mescolarli, costo $D_K^2 \cdot M \cdot D_F^2$;
- *pointwise*: una 1×1 che ricombina i canali, costo $M \cdot N \cdot D_F^2$.

Il rapporto è

$$\frac{D_K^2 M D_F^2 + M N D_F^2}{D_K^2 M N D_F^2} = \frac{1}{N} + \frac{1}{D_K^2}.$$

Con $D_K = 3$ e $N = 256$: $1/256 + 1/9 \approx 0.115$, cioè circa **8,7 volte** meno calcolo e meno parametri. Per N grande il termine dominante è $1/D_K^2$, quindi il guadagno tende a 9 per kernel 3×3 .

L'idea concettuale: si fattorizza un'operazione che fa contemporaneamente filtraggio spaziale e mixing dei canali in due operazioni che ne fanno una ciascuna. Il costo è una lieve perdita di espressività, compensata in pratica aumentando la larghezza. È la base di MobileNet e Xception.

D8. *Come si calcola il receptive field di un neurone dopo 5 layer convoluzionali?*

Ricorrenza:

$$\text{RF}_\ell = \text{RF}_{\ell-1} + (k_\ell - 1) \cdot j_{\ell-1}, \quad j_\ell = j_{\ell-1} \cdot s_\ell,$$

con $\text{RF}_0 = 1$ e $j_0 = 1$. Il *jump* j_ℓ è il passo effettivo cumulato: dice di quanto si sposta la finestra nell'input originale quando ci si sposta di uno nella mappa al layer ℓ .

Cinque conv 3×3 con stride 1: $1 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow 9 \rightarrow 11$. La crescita è lineare, $\text{RF} = 1 + L(k - 1)$.

Con un pooling 2×2 dopo il secondo layer (conv3, conv3, pool2, conv3, conv3): $1 \rightarrow 3 \rightarrow 5 \rightarrow 6$ ($j=2$) $\rightarrow 10 \rightarrow 14$. Il downsampling raddoppia il jump e da lì in poi ogni layer contribuisce il doppio: è il downsampling a far crescere in fretta il campo recettivo.

Per kernel dilatati si sostituisce k con il kernel effettivo $k_{\text{eff}} = d(k - 1) + 1$, che è il modo per allargare l'RF senza pagare né in risoluzione né in parametri. E vale la pena distinguere RF *teorico* da RF *effettivo*: sperimentalmente il contributo dei pixel decade in modo quasi gaussiano dal centro, quindi la regione che conta davvero è più piccola di quella nominale.

Capitolo 15 — Propagazione dell'informazione (RNN, EVGP)

D1. Scrivere la formula per $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-\tau}}$ in una RNN vanilla e analizzare quando il gradiente svanisce.

Con $\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$, per la chain rule

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-\tau}} = \prod_{i=t-\tau+1}^t \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}} = \prod_i \mathbf{W}_h^\top \text{diag}(1 - \tanh^2(\mathbf{a}_i)),$$

dove $\mathbf{a}_i$ è la pre-attivazione. È un *prodotto* di τ Jacobiani, ed è tutto lì il problema.

Prendendo le norme, $\|\partial \mathbf{h}_t / \partial \mathbf{h}_{t-\tau}\| \leq (\|\mathbf{W}_h\| \cdot \gamma)^\tau$ con $\gamma = \max |\tanh'| = 1$. Quindi:

$$\rho(\mathbf{W}_h) < 1 \Rightarrow \text{vanishing}, \quad \rho(\mathbf{W}_h) > 1 \Rightarrow \text{possibile exploding}.$$

Poiché $\tanh'(\cdot) \leq 1$ *sempre*, e vale 1 solo esattamente in zero, il fattore diagonale può solo peggiorare il vanishing: serve $\rho(\mathbf{W}_h) > 1$ anche solo per sperare di conservare il segnale, ma allora la dinamica in avanti tende a saturare o divergere. Il problema è strutturale, non un difetto di ottimizzazione.

Con sigmoid è ancora peggio, perché $\sigma' \leq 1/4$.

D2. Qual è la differenza tra *vanishing* e *exploding gradients*? Quali condizioni su $\mathbf{W}_h$ le causano?

Stessa causa (il prodotto di Jacobiani), due regimi opposti governati dal raggio spettrale $\rho(\mathbf{W}_h)$, cioè dal modulo dell'autovalore più grande.

Vanishing ($\rho < 1$): il gradiente a lungo raggio decade esponenzialmente. La rete addestra, la loss scende, ma le dipendenze lunghe non vengono mai apprese: il modello impara solo il contesto recente. È insidioso perché **non dà nessun sintomo** — non c'è un errore, solo una capacità mancante.

Exploding ($\rho > 1$): il gradiente cresce esponenzialmente, i pesi ricevono aggiornamenti enormi, la loss diverge o va a NaN. È molto più visibile e molto più facile da curare.

I numeri rendono l'idea. Su 50 passi (w^{49}): $w = 0.9$ dà 5.7×10^{-3} , $w = 1.0$ dà 1, $w = 1.1$ dà 107. Fra 0.9 e 1.1 ci sono diciotto ordini di grandezza, e il caso $\rho = 1$ è un punto di misura nulla.

Rimedi, in ordine: clipping per l'exploding; architetture gated (LSTM, GRU), skip connection, inizializzazione ortogonale e normalizzazioni per il vanishing.

D3. Spiegare il meccanismo del *gradient clipping* e quando è più efficace.

Si riscalda il gradiente quando la sua norma supera una soglia, prima dell'aggiornamento:

$$\text{se } \|\mathbf{g}\| > \theta : \quad \mathbf{g} \leftarrow \theta \frac{\mathbf{g}}{\|\mathbf{g}\|}.$$

Questa è la versione *by norm*, che è quella corretta: preserva la *direzione* del gradiente e ne limita solo il modulo. La variante *by value*, che taglia componente per componente, distorce la direzione ed è generalmente peggiore.

È efficace contro l'exploding gradient, e la ragione geometrica è nota: la superficie di loss delle RNN presenta “pareti” — regioni a curvatura altissima — dove un passo di gradiente non clippato catapulta i parametri lontanissimo, distruggendo il lavoro fatto. Il clipping fa scavalcare la parete con un passo di lunghezza controllata.

Due precisazioni. Non serve a nulla contro il vanishing: riscaldare un gradiente già minuscolo non lo rende informativo. E introduce un bias — la direzione di aggiornamento non è più il

gradiente vero — che in pratica si accetta volentieri. Valori tipici della soglia sono fra 1 e 5; una scelta ragionevole è la mediana delle norme osservate nelle prime iterazioni.

D4. *Perché l'inizializzazione ortogonale dei pesi ricorrenti aiuta il gradient flow?*

Una matrice ortogonale ha tutti i valori singolari pari a 1, quindi preserva le norme: $\|Qv\| = \|v\|$. Il prodotto di matrici ortogonali resta ortogonale, quindi il fattore $W_h^\top$ ripetuto τ volte non amplifica né attenua — si parte esattamente sul confine fra vanishing ed exploding, $\rho = 1$.

È il punto di partenza ottimale perché, come visto, la finestra utile fra 0.9 e 1.1 è strettissima: partire a caso significa quasi certamente partire dalla parte sbagliata.

Due limiti da dichiarare. L'ortogonalità vale *all'inizializzazione*: durante il training i pesi si muovono e la proprietà si perde, a meno di imporla esplicitamente (parametrizzazioni unitarie tipo uRNN, o penalità soft). E resta comunque il fattore $\text{diag}(1 - \tanh^2)$, che è sempre ≤ 1 e attenua anche con W ortogonale.

Varianti citabili: l'inizializzazione IRNN (identità più ReLU) di Le, Jaitly e Hinton, che è la stessa idea nella forma più estrema, e Xavier/He per i pesi non ricorrenti.

D5. *Descrivere l'Echo State Property e il ruolo della spectral radius.*

L'Echo State Property afferma che lo stato del reservoir dipende *asintoticamente* solo dalla storia dell'input e non dalle condizioni iniziali: due traiettorie partite da stati diversi, alimentate dalla stessa sequenza, convergono. È la condizione che rende il reservoir un'utile memoria dell'input invece di un sistema con dinamica propria arbitraria.

Il raggio spettrale $\rho(W)$ è il parametro di controllo. La condizione *necessaria* tipicamente usata in pratica è $\rho(W) < 1$; la condizione sufficiente rigorosa riguarda il massimo valore singolare, ed è più stringente. Nella pratica si genera W casuale sparsa e la si riscalda al ρ voluto.

Il compromesso: ρ vicino a zero dà memoria cortissima (lo stato dimentica subito), ρ vicino a 1 dà memoria lunga ma dinamica quasi instabile. Si sceglie in base alla scala temporale del task, e molti risultati empirici indicano che il valore migliore sta *appena sotto* 1 — il cosiddetto *edge of chaos*.

Il punto pratico che giustifica tutto il paradigma: nel reservoir computing W e i pesi di input restano **fissi**, e si addestra solo il readout lineare, in forma chiusa con una ridge regression. Niente BPTT, niente vanishing gradient, training in secondi. Il prezzo è meno espressività di una rete addestrata end-to-end. Bacciu è uno degli autori di riferimento su questo tema, e in particolare sull'estensione ai grafi (Parte V).

D6. *In una sequenza di 100 passi temporali con $\|W_h\| = 0.95$, quanto è ridotto il gradiente per un evento al passo 1 visto dal passo 100?*

Il gradiente attraversa $100 - 1 = 99$ moltiplicazioni, quindi il fattore è

$$0.95^{99} \approx 6.2 \times 10^{-3},$$

cioè una riduzione di circa **160 volte** rispetto al contributo locale.

L'interpretazione conta più del numero. Nell'aggiornamento dei pesi, il contributo dell'evento al passo 1 è 160 volte più piccolo di quello di un evento al passo 99: viene di fatto sommerso dai contributi recenti, e la rete ottimizza le dipendenze corte perché sono quelle che dominano il gradiente. Non si tratta di un errore numerico ma di una gerarchia di priorità imposta dall'architettura.

E questo è il caso *benevolo*. Con $\|W_h\| = 0.9$ si avrebbe $0.9^{99} \approx 2.9 \times 10^{-5}$; con 0.5, circa 10^{-30} , cioè sotto la precisione di macchina. Per confronto, un LSTM con forget gate a 0.99 conserverebbe $0.99^{99} \approx 0.37$: lo stesso orizzonte, due regimi incomparabili.

D7. Quali attivazioni sono preferibili per le RNN e perché?

Nella cella ricorrente si usa quasi sempre $\tanh$, per tre ragioni: è centrata in zero (a differenza della sigmoide, il cui output sempre positivo introduce un bias sistematico nelle attivazioni successive), è limitata in $[-1, 1]$ e quindi impedisce alla dinamica di divergere in avanti, e ha derivata massima 1 contro il 1/4 della sigmoide, quindi attenua meno il gradiente.

La *sigmoide* resta la scelta obbligata per le **porte** di LSTM e GRU, perché lì l'output deve essere interpretabile come una frazione in $[0, 1]$: quanto lasciar passare. Non è un'attivazione di stato, è un moltiplicatore.

La *ReLU* è problematica nella ricorrenza: non essendo limitata, iterarla può far divergere lo stato, e le unità morte in un ciclo ricorrente sono difficili da recuperare. Funziona solo con accorgimenti forti — inizializzazione a identità (IRNN) e clipping aggressivo.

Le *unità lineari* (nessuna nonlinearietà nel cammino di memoria) sono in realtà la soluzione vincente, ed è esattamente ciò che fanno il cell state dell'LSTM e le skip connection: si tiene un canale lineare per la memoria e si mettono le nonlinearietà fuori da quel cammino.

Capitolo 16 — Modelli ricorrenti con porte

D1. Descrivere la struttura completa di una cella LSTM e il ruolo di ciascuna porta.

$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$	forget gate
$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$	input gate
$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$	candidato
$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$	cell state
$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$	output gate
$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$	hidden state

Ruoli: il *forget* decide quanto del contenuto attuale sopravvive; l'*input* quanto della nuova informazione candidata entra; l'*output* quanto della cella viene esposto all'esterno. Le porte usano la sigmoide perché devono essere frazioni; il candidato usa $\tanh$ perché è un contenuto, non un moltiplicatore.

Due punti che distinguono la risposta buona. Primo: la separazione fra $\mathbf{c}_t$ (memoria interna) e $\mathbf{h}_t$ (ciò che si mostra) permette alla cella di *trattenere* qualcosa senza usarlo subito — è la vera differenza con una RNN semplice. Secondo: il bias del forget gate va inizializzato a un valore positivo (tipicamente 1) perché all'inizio la cella tenda a ricordare; con bias nullo $\mathbf{f}_t \approx 0.5$ e la memoria decade come 0.5^t , cioè si riparte dal problema che si voleva risolvere.

D2. Perché l'aggiornamento della cell state in LSTM è additivo e non moltiplicativo? Quale vantaggio porta?

Perché la derivata lungo un cammino additivo è controllata, quella lungo un cammino moltiplicativo no.

In una RNN vanilla $\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{h}_{t-1} + \dots)$, il gradiente a ritroso attraversa un prodotto di Jacobiani $\mathbf{W}^\top \text{diag}(\cdot)$, il cui modulo dipende dal raggio spettrale e non è governabile dalla rete.

In un LSTM $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \dots$: lungo la cella la derivata è $\mathbf{f}_t$, un numero fra 0 e 1 *scelto dalla rete*, per ogni dimensione separatamente. La rete decide dove tenere il canale aperto ($f \approx 1$) e dove chiuderlo. È il “constant error carousel” dell'articolo originale.

Il vantaggio non è quindi che il gradiente non decada mai — con $f < 1$ decade — ma che il tasso di decadimento diventa **un parametro appreso, dipendente dal contenuto e**

dalla dimensione, invece di una costante imposta dall'inizializzazione. Su 50 passi, $f = 0.95$ dà 0.081 contro i 5.7×10^{-3} di una vanilla con $w = 0.9$.

Stessa struttura, contesti diversi: skip connection nelle ResNet, connessioni residue nel Transformer.

D3. Derivare $\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}}$ in LSTM e confrontare con la RNN vanilla.

Da $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$, considerando il cammino diretto (le porte dipendono da $\mathbf{h}_{t-1}$ e quindi indirettamente da $\mathbf{c}_{t-1}$, ma quel contributo è secondario):

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \approx \text{diag}(\mathbf{f}_t) \implies \frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-\tau}} \approx \prod_{i=t-\tau+1}^t \text{diag}(\mathbf{f}_i).$$

Confronto:

	fattore per passo	governato da
RNN vanilla	$\mathbf{W}^\top \text{diag}(1 - \tanh^2)$	$\rho(\mathbf{W})$, fisso e non appreso in modo mirato
LSTM (cella)	$\text{diag}(\mathbf{f}_t)$	le porte, apprese e dipendenti dall'input

Su 50 passi: $f = 0.99 \Rightarrow 0.61$; $f = 0.95 \Rightarrow 0.081$; contro $w = 0.9 \Rightarrow 5.7 \times 10^{-3}$.

Va detto con onestà che l'LSTM *mitiga*, non elimina: se le porte imparano valori piccoli il gradiente svanisce comunque. La differenza è che ora è una decisione del modello, e la rete può tenere aperte poche dimensioni dedicate alla memoria lunga chiudendo le altre.

D4. Quali sono le differenze principali tra LSTM e GRU? Quale scegliereste per un task a risorse limitate?

La GRU fonde cella e stato nascosto in un solo $\mathbf{h}_t$ e riduce a due porte:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t]), \quad \mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]), \quad \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t.$$

Differenze sostanziali: input e forget sono *legati* in un unico $\mathbf{z}_t$ (quello che entra è esattamente quello che non resta), non c'è output gate quindi lo stato è sempre interamente esposto, e non c'è memoria separata dallo stato visibile. Parametri: 3 matrici contro 4, cioè circa il 25% in meno.

Per un task a risorse limitate: **GRU**. Meno parametri, meno memoria, training più veloce, e la letteratura empirica (Chung et al. 2014, Greff et al. 2017) non trova differenze sistematiche di accuratezza sulla maggior parte dei task.

Quando invece preferire LSTM: sequenze molto lunghe, dove poter trattenerne informazione senza esporla (output gate) fa differenza, e task in cui serve controllo fine e indipendente su “quanto dimenticare” e “quanto scrivere”. La regola pratica onesta è: si prova la GRU, e si passa all'LSTM solo se serve.

D5. Spiegare il concetto di teacher forcing e l'exposure bias che causa.

Durante il training di un modello autoregressivo si dà in input al passo t il *token vero* y_{t-1} del target, non la predizione del modello. Vantaggi: i passi diventano indipendenti fra loro e quindi parallelizzabili, la convergenza è molto più rapida, e il modello non insegue i propri errori nelle prime epoche quando è ancora casuale.

L'*exposure bias* è la conseguenza: a inferenza il token vero non esiste, e il modello riceve in input le proprie predizioni. Si trova quindi in una distribuzione di stati che non ha mai visto in training. Un errore iniziale porta in una regione mai esplorata, dove il modello è meno affidabile, generando altri errori: gli errori si *compongono*, e le generazioni lunghe degenerano.

È lo stesso fenomeno del distribution shift dell'imitation learning, dove si cura con DAgger.

Rimedi: *scheduled sampling* (Bengio et al. 2015), che alterna token veri e predetti con probabilità crescente sul predetto — semplice ma con un difetto teorico noto, perché lo stimatore risultante non è consistente; *professor forcing*, che usa un discriminatore avversariale per allineare le due dinamiche; e obiettivi a livello di sequenza (minimum risk training, RL con reward BLEU). Il beam search a inferenza mitiga il sintomo senza toccare la causa.

D6. *Come si estendono LSTM e GRU a bidirectional RNNs? Quando è utile?*

Si eseguono due reti indipendenti, una in avanti e una all'indietro, e si combinano gli stati:

$$\mathbf{h}_t = [\mathbf{h}_t^{\rightarrow}; \mathbf{h}_t^{\leftarrow}], \quad \mathbf{h}_t^{\rightarrow} = \text{LSTM}_{\rightarrow}(\mathbf{x}_t, \mathbf{h}_{t-1}^{\rightarrow}), \quad \mathbf{h}_t^{\leftarrow} = \text{LSTM}_{\leftarrow}(\mathbf{x}_t, \mathbf{h}_{t+1}^{\leftarrow}).$$

Oltre alla concatenazione si usano somma o media; il numero di parametri raddoppia.

È utile ogni volta che l'intera sequenza è disponibile e il contesto *futuro* è informativo: POS tagging e NER (“bank” si disambigua con ciò che segue), speech recognition offline, classificazione di sequenze, e l'encoder di un modello seq2seq.

È **impossibile** quando la sequenza arriva in streaming o quando il task è generativo autoregressivo: non si può guardare avanti a ciò che non è ancora stato prodotto. Per questo il decoder resta unidirezionale.

Il collegamento in avanti: BERT è la versione Transformer di questa idea — attenzione bidirezionale in encoder — mentre GPT è l'analogo unidirezionale. E lì la bidirezionalità richiede un obiettivo diverso (masked language modeling) proprio perché con accesso a tutto il contesto la predizione del token successivo sarebbe banale.

D7. *In un LSTM, se tutte le forget gates imparano a outputs $f_t \approx 0$, cosa succede alle informazioni a lungo termine? Quando sarebbe desiderabile?*

Con $\mathbf{f}_t \approx 0$ si ha $\mathbf{c}_t \approx \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$: la cella viene azzerata a ogni passo e riscritta da capo. La memoria a lungo termine **sparisce**, e la rete degenera in qualcosa di simile a una rete feed-forward sull'input corrente, con memoria di un solo passo. Il gradiente a lungo raggio è nullo: $\prod_i f_i \approx 0$.

Quando è desiderabile, e sono casi reali:

- *Reset a un confine*: fine di una frase, di un documento, di un episodio. Il contenuto precedente è irrilevante e trattenerlo danneggerebbe. Un forget gate che impara ad azzerarsi sul token di separazione è il comportamento corretto.
- *Processi genuinamente markoviani*: se il task dipende solo dall'input recente, la memoria lunga è rumore, e il modello fa bene a scartarla.
- *Cambio di regime*: serie temporali con rotture strutturali, dove ricordare il regime precedente peggiora la predizione.

Il punto generale, ed è la risposta che l'esaminatore cerca: le porte non servono solo a *ricordare*, servono a *decidere*. Un forget gate che sa azzerarsi è utile quanto uno che sa restare a 1; il difetto sarebbe averlo bloccato su un valore fisso in un senso o nell'altro. Ed è anche il motivo per cui il suo bias si inizializza positivo ma non enorme: si vuole una predisposizione a ricordare, non un vincolo.

Capitolo 17 — Encoder-decoder e cross-attention

D1. *Descrivere il problema del bottleneck nel encoder-decoder senza attention.*

Nel seq2seq classico l'encoder comprime l'intera sequenza di input in un solo vettore $\mathbf{c} = \mathbf{h}_T$, che è l'unica cosa che il decoder riceve. Quel vettore ha dimensione fissa, mentre la sequenza ha lunghezza variabile: è un collo di bottiglia informativo.

Le conseguenze sono tre. La capacità è costante mentre l'informazione da comprimere cresce con T , quindi la qualità degrada al crescere della lunghezza — e la degradazione è documentata empiricamente (Cho et al. 2014): oltre le 30 parole circa il BLEU crolla. I token iniziali sono i più penalizzati, perché devono sopravvivere a T aggiornamenti dello stato prima di essere usati. E la distanza fra un token di input e il punto in cui serve nell'output è $\mathcal{O}(T)$ passi, con tutto il vanishing gradient che ne consegue.

L'attenzione rimuove il vincolo alla radice: il decoder non riceve più un riassunto, ma può interrogare l'intera sequenza di stati dell'encoder a ogni passo, costruendosi un contesto $\mathbf{c}_t$ diverso ogni volta. La distanza diventa $\mathcal{O}(1)$.

D2. Derivare la formula per i pesi di attention $\alpha_{t,j}$ e spiegare l'uso della softmax.

Si calcola un punteggio di compatibilità fra lo stato del decoder $\mathbf{s}_{t-1}$ e ciascuno stato dell'encoder $\mathbf{h}_j$, poi lo si normalizza:

$$e_{t,j} = a(\mathbf{s}_{t-1}, \mathbf{h}_j), \quad \alpha_{t,j} = \frac{\exp(e_{t,j})}{\sum_{k=1}^T \exp(e_{t,k})}, \quad \mathbf{c}_t = \sum_{j=1}^T \alpha_{t,j} \mathbf{h}_j.$$

La softmax fa tre cose necessarie. Garantisce $\alpha_{t,j} > 0$ e $\sum_j \alpha_{t,j} = 1$, quindi $\mathbf{c}_t$ è una *media pesata* — una combinazione convessa degli stati dell'encoder, e resta nella loro regione dello spazio invece di finire da qualche parte a caso. È differenziabile, a differenza di un $\arg \max$ hard, quindi si può addestrare end-to-end senza stimatori ad alta varianza. E dà un'interpretazione probabilistica: $\alpha_{t,j}$ è “quanta attenzione” il decoder dedica alla posizione j .

Il prezzo, da dire: la softmax non azzeri mai nulla, quindi l'allineamento è *soft* e ogni posizione contribuisce sempre un po'. Le varianti hard usano campionamento e REINFORCE, con varianza molto più alta.

D3. Qual è la differenza tra attention additiva e moltiplicativa (dot-product)? Quali sono i vantaggi di ciascuna?

$$\text{additiva (Bahdanau)} : \quad e_{t,j} = \mathbf{v}^\top \tanh(\mathbf{W}_1 \mathbf{s}_{t-1} + \mathbf{W}_2 \mathbf{h}_j),$$

$$\text{moltiplicativa (Luong)} : \quad e_{t,j} = \mathbf{s}_{t-1}^\top \mathbf{h}_j \quad \text{oppure} \quad \mathbf{s}_{t-1}^\top \mathbf{W} \mathbf{h}_j \quad (\text{general}).$$

L'additiva ha parametri propri e una nonlinearietà, quindi è più espressiva e funziona anche quando le due rappresentazioni hanno dimensioni diverse o non sono nello stesso spazio. Storicamente è la prima (2015).

La moltiplicativa non ha parametri (nella forma dot-product pura) ed è *molto* più veloce, perché tutti i punteggi si ottengono con una singola moltiplicazione matriciale, che le GPU eseguono in modo estremamente efficiente. È il motivo per cui il Transformer usa questa.

Il punto quantitativo: sono comparabili in accuratezza per d_k piccolo, ma per d_k grande la dot-product degrada, perché i prodotti scalari crescono in modulo come $\sqrt{d_k}$ e saturano la softmax. La correzione è la scalatura $1/\sqrt{d_k}$ — che è precisamente il motivo per cui il Transformer usa *scaled* dot-product e non dot-product semplice.

D4. In cross-attention, quale componente (Q , K , V) proviene dal decoder e quale dall'encoder?

La **query** viene dal *decoder*; **key** e **value** vengono dall'*encoder*:

$$\mathbf{Q} = \mathbf{S} \mathbf{W}^Q \text{ (decoder)}, \quad \mathbf{K} = \mathbf{H} \mathbf{W}^K, \quad \mathbf{V} = \mathbf{H} \mathbf{W}^V \text{ (encoder)}.$$

La logica è quella di un dizionario: chi *cerca* formula la query, chi *offre* espone chiavi ed etichette. Il decoder sta chiedendo “di che cosa ho bisogno adesso per produrre il prossimo token?”, e l'encoder risponde con il contenuto pertinente.

Corollario dimensionale utile: la matrice di attenzione ha forma $T_{\text{dec}} \times T_{\text{enc}}$, cioè non è quadrata, a differenza della self-attention. Ed è anche il motivo per cui la cross-attention non usa mai il masking causale: il decoder può guardare l'intero input, non c'è nulla da nascondere — il masking serve solo sulla *propria* sequenza già generata.

D5. *Come si interpreta geometricamente l'attention matrix in machine translation?*

La matrice $\mathbf{A}$ con $A_{t,j} = \alpha_{t,j}$ ha righe indicizzate dai token di output e colonne da quelli di input; ogni riga è una distribuzione di probabilità. Visualizzata come mappa di calore, è una *matrice di allineamento* soft fra le due lingue.

Per lingue con ordine delle parole simile (inglese-francese) la massa si dispone lungo la *diagonale*. Dove l'ordine differisce, la diagonale si spezza: nella traduzione inglese-tedesco il verbo che va in fondo alla subordinata produce un salto ben visibile. Un token che si allinea a più token sorgente (o viceversa) produce una riga sparsa su più colonne, ed è il caso di *fertility* $\neq 1$ della traduzione statistica classica.

Il valore storico è che questo allineamento **emerge senza supervisione**: nella traduzione statistica pre-neurale l'allineamento era un modello a sé (IBM models), qui è un sottoprodotto dell'addestramento end-to-end.

Un avvertimento che vale la pena aggiungere: i pesi di attenzione sono una spiegazione *suggestiva* ma non necessariamente fedele di che cosa il modello stia usando — c'è una letteratura specifica (*attention is not explanation*) che mostra come pesi diversi possano dare le stesse predizioni.

D6. *Spiegare come l'attention mitiga il vanishing information problem per sequenze lunghe.*

Tre meccanismi, che vanno distinti.

Rimozione del bottleneck: non c'è più un solo vettore che deve contenere tutto. Il decoder accede a T vettori e ne combina un sottoinsieme pertinente a ogni passo, quindi la capacità disponibile cresce con la lunghezza dell'input invece di restare costante.

Accorciamento del cammino: la distanza fra il token di input j e il passo di output t passa da $\mathcal{O}(|t - j|)$ passi ricorrenti a un solo passo di attenzione. In backpropagation, il gradiente da $\mathbf{c}_t$ a $\mathbf{h}_j$ è diretto, con fattore $\alpha_{t,j}$, e non attraversa una catena di Jacobiani.

Selezione adattiva: il modello sceglie *quali* stati usare invece di ricevere un riassunto compresso deciso a priori. L'informazione rilevante non deve sopravvivere all'intero encoding, deve solo essere presente da qualche parte.

Il limite residuo va detto: l'attenzione risolve l'accesso, non la *rappresentazione*. Se lo stato $\mathbf{h}_j$ prodotto dall'encoder ricorrente ha già perso l'informazione, l'attenzione non può recuperarla. È per questo che il passo successivo è stato eliminare del tutto la ricorrenza dall'encoder — cioè il Transformer.

D7. *In image captioning, come differisce l'attention rispetto al machine translation?*

La struttura è la stessa (query dal decoder, key/value dalla sorgente), ma cambiano tre cose.

La sorgente non è una sequenza: sono le feature map di una CNN, tipicamente una griglia $14 \times 14 \times 512$ appiattita in 196 “posizioni”. Non c'è un ordine lineare né una nozione di precedenza, quindi il concetto di allineamento monotono — che nella traduzione è un prior forte — qui non ha senso.

L'output dell'attenzione è spaziale: i pesi $\alpha_{t,j}$ si visualizzano come una *mappa* sovrapposta all'immagine, che mostra quale regione il modello sta guardando mentre genera ciascuna parola. È il risultato più noto di Xu et al. 2015 (*Show, Attend and Tell*), e ha un potere illustrativo che il caso testuale non ha.

Esistono due varianti: *soft* (media pesata differenziabile, addestrabile con backprop) e *hard* (si campiona una regione, non differenziabile, addestrata con REINFORCE). Nella traduzione

la variante hard è rara; qui è stata studiata seriamente perché la nozione di “guardare un punto” è naturale.

Un’ultima differenza pratica: la lunghezza della sorgente è *fissa* (196 regioni indipendentemente dall’immagine), mentre nella traduzione varia con la frase.

Capitolo 18 — Transformers

D1. *Spiegare il meccanismo di self-attention e come differisce dalla cross-attention del capitolo precedente.*

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}.$$

Nella self-attention $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ derivano *tutte dalla stessa sequenza*: $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}^K$, $\mathbf{V} = \mathbf{X}\mathbf{W}^V$. Ogni posizione interroga tutte le altre della propria sequenza, sé stessa inclusa.

Nella cross-attention le query vengono da una sequenza e key/value da un’altra.

Il meccanismo è identico; cambia solo da dove si pescano i tre ingredienti. Le conseguenze però sono diverse: la matrice di self-attention è *quadrata* $n \times n$ e ammette il masking causale (che nella cross non serve); e soprattutto la self-attention costruisce *rappresentazioni contestualizzate* — ogni token viene riscritto in funzione del suo contesto — mentre la cross-attention trasferisce informazione fra due sequenze già rappresentate.

Il collegamento da fare all’orale: la self-attention è message passing su un **grafo completo**, con l’attenzione come funzione di aggregazione. Un Transformer è una GNN dove si assume che tutto sia connesso a tutto, e le posizioni entrano solo tramite il positional encoding.

D2. *Derivare la formula per multi-head attention e spiegare perché more heads permettono di catturare diverse dipendenze.*

$$\text{head}_i = \text{Attention}(\mathbf{X}\mathbf{W}_i^Q, \mathbf{X}\mathbf{W}_i^K, \mathbf{X}\mathbf{W}_i^V), \quad \text{MHA}(\mathbf{X}) = [\text{head}_1; \dots; \text{head}_h] \mathbf{W}^O,$$

con $\mathbf{W}_i^Q, \mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ e tipicamente $d_k = d_v = d_{\text{model}}/h$. Con questa scelta il costo totale è circa quello di una singola testa a dimensione piena: il multi-head è gratis.

Il motivo per cui più teste aiutano: una singola softmax produce *una* distribuzione per posizione, quindi il token può concentrarsi su un tipo di relazione alla volta. Con h teste si hanno h distribuzioni indipendenti, calcolate in sottospazi di proiezione diversi, quindi il modello può attendere contemporaneamente a relazioni di natura diversa — analisi empiriche su BERT trovano teste specializzate in dipendenze sintattiche, in coreferenze, o in pattern posizionali locali.

Due precisazioni oneste: la specializzazione delle teste è un fatto empirico e non una garanzia, e diversi lavori (Michel et al. 2019) mostrano che molte teste possono essere potate dopo il training senza perdita significativa — quindi la ridondanza è alta e il beneficio principale potrebbe essere di *ottimizzazione*, non di espressività.

D3. *Perché il fattore di scala $\frac{1}{\sqrt{d_k}}$ è importante nella self-attention?*

Se le componenti di $\mathbf{q}$ e $\mathbf{k}$ sono indipendenti con media 0 e varianza 1, allora $\mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$ ha media 0 e **varianza** d_k , quindi deviazione standard $\sqrt{d_k}$. Con $d_k = 64$ i logit hanno scala tipica 8.

Logit di quella scala saturano la softmax: la distribuzione diventa quasi one-hot, e la sua derivata $\alpha_i(\delta_{ij} - \alpha_j)$ va a zero ovunque. Il gradiente non passa più, e la rete non addestra.

Con numeri concreti: punteggi (8, 0, 5.6) danno $\alpha = (0.917, 0.0003, 0.083)$, entropia 0.289 nat; divisi per $\sqrt{64} = 8$ danno (0.474, 0.175, 0.351), entropia 1.026 su un massimo di $\log 3 = 1.099$.

Dividere per $\sqrt{d_k}$ riporta i logit a varianza unitaria *indipendentemente dalla dimensione*, quindi la temperatura effettiva della softmax non dipende più da una scelta architetturale. Non è un fattore cosmetico: senza, i Transformer con d_k grande non convergono.

D4. Descrivere le *positional encodings sinusoidali*. Quali proprietà hanno?

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right),$$

sommate (non concatenate) agli embedding dei token.

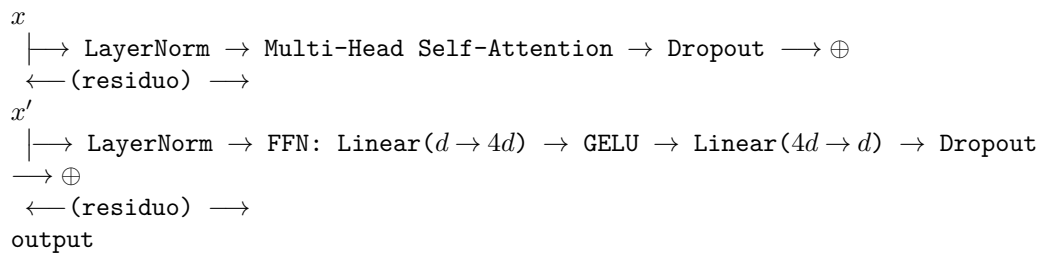
Servono perché la self-attention è **permutation-equivariante**: senza informazione posizionale, mescolare i token darebbe lo stesso risultato mescolato, e il modello non distinguerebbe “il cane morde l’uomo” da “l’uomo morde il cane”.

Proprietà da citare. Le frequenze coprono scale geometricamente distribuite da 2π a $10000 \cdot 2\pi$, quindi si codificano sia posizioni fini sia grossolane. Il prodotto scalare $PE_{pos} \cdot PE_{pos+k}$ dipende solo da k , quindi la codifica ha una nozione naturale di distanza *relativa*. E vale l’identità chiave: PE_{pos+k} è una **funzione lineare** di PE_{pos} , perché una traslazione di fase è una rotazione — quindi il modello può imparare l’offset relativo come una trasformazione lineare. Infine, essendo definita da una formula e non appresa, si estende a lunghezze mai viste in training.

Il confronto onesto: gli *embedding posizionali appresi* (usati da BERT e GPT) danno risultati comparabili ma non estrapolano. Le alternative moderne, RoPE e ALiBi, codificano la posizione relativa direttamente nel calcolo dell’attenzione e generalizzano meglio a contesti lunghi.

D5. Disegnare un blocco Transformer (*pre-norm variant*) e etichettare tutti i componenti.

Nella variante *pre-norm* la normalizzazione precede il sottolivello e il residuo scavalca entrambi:



In formule: $\mathbf{x}' = \mathbf{x} + \text{MHA}(\text{LN}(\mathbf{x}))$, poi $\mathbf{y} = \mathbf{x}' + \text{FFN}(\text{LN}(\mathbf{x}'))$.

La differenza con la variante *post-norm* dell’articolo originale ($\mathbf{x}' = \text{LN}(\mathbf{x} + \text{MHA}(\mathbf{x}))$) non è cosmetica: nel pre-norm esiste un cammino residuo *completamente lineare* dall’input all’output, non interrotto da alcuna LayerNorm, quindi il gradiente arriva intatto. Il post-norm richiede un warmup del learning rate accurato per non divergere, il pre-norm è molto più stabile — ed è la ragione per cui tutti i modelli grandi moderni lo usano.

Dettagli da nominare: la LayerNorm normalizza sulla dimensione delle feature per ogni token separatamente (non sul batch, quindi funziona con batch di dimensione uno e con sequenze di lunghezza variabile); la FFN espande di un fattore 4 ed è applicata *indipendentemente a ogni posizione*, il che la rende equivalente a due convoluzioni 1×1 .

D6. Qual è la differenza tra attention mascherato (nel decoder) e non mascherato (nell'encoder)?

Il masking causale impone $A_{ij} = -\infty$ per $j > i$ prima della softmax, così che dopo l'esponenziale quei pesi siano esattamente zero:

$$\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right), \quad M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i. \end{cases}$$

Nell'encoder non c'è maschera: ogni posizione vede tutte le altre, in entrambe le direzioni. È corretto perché l'input è interamente disponibile e il compito è rappresentarlo.

Nel decoder la maschera è *necessaria per la correttezza del training*, non un'ottimizzazione. Senza, la posizione i vedrebbe il token $i+1$, che è precisamente ciò che deve predire: il modello imparerebbe a copiare e a inferenza — dove il futuro non esiste — fallirebbe completamente. La maschera fa sì che il training in parallelo su tutta la sequenza sia *equivalente* alla generazione sequenziale.

Da aggiungere: si sottrae un valore grande finito (per esempio -10^9) invece di $-\infty$ per stabilità numerica; e nella pratica si combinano più maschere — quella causale e quella di padding, che esclude i token di riempimento nei batch a lunghezza variabile.

D7. Confrontare complexity time e space di Transformer vs RNN. Quando è Transformer più efficiente?

	complessità per layer	operazioni sequenziali	max path length
Self-attention	$\mathcal{O}(n^2d)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
RNN	$\mathcal{O}(nd^2)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Convoluzione (k)	$\mathcal{O}(knd^2)$	$\mathcal{O}(1)$	$\mathcal{O}(\log_k n)$

Sul *numero di operazioni*, la self-attention conviene quando $n < d$. Con $n = 512$ e $d = 64$ si ha 1.7×10^7 contro 2.1×10^6 : la self-attention costa 8 volte tanto, e il pareggio è a $n = d = 64$.

Ma la colonna che decide è la seconda. La RNN è *intrinsecamente sequenziale* — non si calcola $\mathbf{h}_t$ senza $\mathbf{h}_{t-1}$ — quindi lascia le GPU largamente inutilizzate; il Transformer fa più FLOP ma tutti in parallelo, e il tempo di parete crolla. Per questo il Transformer è più efficiente *in pratica* anche dove sulla carta costa di più.

Il vero limite è la *memoria*: la matrice $n \times n$ per testa e per layer. È il vincolo che rende difficili i contesti lunghi, e la ragione dell'intera letteratura sugli efficient Transformer (Linformer, Performer, sparse attention, FlashAttention — quest'ultima non riduce l'ordine ma evita di materializzare la matrice).

A inferenza autoregressiva il quadro si ribalta parzialmente: il Transformer genera un token alla volta, e senza KV-cache ricalcolerebbe tutto ogni volta. Con la cache il costo per token è $\mathcal{O}(nd)$ contro l' $\mathcal{O}(d^2)$ costante di una RNN — motivo per cui le architetture a stato ricorrente stanno tornando di interesse per l'inferenza.

D8. Spiegare come il Transformer affronta il problema di vanishing/exploding gradients meglio delle RNN.

Tre meccanismi concorrenti.

Cammino di lunghezza costante. In una RNN il gradiente fra due posizioni distanti n attraversa n Jacobiani, e decade o esplode esponenzialmente. Nella self-attention ogni coppia di posizioni è collegata *direttamente*: il cammino è $\mathcal{O}(1)$, e il gradiente passa attraverso un peso α_{ij} senza catene moltiplicative sulla dimensione temporale.

Connessioni residue. $\mathbf{y} = \mathbf{x} + \text{Sublayer}(\mathbf{x})$ dà $\partial \mathbf{y} / \partial \mathbf{x} = \mathbf{I} + \partial \text{Sublayer} / \partial \mathbf{x}$: esiste sempre un cammino non attenuato attraverso la profondità. È la stessa idea delle ResNet e del cell state dell'LSTM.

LayerNorm. Riporta le attivazioni a scala controllata a ogni sottolivello, impedendo che si accumulino derive di magnitudine attraverso i layer. Nella variante pre-norm, in particolare, non interrompe il cammino residuo.

La precisazione onesta: il problema non è *eliminato*, è spostato. Il Transformer non ha vanishing gradient lungo la sequenza, ma resta profondo, e i modelli molto grandi hanno comunque bisogno di warmup del learning rate, di inizializzazioni scalate con la profondità e di clipping. Semplicemente, la profondità è una dimensione molto più governabile della lunghezza della sequenza.

D9. *Descrivere il processo di pre-training e fine-tuning in BERT e GPT.*

BERT (encoder-only). Pre-training self-supervised su corpus non etichettati con due obiettivi: **Masked Language Modeling** — si maschera circa il 15% dei token e si predicono, con lo split 80/10/10 fra [MASK], token casuale e token invariato, introdotto per ridurre il disallineamento fra training e inferenza dove [MASK] non compare — e **Next Sentence Prediction**, poi mostrato di scarsa utilità e rimosso in RoBERTa. Fine-tuning: si aggiunge una testa leggera (classificatore su [CLS], o token-level per NER e QA) e si aggiorna *tutto* il modello con learning rate piccolo per poche epoche.

GPT (decoder-only). Pre-training con puro **language modeling** autoregressivo, $\max \sum_t \log P(x_t | x_{<t})$, con attenzione mascherata. Fine-tuning inizialmente analogo a BERT; dalla generazione GPT-3 in poi il paradigma è cambiato in *in-context learning*, cioè si condiziona il modello con esempi nel prompt senza aggiornare i pesi.

Il principio comune, ed è quello che conta: si separa l'apprendimento della *rappresentazione* — costoso, su dati non etichettati, fatto una volta — da quello del *compito* — economico, su pochi dati etichettati. È il transfer learning che nelle CNN si faceva con ImageNet, portato al testo con un obiettivo che non richiede annotazione.

D10. *In BERT, perché usiamo bidirectional attention durante pre-training, mentre in GPT usiamo masked attention?*

Perché i due modelli risolvono compiti diversi, e il tipo di attenzione discende dall'obiettivo.

GPT modella $P(x_t | x_{<t})$: è generativo autoregressivo, quindi *deve* essere causale. Se la posizione t vedesse x_t o oltre, l'obiettivo diventerebbe banale — il modello copierebbe — e a inferenza, dove il futuro non esiste, fallirebbe. La maschera non è una scelta ma un vincolo di correttezza.

BERT non genera: produce rappresentazioni. Per rappresentare bene un token serve tutto il contesto, e “bank” si disambigua guardando anche a destra. Quindi si vuole l'attenzione bidirezionale.

Ma con attenzione bidirezionale l'obiettivo autoregressivo non è utilizzabile — sarebbe banale per lo stesso motivo di prima. Da qui il masked language modeling: si nasconde una frazione dei token e si chiede di ricostruirli. È il modo di avere un obiettivo self-supervised non banale *pur* vedendo entrambi i lati.

Il compromesso, da nominare: BERT non è un modello generativo (non definisce una distribuzione su sequenze e non può campionare testo), e paga un disallineamento fra pre-training e fine-tuning perché il token [MASK] non compare mai a valle. GPT genera nativamente ma ogni rappresentazione è unilaterale. Le architetture encoder-decoder (T5, BART) cercano di prendere entrambi i vantaggi con obiettivi di denoising sequence-to-sequence.

Chapter 4

Parte IV — Generative deep learning

Capitolo 19 — Autoencoders e manifold learning

D1. Cosa significa che il DAE impara un'approssimazione della score function? Deriva l'identità di Tweedie e mostra come si arriva a $\nabla_{\hat{\mathbf{x}}} \log p(\hat{\mathbf{x}}) = (g(f(\hat{\mathbf{x}})) - \hat{\mathbf{x}})/\sigma^2$.

Sia $\hat{\mathbf{x}} = \mathbf{x} + \sigma\epsilon$ con $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, e $p(\hat{\mathbf{x}}) = \int p(\mathbf{x})\mathcal{N}(\hat{\mathbf{x}} | \mathbf{x}, \sigma^2\mathbf{I})d\mathbf{x}$ la densità del dato corrotto.

Identità di Tweedie. Derivando sotto il segno di integrale e usando $\nabla_{\hat{\mathbf{x}}}\mathcal{N}(\hat{\mathbf{x}} | \mathbf{x}, \sigma^2\mathbf{I}) = \frac{\mathbf{x} - \hat{\mathbf{x}}}{\sigma^2}\mathcal{N}(\cdot)$:

$$\nabla_{\hat{\mathbf{x}}}p(\hat{\mathbf{x}}) = \int p(\mathbf{x}) \frac{\mathbf{x} - \hat{\mathbf{x}}}{\sigma^2} \mathcal{N}(\hat{\mathbf{x}} | \mathbf{x}, \sigma^2\mathbf{I}) d\mathbf{x}.$$

Dividendo per $p(\hat{\mathbf{x}})$ e riconoscendo $p(\mathbf{x})\mathcal{N}(\hat{\mathbf{x}} | \mathbf{x})/p(\hat{\mathbf{x}}) = p(\mathbf{x} | \hat{\mathbf{x}})$:

$$\nabla_{\hat{\mathbf{x}}} \log p(\hat{\mathbf{x}}) = \frac{\mathbb{E}[\mathbf{x} | \hat{\mathbf{x}}] - \hat{\mathbf{x}}}{\sigma^2}$$

Il ponte col DAE. Il DAE minimizza $\mathbb{E}\|g(f(\hat{\mathbf{x}})) - \mathbf{x}\|^2$, e il minimizzatore dell'errore quadratico è la media condizionata: all'ottimo $g(f(\hat{\mathbf{x}})) = \mathbb{E}[\mathbf{x} | \hat{\mathbf{x}}]$. Sostituendo:

$$\nabla_{\hat{\mathbf{x}}} \log p(\hat{\mathbf{x}}) \approx \frac{g(f(\hat{\mathbf{x}})) - \hat{\mathbf{x}}}{\sigma^2}.$$

Come leggerlo: il vettore di ricostruzione, cioè lo spostamento che il DAE applica al punto rumoroso, punta nella direzione di massima crescita della log-densità, riscalato di $1/\sigma^2$. Un denoiser addestrato con MSE sta imparando, senza saperlo, il campo di score.

È il primo mattone della catena che porta a diffusion e flow matching: se conosco la score posso campionare con Langevin, e se la conosco a più livelli di rumore posso costruire un processo di generazione. Vale la pena dirlo esplicitamente all'orale, perché mostra che si è visto il filo.

D2. Differenza concettuale e formale fra sparse AE, denoising AE e contractive AE. Quando ne preferiresti uno rispetto agli altri?

Tutti e tre rispondono allo stesso rischio — un AE abbastanza espressivo impara l'identità — ma con vincoli diversi, e tutti permettono latenti *sovracompleti*, cosa che il solo bottleneck non consente.

$$\text{Sparse : } \mathcal{L} + \lambda\|\mathbf{h}\|_1, \quad \text{Denoising : } \mathbb{E}_{\tilde{\mathbf{x}}}\|g(f(\tilde{\mathbf{x}})) - \mathbf{x}\|^2, \quad \text{Contractive : } \mathcal{L} + \lambda\left\|\frac{\partial \mathbf{h}}{\partial \mathbf{x}}\right\|_F^2.$$

Concettualmente: lo *sparse* limita la capacità *per campione* (poche unità attive alla volta) invece che in assoluto; il *denoising* rende l'identità una soluzione sbagliata, perché copierebbe

anche il rumore, e costringe a imparare che cosa *non* appartiene ai dati; il *contractive* chiede esplicitamente insensibilità a piccole perturbazioni, tranne che nelle direzioni tangenti alla varietà.

Quando preferirli. *Sparse* se serve interpretabilità o un dizionario di feature quasi-indipendenti. *Denoising* se il fine è la robustezza o se poi si vuole usare il legame con la score — è anche il più economico, perché non richiede derivate aggiuntive. *Contractive* se interessa la geometria della varietà, accettando il costo del Jacobiano (che per latenti grandi è pesante).

DAE e CAE arrivano alla stessa conclusione da direzioni diverse — imparare il comportamento *attorno* alla varietà, non solo sopra — con la differenza che il DAE lo fa in modo stocastico e il CAE analitico.

D3. *Spiega il layerwise pretraining di un deep AE. Perché era necessario in passato e perché oggi è meno usato?*

Si addestra un AE a strato singolo su $\mathbf{x}$ ottenendo $\mathbf{W}_1$; lo si congela e si addestra un secondo AE su $\mathbf{z}_1 = f_1(\mathbf{x})$; si ripete fino allo strato L ; infine, opzionalmente, si fa un fine-tuning end-to-end dell'intera pila.

Era necessario perché prima del 2012 addestrare reti profonde end-to-end non funzionava: il vanishing gradient impediva agli strati bassi di ricevere segnale, e da inizializzazioni casuali l'ottimizzazione si bloccava. Il pretraining risolveva due problemi insieme — dava un'inizializzazione già ragionevole e fungeva da regolarizzatore, perché ogni strato era vincolato a rappresentare bene l'input dello strato precedente.

Oggi è raro perché i suoi motivi sono venuti a mancare uno per uno: inizializzazioni migliori (Xavier, He), attivazioni non saturanti (ReLU), ottimizzatori adattivi (Adam), normalizzazioni (batch e layer norm), connessioni residue, e soprattutto dataset molto più grandi che riducono il bisogno di regolarizzazione non supervisionata.

Da dire per chiudere: l'idea non è morta, si è *spostata*. Il pre-training non supervisionato seguito da fine-tuning supervisionato è esattamente il paradigma di BERT, GPT e dei modelli auto-supervisionati per immagini. È cambiato il come (obiettivo globale invece che strato per strato), non il perché.

D4. *Differenza fra DBN e DBM: quale è diretto, quale no, come differisce l'inferenza.*

La **DBN** è un modello *ibrido*: gli strati inferiori formano un grafo *diretto* top-down (una sigmoid belief network), mentre i due strati superiori restano una RBM *non diretta*. Si costruisce impilando RBM addestrate con contrastive divergence.

La **DBM** è *completamente non diretta*: è un MRF a più strati, con connessioni bidirezionali fra tutti i livelli adiacenti.

L'inferenza è la differenza pratica decisiva. Nella DBN, per la direzionalità top-down, l'inferenza approssimata “a salire” è semplice e greedy: si può stimare $\mathbf{h}_1$ da $\mathbf{x}$ senza tener conto degli strati superiori. Nella DBM ogni strato riceve input *sia* da sotto *sia* da sopra, quindi l'inferenza richiede un'iterazione a punto fisso (mean field) fra tutti gli strati. È più costosa ma anche più corretta, perché il contesto superiore disambigua quello inferiore.

Dettaglio tecnico da nominare: il pretraining di una DBM richiede il *double counting trick*, cioè dimezzare i pesi degli strati intermedi, perché nella DBM ogni unità nascosta riceve contributi da entrambe le direzioni e senza correzione verrebbero contati due volte.

D5. *Perché un AE classico non è un buon modello generativo, anche se ha un bottleneck?*

Perché un AE impara una *mappa*, non una *distribuzione*. L'addestramento vincola $g(f(\mathbf{x})) \approx \mathbf{x}$ per i punti del training set, ma non dice assolutamente nulla su come sia distribuito $\mathbf{z} = f(\mathbf{x})$ nello spazio latente.

Il risultato è che il latente è un insieme di isole: regioni popolate corrispondenti ai dati,

separate da vaste zone vuote. Se si prova a generare campionando un $\mathbf{z}$ a caso — da una gaussiana, dall'uniforme, da qualunque cosa — si cade quasi certamente in una zona mai vista, e il decoder, che lì non è vincolato da nulla, produce spazzatura. Anche l'interpolazione fra due codici validi attraversa il vuoto e genera immagini incoerenti a metà strada.

Detto formalmente: manca un prior $p(\mathbf{z})$ e manca qualunque termine che costringa la distribuzione aggregata $q(\mathbf{z}) = \mathbb{E}_{p_{\text{data}}}[q(\mathbf{z} | \mathbf{x})]$ ad avvicinarsi a una distribuzione campionabile.

Ed è esattamente il buco che il VAE riempie: il termine $D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) || p(\mathbf{z}))$ forza i posterior a sovrapporsi e a coprire il prior, riempiendo i vuoti. L'AAE fa la stessa cosa con un discriminatore invece che con una KL analitica.

D6. *Confronto fra Tweedie's formula e reparametrization trick: sono lo stesso strumento?*

No, e il confronto è utile proprio perché sembrano parenti.

Tweedie è un'*identità statistica*: lega la media a posteriori del segnale pulito al gradiente della log-densità del segnale rumoroso, $\mathbb{E}[\mathbf{x} | \hat{\mathbf{x}}] = \hat{\mathbf{x}} + \sigma^2 \nabla \log p(\hat{\mathbf{x}})$. È un fatto vero sulle distribuzioni, indipendente da qualunque algoritmo. Serve a *interpretare*: dice che un denoiser sta imparando la score.

Il reparametrization trick è una *tecnica di stima del gradiente*. Riscrive $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}_\phi, \boldsymbol{\sigma}_\phi^2)$ come $\mathbf{z} = \boldsymbol{\mu}_\phi + \boldsymbol{\sigma}_\phi \odot \boldsymbol{\epsilon}$ con $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$, spostando la casualità fuori dal cammino del gradiente. Non afferma nulla sulle distribuzioni: risolve un problema di differenziabilità.

Le due cose che hanno in comune sono superficiali — entrambe sfruttano proprietà della gaussiana — e i loro ruoli sono complementari: Tweedie spiega *che cosa* un modello sta imparando, la riparametrizzazione rende possibile *addestrarlo*. Si potrebbe addestrare un VAE senza Tweedie (e infatti lo si fa), e si può usare Tweedie in modelli che non hanno alcun encoder da riparametrizzare (i diffusion).

Capitolo 20 — Variational Autoencoders

D1. *Deriva l'ELBO partendo da $\log p_\theta(\mathbf{x}) = \log \int p(\mathbf{x}, \mathbf{z}) d\mathbf{z}$ usando Jensen. Mostra come compaiono ricostruzione e KL.*

$$\log p_\theta(\mathbf{x}) = \log \int q_\phi(\mathbf{z} | \mathbf{x}) \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} | \mathbf{x})} d\mathbf{z} \stackrel{\text{Jensen}}{\geq} \mathbb{E}_{q_\phi} \left[\log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z} | \mathbf{x})} \right].$$

Fattorizzando $p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{x} | \mathbf{z})p(\mathbf{z})$ e separando:

$$\mathcal{L} = \mathbb{E}_{q_\phi} [\log p_\theta(\mathbf{x} | \mathbf{z})] + \mathbb{E}_{q_\phi} \left[\log \frac{p(\mathbf{z})}{q_\phi(\mathbf{z} | \mathbf{x})} \right] = \underbrace{\mathbb{E}_{q_\phi} [\log p_\theta(\mathbf{x} | \mathbf{z})]}_{\text{ricostruzione}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) || p(\mathbf{z}))}_{\text{regolarizzazione}}.$$

Il secondo termine è una KL perché $\mathbb{E}_q[\log(p/q)] = -D_{\text{KL}}(q||p)$ per definizione: è il passaggio da mostrare esplicitamente.

Vale anche l'identità esatta $\log p_\theta(\mathbf{x}) = \mathcal{L} + D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) || p_\theta(\mathbf{z} | \mathbf{x}))$, che dice tre cose: il bound è tale perché la KL è non negativa; massimizzare $\mathcal{L}$ equivale ad avvicinare q_ϕ al posterior vero; il gap è l'errore di approssimazione dell'encoder.

Interpretazione dei due termini: il primo vuole ricostruire bene (e da solo darebbe un autoencoder), il secondo vuole che il latente sia distribuito come il prior (e da solo ignorerebbe i dati). Il VAE è l'equilibrio fra i due.

D2. *Cos'è il reparametrization trick e perché è necessario? Cosa succederebbe senza?*

$$\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_\phi^2(\mathbf{x}))) \implies \mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}).$$

Il problema che risolve: il gradiente $\nabla_{\phi} \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[f(\mathbf{z})]$ non si può portare dentro l'attesa, perché la distribuzione rispetto a cui si media *dipende* da ϕ . L'operazione di campionamento è un nodo stocastico non differenziabile, e la backpropagation si ferma lì.

Con la riparametrizzazione l'attesa è rispetto a ϵ , che non dipende da ϕ , quindi

$$\nabla_{\phi} \mathbb{E}_{\epsilon}[f(\boldsymbol{\mu}_{\phi} + \boldsymbol{\sigma}_{\phi} \odot \epsilon)] = \mathbb{E}_{\epsilon}[\nabla_{\phi} f(\cdot)],$$

e il grafo è differenziabile da capo a fondo. La casualità non è stata eliminata: è stata spostata da *dentro* il grafo a un *ingresso* del grafo.

Senza il trucco resterebbe lo *score function estimator* (REINFORCE): $\nabla_{\phi} \mathbb{E}_q[f] = \mathbb{E}_q[f(\mathbf{z}) \nabla_{\phi} \log q_{\phi}(\mathbf{z} | \mathbf{x})]$. È non distorto e non richiede differenziabilità di f , ma ha varianza *ordini di grandezza* superiore, perché usa solo il valore scalare f invece della sua derivata. In pratica un VAE addestrato così non converge in tempi ragionevoli.

Limite del trucco: richiede una riparametrizzazione location-scale, quindi non si applica a latenti discreti. Da lì Gumbel-Softmax, straight-through e VQ-VAE.

D3. Deriva $D_{\text{KL}}(\mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2)) \| \mathcal{N}(0, \mathbf{I}))$ in forma chiusa.

Caso unidimensionale, poi si somma sulle dimensioni per la fattorizzazione. Con $q = \mathcal{N}(\mu, \sigma^2)$ e $p = \mathcal{N}(0, 1)$:

$$D_{\text{KL}}(q \| p) = \mathbb{E}_q[\log q - \log p] = \mathbb{E}_q \left[-\frac{1}{2} \log(2\pi\sigma^2) - \frac{(z - \mu)^2}{2\sigma^2} + \frac{1}{2} \log 2\pi + \frac{z^2}{2} \right].$$

Usando $\mathbb{E}_q[(z - \mu)^2] = \sigma^2$ e $\mathbb{E}_q[z^2] = \mu^2 + \sigma^2$:

$$= -\frac{1}{2} \log \sigma^2 - \frac{1}{2} + \frac{\mu^2 + \sigma^2}{2} = \frac{1}{2}(\mu^2 + \sigma^2 - 1 - \log \sigma^2).$$

In d dimensioni con covarianza diagonale:

$$D_{\text{KL}}(q \| p) = \frac{1}{2} \sum_{j=1}^d (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2).$$

Verifiche: con $\mu = 0, \sigma = 1$ dà esattamente 0; con $\mu = 0.5, \sigma = 0.8$ dà $\frac{1}{2}(0.25 + 0.64 - 1 + 0.446) = 0.168$.

Il termine $-\log \sigma^2$ è quello da commentare: diverge per $\sigma \rightarrow 0$, quindi penalizza all'infinito un encoder che voglia diventare deterministico. È ciò che impedisce al VAE di collassare in un autoencoder ordinario e ciò che riempie i vuoti dello spazio latente.

Nota implementativa: la rete emette $\log \sigma^2$ e non σ , così da garantire positività senza vincoli e da rendere il termine numericamente stabile.

D4. Cos'è il *posterior collapse*? In quali condizioni si verifica e come si mitiga?

È la situazione in cui $q_{\phi}(\mathbf{z} | \mathbf{x}) \rightarrow p(\mathbf{z})$ per ogni $\mathbf{x}$: la KL va a zero, il latente non porta più informazione sull'input, e il decoder impara a ignorarlo. Si riconosce guardando la KL durante il training: se tende a zero mentre la ricostruzione non migliora, è questo.

Condizioni che lo favoriscono, in ordine di importanza. Un *decoder troppo espressivo*, tipicamente autoregressivo (PixelCNN, RNN, Transformer): se il decoder può modellare bene $p(\mathbf{x})$ da solo, l'ottimizzatore trova conveniente pagare zero KL e ignorare $\mathbf{z}$. Un peso eccessivo sulla KL, come in β -VAE con β grande. E le prime epoche di training, quando il decoder è ancora debole e il latente porta rumore invece che segnale: la via di minor resistenza è spegnerlo, e una volta spento non si riaccende.

Mitigazioni: *KL annealing* (peso della KL portato da 0 a 1 gradualmente); *free bits*, che impone un minimo di KL per dimensione, $\max(\lambda, D_{\text{KL}}(\cdot \| \cdot)_j)$, così che ogni dimensione debba

portare almeno un po' di informazione; indebolire deliberatamente il decoder (word dropout, contesto troppo corto); latenti discreti (VQ-VAE), che non hanno questo failure mode; e obiettivi alternativi come InfoVAE o δ -VAE.

D5. *Confronta VAE con DAE: entrambi usano rumore gaussiano, dov'è la differenza?*

Il rumore sta in due punti diversi e serve a due cose diverse.

Nel **DAE** il rumore è *sull'input*: si corrompe $\tilde{\mathbf{x}} = \mathbf{x} + \sigma\epsilon$ e si chiede di ricostruire l'originale. Serve a impedire che l'identità sia una soluzione e a costringere il modello a imparare la struttura dei dati (e implicitamente la score, D1 del capitolo precedente). Non c'è nessun prior sul latente, quindi il DAE resta un modello *non generativo*: da esso non si sa campionare.

Nel **VAE** il rumore è *nel latente*: $\mathbf{z} = \mu_\phi + \sigma_\phi \odot \epsilon$, con σ_ϕ appresa e dipendente dall'input. Serve a rendere il posterior una distribuzione e a permettere che la KL lo allinei a un prior campionabile. È questo che rende il VAE generativo.

Riassumendo in una riga: nel DAE il rumore è un *regolarizzatore di training*, con varianza fissata a mano e assente a inferenza; nel VAE è *parte del modello*, appreso, e presente anche in generazione.

Il legame profondo va però nominato: entrambi imparano informazione sulla struttura locale della densità, e sono i due antenati diretti della diffusion — che è, letteralmente, un DAE gerarchico su molti livelli di rumore addestrato con un ELBO alla VAE.

D6. *Perché i campioni VAE sono tipicamente sfocati? È un problema dell'ELBO o della parametrizzazione gaussiana del decoder?*

La causa principale è la **parametrizzazione del decoder**, non l'ELBO in sé.

Con $p_\theta(\mathbf{x} | \mathbf{z}) = \mathcal{N}(g_\theta(\mathbf{z}), \sigma^2 \mathbf{I})$, il termine di ricostruzione diventa $-\|\mathbf{x} - g_\theta(\mathbf{z})\|^2 / 2\sigma^2$ più costanti: un errore quadratico. E l'MSE è *mode-averaging*: di fronte a un'ambiguità — dato quel latente, il dato vero potrebbe essere questo o quello — il minimizzatore è la *media* delle alternative plausibili, non una qualunque di esse. La media di più volti nitidi è un volto sfocato.

Contributi secondari, che vanno nominati per completezza. L'*approssimazione del posterior*: una q gaussiana fattorizzata è troppo povera, il gap di KL resta, e il decoder riceve latenti più rumorosi del necessario. E il termine KL stesso, che comprime informazione e sacrifica dettagli fini.

La prova che la colpa sia della likelihood e non dell'ELBO: cambiando il decoder si eliminano le sfocature a parità di obiettivo. Decoder autoregressivi (PixelVAE), likelihood discretizzate, VQ-VAE con prior autoregressivo producono campioni nitidi. E all'estremo opposto, la diffusion ottimizza anch'essa un ELBO e non è affatto sfocata, perché ogni passo risolve un problema di denoising quasi deterministico, in cui l'ambiguità da mediare è minima.

Il GAN non ha il problema per la ragione opposta: non ottimizza nessuna verosimiglianza per pixel, quindi non è mai spinto a mediare — e paga con il mode collapse, che è il difetto simmetrico.

D7. *Come modificherei l'ELBO per condizionare la generazione su un'etichetta $\mathbf{y}$? Scrivi la loss CVAE.*

Si condiziona *ogni* componente su $\mathbf{y}$: encoder, decoder e prior.

$$\mathcal{L}_{\text{CVAE}}(\theta, \phi; \mathbf{x}, \mathbf{y}) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x}, \mathbf{y})} [\log p_\theta(\mathbf{x} | \mathbf{z}, \mathbf{y})] - D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}, \mathbf{y}) \| p_\theta(\mathbf{z} | \mathbf{y})).$$

In pratica $\mathbf{y}$ (one-hot o embedding) viene concatenato all'input dell'encoder e a $\mathbf{z}$ in ingresso al decoder. Il prior condizionale $p_\theta(\mathbf{z} | \mathbf{y})$ può essere una rete a sua volta, oppure si semplifica a $\mathcal{N}(0, \mathbf{I})$ indipendente da $\mathbf{y}$ — scelta comune, che però costringe tutta l'informazione sull'etichetta a passare per il decoder.

Generazione: si fissa $\mathbf{y}$, si campiona $\mathbf{z} \sim p(\mathbf{z} | \mathbf{y})$ e si decodifica.

Il punto qualificante: il ruolo di $\mathbf{z}$ cambia. Non deve più codificare *tutto* il dato, ma solo ciò che $\mathbf{y}$ non determina — lo stile, la posa, la variazione entro la classe. È una decomposizione esplicita in fattore noto e fattore residuo, ed è la stessa logica del condizionamento nei diffusion, dove però si preferisce la classifier-free guidance perché permette di regolare a inferenza *quanto* pesare la condizione.

Capitolo 21 — Generative Adversarial Networks

D1. *Scrivi l'obiettivo minimax di Goodfellow e ricava il discriminatore ottimale D^* per G fisso.*

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

Riscrivendo il secondo termine come integrale su $\mathbf{x}$ con densità p_G :

$$V(D, G) = \int [p_{\text{data}}(\mathbf{x}) \log D(\mathbf{x}) + p_G(\mathbf{x}) \log(1 - D(\mathbf{x}))] d\mathbf{x}.$$

L'integrando si massimizza *puntualmente*, perché D può assumere valori indipendenti in ogni $\mathbf{x}$. Posto $f(d) = a \log d + b \log(1 - d)$:

$$f'(d) = \frac{a}{d} - \frac{b}{1-d} = 0 \implies d = \frac{a}{a+b}, \quad f''(d) = -\frac{a}{d^2} - \frac{b}{(1-d)^2} < 0,$$

quindi è un massimo. Sostituendo $a = p_{\text{data}}$ e $b = p_G$:

$$D^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_G(\mathbf{x})}.$$

Da notare: $D^* = 1/2$ ovunque quando $p_G = p_{\text{data}}$, cioè il discriminatore ottimo è quello che non sa più distinguere. E l'argomento di ottimizzazione puntuale richiede che D sia arbitrariamente espressivo: con una rete di capacità finita D^* non è raggiungibile, ed è da lì che vengono buona parte dei problemi pratici.

D2. *Mostra che sostituendo D^* il problema equivale a minimizzare la Jensen-Shannon. Qual è l'ottimo globale?*

Sostituendo D^* e ponendo $M = (p_{\text{data}} + p_G)/2$:

$$V(D^*, G) = \mathbb{E}_{p_{\text{data}}} \left[\log \frac{p_{\text{data}}}{2M} \right] + \mathbb{E}_{p_G} \left[\log \frac{p_G}{2M} \right] = -\log 4 + D_{\text{KL}}(p_{\text{data}} \| M) + D_{\text{KL}}(p_G \| M).$$

Poiché $\text{JSD}(p \| q) = \frac{1}{2} D_{\text{KL}}(p \| M) + \frac{1}{2} D_{\text{KL}}(q \| M)$:

$$V(D^*, G) = -\log 4 + 2 \text{JSD}(p_{\text{data}} \| p_G).$$

La JSD è non negativa e si annulla se e solo se le due distribuzioni coincidono, quindi il minimo globale è

$$V = -\log 4 \approx -1.386 \quad \text{raggiunto sse } p_G = p_{\text{data}},$$

e in quel punto $D^* \equiv 1/2$.

Il commento che qualifica la risposta: questo risultato vale *all'ottimo del discriminatore*. Nella pratica D non è mai ottimo — si alternano pochi passi per parte — quindi il generatore sta minimizzando un'approssimazione della JSD, e le garanzie teoriche non si applicano.

Inoltre il risultato è nello spazio delle *densità*, non dei parametri: dice che l'ottimo esiste, non che la dinamica del gradiente ci arrivi.

D3. *Cos'è il mode collapse? Cosa lo causa intuitivamente?*

Il generatore produce campioni concentrati su pochi modi della distribuzione vera, ignorandone altri: le immagini generate sono di buona qualità ma poco varie, e nei casi estremi quasi identiche fra loro.

L'intuizione della causa sta nell'asimmetria dell'obiettivo. Il generatore è premiato per *ingannare* D , non per *coprire* p_{data} . Se esiste una singola regione dove D è debole, portarci tutta la massa è una strategia ottima dal punto di vista della loss. Non c'è nessun termine che penalizzi la mancata copertura: la likelihood, che avrebbe questo effetto perché $D_{\text{KL}}(p_{\text{data}} \| p_G)$ diverge dove p_G è zero e i dati no, semplicemente non compare.

Contribuisce anche la dinamica dell'alternanza: G si sposta sul modo dove D è debole, D impara a riconoscerlo, G si sposta altrove. Il risultato può essere un ciclo (mode hopping) invece di una convergenza.

Rimedi: *minibatch discrimination* e *PacGAN*, che danno a D più campioni insieme così che possa rilevare la mancanza di varietà; *unrolled GAN*, che fa guardare a G qualche passo avanti della risposta di D ; e soprattutto **WGAN**, perché la distanza di Wasserstein misura anche quanto manca da coprire, non solo se si è distinguibili.

D4. *Perché la WGAN funziona meglio della GAN classica quando le distribuzioni hanno supporti disgiunti?*

Perché la JSD è *costante* fra distribuzioni con supporti disgiunti, e una costante ha gradiente nullo.

L'esempio minimo: p concentrata sul segmento $\{0\} \times [0, 1]$ e q_θ su $\{\theta\} \times [0, 1]$. Allora

$$\text{JSD}(p \| q_\theta) = \log 2 \quad \forall \theta \neq 0, \quad \text{JSD} = 0 \text{ per } \theta = 0,$$

mentre $W(p, q_\theta) = |\theta|$. La JSD è discontinua e non fornisce nessuna direzione di discesa; la Wasserstein è continua, differenziabile quasi ovunque, e il suo gradiente indica *da che parte* muoversi.

Perché la situazione è la norma e non un caso patologico: p_G vive sull'immagine di G , una varietà di dimensione al più $\dim \mathbf{z}$, immersa in uno spazio molto più grande; e i dati reali vivono anch'essi su una varietà di bassa dimensione. Due varietà di bassa dimensione in alta dimensione si intersecano in un insieme di misura nulla, quasi sempre. Quindi supporti (quasi) disgiunti sono il caso generico, ed è la ragione teorica per cui la GAN classica può avere gradienti nulli o instabili.

Conseguenza pratica ulteriore: nella WGAN si può addestrare il critico *a convergenza* prima di ogni passo del generatore — cosa che nella GAN classica peggiora le cose, perché un D perfetto azzerava il gradiente — e la loss del critico diventa una stima di W , quindi correla con la qualità dei campioni. È l'unica GAN con una metrica di training interpretabile.

D5. *Qual è l'enunciato del teorema di dualità Kantorovich-Rubinstein, e come si usa per derivare la loss WGAN?*

Il problema di trasporto ottimo primale è

$$W(p, q) = \inf_{\gamma \in \Pi(p, q)} \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \gamma} [\|\mathbf{x} - \mathbf{y}\|],$$

con $\Pi(p, q)$ l'insieme dei piani di trasporto con marginali p e q . È intrattabile: si ottimizza su distribuzioni congiunte.

La *dualità di Kantorovich-Rubinstein* lo riscrive come

$$W(p, q) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{\mathbf{x} \sim p}[f(\mathbf{x})] - \mathbb{E}_{\mathbf{x} \sim q}[f(\mathbf{x})]$$

dove il sup è su tutte le funzioni 1-Lipschitz. Si è passati da un'ottimizzazione su piani di trasporto a una su funzioni scalari, che una rete può parametrizzare.

Da qui la loss WGAN: si sostituisce f con un *critico* f_w (non un discriminatore: non c'è sigmoide finale, l'uscita non è una probabilità) e si ottiene

$$\min_G \max_{w: \|f_w\|_L \leq 1} \mathbb{E}_{p_{\text{data}}}[f_w(\mathbf{x})] - \mathbb{E}_z[f_w(G(\mathbf{z}))].$$

Il vincolo di Lipschitz è la parte delicata, ed è dove le implementazioni si distinguono: *weight clipping* nell'articolo originale (semplice ma grezzo, riduce la capacità del critico e può causare gradienti patologici), *gradient penalty* in WGAN-GP, che aggiunge $\lambda(\|\nabla_{\hat{\mathbf{x}}} f_w(\hat{\mathbf{x}})\|_2 - 1)^2$ su punti interpolati fra reali e generati, e *spectral normalization*, che normalizza ogni matrice di pesi per il suo massimo valore singolare.

D6. In una GAN il generatore vede mai i dati reali? Spiega.

No, mai direttamente. Il generatore riceve solo $\mathbf{z} \sim p(\mathbf{z})$ e produce $G(\mathbf{z})$; l'unico segnale che riceve dai dati è il *gradiente* che risale attraverso il discriminatore, $\nabla_{\theta_G} \log D(G(\mathbf{z}))$. I dati reali entrano solo nella loss di D .

Le conseguenze sono interessanti e vale la pena elencarle.

È un *vantaggio per la privacy*: il generatore non memorizza esempi, e questo è alla base di alcune proposte di generazione di dati sintetici. Con l'avvertenza che la memorizzazione indiretta è comunque possibile, e infatti è documentata.

Spiega perché il generatore è “cieco”: non ha nessuna nozione di quale porzione della distribuzione stia coprendo. È un altro modo di vedere il mode collapse — non gli manca la volontà di coprire i modi, gli manca proprio l'informazione.

Spiega il ruolo critico di D : se il discriminatore è troppo debole il segnale è inutile, se è troppo forte il gradiente satura. La qualità del generatore è limitata dalla qualità dell'informazione che D riesce a trasmettergli.

Il contrasto con VAE e diffusion è netto: entrambi hanno un termine di ricostruzione che confronta direttamente output e dato vero, quindi “vedono” i dati.

D7. Differenza concettuale fra AAE e VAE: cosa cambia nel termine di regolarizzazione e quali vantaggi/svantaggi porta.

Entrambi vogliono che il latente segua un prior campionabile; cambia *come* lo impongono.

Il VAE usa $D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) || p(\mathbf{z}))$, calcolata analiticamente e *per campione*. L'AAE sostituisce quel termine con un *discriminatore* che distingue $\mathbf{z} \sim q(\mathbf{z})$ (posterior aggregato) da $\mathbf{z} \sim p(\mathbf{z})$, e l'encoder è addestrato a ingannarlo.

Le differenze che ne discendono. Primo, l'AAE regolarizza il posterior *aggregato* $q(\mathbf{z}) = \mathbb{E}_{p_{\text{data}}}[q(\mathbf{z} | \mathbf{x})]$, il VAE i posterior individuali: è una richiesta più debole e più aderente a ciò che serve davvero per campionare. Secondo, il prior può essere **qualunque distribuzione da cui si sappia campionare** — una mistura, una distribuzione su una sfera, una swiss roll — senza bisogno di una KL in forma chiusa. È il vantaggio principale.

Svantaggi: si eredita l'instabilità avversariale e si perde il bound sulla likelihood, quindi non c'è più un ELBO da monitorare né un criterio principiato di valutazione.

Il collegamento da fare: l'AAE è il punto in cui la famiglia VAE e quella GAN si toccano, e prefigura tutta la linea dei modelli che sostituiscono divergenze esplicite con critici appresi.

D8. Confronta VAE e GAN su likelihood, qualità campioni, stabilità training, possibilità di inferenza.

	VAE	GAN
Likelihood	bound (ELBO), valutabile	nessuna: modello implicito
Qualità campioni	sfocati (MSE mode-averaging)	nitidi
Copertura modi	buona (KL penalizza i buchi)	scarsa: mode collapse
Stabilità	alta, obiettivo singolo	bassa: punto di sella
Inferenza $\mathbf{x} \rightarrow \mathbf{z}$	nativa (encoder)	assente, va aggiunta
Valutazione	ELBO + metriche	solo FID/IS, con difetti noti

La lettura da dare a voce: i due difetti sono *simmetrici e speculari*. Il VAE ottimizza una verosimiglianza per pixel, quindi è spinto a mediare fra le alternative plausibili (sfocatura) ma non può permettersi di lasciare regioni scoperte, perché la likelihood esploderebbe. La GAN non ottimizza nessuna verosimiglianza, quindi non media mai (nitidezza) ma non paga nulla per ignorare interi modi.

Sull'inferenza: la GAN non ha un encoder per costruzione, e recuperarlo richiede lavoro aggiuntivo (BiGAN/ALI addestrano un encoder avversarialmente, GAN inversion ottimizza $\mathbf{z}$ per singola immagine). È una differenza pratica pesante per l'editing.

Le architetture ibride (VAE-GAN, AAE) cercano il meglio dei due: latente strutturato e inferenza dal VAE, nitidezza dal critico avversariale.

Capitolo 22 — Normalizing Flows

D1. *Scrivi e dimostra la formula del cambio di variabile multidimensionale. Spiega perché compare $|\det J|^{-1}$.*

Sia $\mathbf{x} = f(\mathbf{z})$ con f biiettiva e differenziabile. Per conservazione della probabilità su regioni corrispondenti, $P(\mathbf{x} \in A) = P(\mathbf{z} \in f^{-1}(A))$, cioè

$$\int_A p_X(\mathbf{x}) d\mathbf{x} = \int_{f^{-1}(A)} p_Z(\mathbf{z}) d\mathbf{z}.$$

Sostituendo $\mathbf{z} = f^{-1}(\mathbf{x})$ nel secondo integrale, l'elemento di volume si trasforma con $d\mathbf{z} = |\det J_{f^{-1}}(\mathbf{x})| d\mathbf{x}$, e poiché A è arbitrario gli integrandi coincidono:

$$p_X(\mathbf{x}) = p_Z(f^{-1}(\mathbf{x})) |\det J_{f^{-1}}(\mathbf{x})| = p_Z(\mathbf{z}) |\det J_f(\mathbf{z})|^{-1},$$

usando $J_{f^{-1}} = J_f^{-1}$ e $\det(\mathbf{A}^{-1}) = 1/\det \mathbf{A}$. In log:

$$\log p_X(\mathbf{x}) = \log p_Z(\mathbf{z}) - \log |\det J_f(\mathbf{z})|.$$

Perché $|\det J|^{-1}$: il determinante dello Jacobiano è il fattore di dilatazione locale del volume. Se f espande il volume in un punto, la stessa massa di probabilità si distribuisce su una regione più grande, quindi la densità *cala*: da qui l'esponente -1 , ossia il segno meno in forma logaritmica. Il valore assoluto serve perché una trasformazione può invertire l'orientamento; il volume no.

Verifica unidimensionale: $z \sim \mathcal{N}(0, 1)$, $x = 2z + 1$. Allora $\log p_X(3) = \log p_Z(1) - \log 2 = -0.5 - 0.919 - 0.693 = -2.112$, che coincide con $\log \mathcal{N}(3 | 1, 4) = -2.112$.

D2. *Scrivi la log-likelihood di un flow composto da K layer. Quali sono i requisiti pratici sui layer?*

Con $\mathbf{x} = f_K \circ \dots \circ f_1(\mathbf{z}_0)$ e $\mathbf{z}_k = f_k(\mathbf{z}_{k-1})$:

$$\log p_X(\mathbf{x}) = \log p_Z(\mathbf{z}_0) - \sum_{k=1}^K \log |\det J_{f_k}(\mathbf{z}_{k-1})|.$$

I log-determinanti si *sommano*, perché il determinante di una composizione è il prodotto dei determinanti. È questa additività a rendere i flow componibili senza esplosione di costo.

Requisiti pratici su ogni layer, tutti e tre necessari:

1. **invertibilità**, con inversa calcolabile — serve per campionare (in avanti) e per valutare la likelihood (all'indietro);
2. **determinante dello Jacobiano calcolabile in tempo lineare**: $\mathcal{O}(d^3)$ è proibitivo, quindi si impongono strutture triangolari, diagonali, o si usano stimatori stocastici;
3. **espressività sufficiente** per giustificare la composizione, altrimenti servirebbero troppi layer.

Vincolo strutturale da dichiarare sempre: l'invertibilità impone $\dim \mathbf{z} = \dim \mathbf{x}$ a ogni livello. Un flow non può comprimere, quindi non ha uno spazio latente più piccolo dei dati — a differenza di VAE e latent diffusion. Ottiene in cambio la likelihood *esatta*.

D3. *Spiega come funziona un coupling layer (RealNVP). Perché il Jacobiano è triangolare? Quanto costa l'inversione?*

Si divide il vettore in due blocchi e si trasforma solo il secondo, condizionando sul primo:

$$\mathbf{x}_{1:d} = \mathbf{z}_{1:d}, \quad \mathbf{x}_{d+1:D} = \mathbf{z}_{d+1:D} \odot \exp(s(\mathbf{z}_{1:d})) + t(\mathbf{z}_{1:d}).$$

Lo Jacobiano ha la forma a blocchi

$$J = \begin{pmatrix} \mathbf{I}_d & \mathbf{0} \\ \partial \mathbf{x}_{2:d} / \partial \mathbf{z}_1 & \text{diag}(e^{s(\mathbf{z}_1)}) \end{pmatrix},$$

triangolare inferiore perché il primo blocco non dipende dal secondo (blocco in alto a destra nullo). Il determinante di una matrice triangolare a blocchi è il prodotto dei determinanti diagonali, quindi

$$\log |\det J| = \sum_{j=d+1}^D s_j(\mathbf{z}_{1:d}),$$

calcolabile in $\mathcal{O}(D)$ **qualunque cosa siano s e t** , che possono essere reti profonde arbitrarie.

L'inversione è esplicita e costa quanto il forward, un solo passaggio:

$$\mathbf{z}_{1:d} = \mathbf{x}_{1:d}, \quad \mathbf{z}_{d+1:D} = (\mathbf{x}_{d+1:D} - t(\mathbf{x}_{1:d})) \odot \exp(-s(\mathbf{x}_{1:d})),$$

perché $\mathbf{x}_{1:d} = \mathbf{z}_{1:d}$ è disponibile subito e permette di ricalcolare s e t .

Due dettagli: s e t *non* devono essere invertibili — è il layer a esserlo per costruzione, ed è ciò che permette di usare reti espressive dentro un flow. E poiché metà delle componenti resta ferma, si *alternano* le partizioni fra layer consecutivi (in RealNVP con maschere a scacchiera e a canali), altrimenti quelle dimensioni non verrebbero mai trasformate.

D4. *Differenza fra MAF e IAF: in quali condizioni preferiresti uno o l'altro?*

Entrambi sono flow autoregressivi con trasformazione affine per componente, ma condizionano su variabili diverse.

$$\text{MAF: } x_i = z_i \sigma_i(\mathbf{x}_{<i}) + \mu_i(\mathbf{x}_{<i}), \quad \text{IAF: } x_i = z_i \sigma_i(\mathbf{z}_{<i}) + \mu_i(\mathbf{z}_{<i}).$$

La differenza è quale variabile compare nei condizionanti, e da lì discende tutto:

	MAF	IAF
Valutare $p(\mathbf{x})$ (densità)	$\mathcal{O}(1)$ passaggio	$\mathcal{O}(D)$ sequenziale
Campionare	$\mathcal{O}(D)$ sequenziale	$\mathcal{O}(1)$ passaggio

Il motivo: nel MAF, dato $\mathbf{x}$ tutti i condizionanti sono noti in parallelo, quindi la densità si valuta in un colpo; ma per campionare serve generare x_1 , poi x_2 da x_1 , e così via. Nell'IAF è esattamente il contrario, perché i condizionanti sono le $\mathbf{z}$, tutte disponibili all'inizio.

Quindi: **MAF** per *density estimation* (dove si valuta la likelihood su molti dati e si campiona poco); **IAF** per *generazione veloce*, e in particolare come distribuzione variazionale in un VAE, dove serve campionare e valutare la densità del *proprio* campione — entrambe $\mathcal{O}(1)$ in quel caso d'uso.

Il caso da citare è *Parallel WaveNet*: si addestra un MAF (lento a campionare ma addestrabile in parallelo) e se ne distilla un IAF studente per la generazione in tempo reale. Il coupling layer di RealNVP è il compromesso che è $\mathcal{O}(1)$ in entrambe le direzioni, al prezzo di minore espressività per layer.

D5. *Cos'è un Continuous Normalizing Flow? Mostra la formula per il log-determinante. Vantaggi e costi.*

Invece di comporre un numero finito di trasformazioni discrete, si definisce la dinamica con un'ODE:

$$\frac{d\mathbf{z}(t)}{dt} = h_\theta(\mathbf{z}(t), t), \quad \mathbf{x} = \mathbf{z}(1), \quad \mathbf{z}(0) \sim p_Z.$$

Il teorema dell'*instantaneous change of variables* (Chen et al. 2018) sostituisce il log-determinante con una traccia:

$$\frac{\partial \log p(\mathbf{z}(t))}{\partial t} = -\text{tr}\left(\frac{\partial h_\theta}{\partial \mathbf{z}(t)}\right) \implies \log p(\mathbf{x}) = \log p_Z(\mathbf{z}(0)) - \int_0^1 \text{tr}\left(\frac{\partial h_\theta}{\partial \mathbf{z}}\right) dt.$$

Il guadagno concettuale è grosso: la *traccia* costa $\mathcal{O}(d)$ per valutazione (o si stima con Hutchinson), mentre il determinante costa $\mathcal{O}(d^3)$. Sparisce quindi ogni vincolo architetturale su h_θ : non serve che sia triangolare, non serve che sia invertibile — l'invertibilità viene dall'ODE, che è reversibile integrando all'indietro.

Vantaggi: espressività senza restrizioni strutturali, profondità “continua” e adattiva, memoria costante in backward grazie al metodo aggiunto (adjoint), che non memorizza le attivazioni intermedie.

Costi: si paga con il solver numerico. Il numero di valutazioni della funzione (NFE) non è controllabile a priori e cresce quando la dinamica diventa rigida; l'accuratezza dipende dalla tolleranza scelta; il training è sensibilmente più lento dei flow discreti.

Il seguito della storia: FJORD usa Hutchinson per rendere praticabile il tutto, e il *flow matching* elimina l'ODE dal training regredendo direttamente il campo di velocità — l'ODE resta solo a inferenza.

D6. *Perché serve la dequantizzazione su immagini? Cosa succede se non la fai?*

Le immagini sono *discrete*: ogni pixel assume valori in $\{0, \dots, 255\}$. Un flow modella una densità *continua*, e una densità continua su un supporto discreto è degenerare.

Se non si dequantizza, il modello può mandare la likelihood a $+\infty$ concentrando massa arbitrariamente stretta sui 256 valori ammessi: si formano picchi di densità sempre più alti e sottili, la log-likelihood diverge e il numero riportato è privo di significato. Non è un problema numerico marginale ma una patologia dell'obiettivo.

La *uniform dequantization* aggiunge rumore uniforme, $\mathbf{y} = \mathbf{x} + \mathbf{u}$ con $\mathbf{u} \sim \mathcal{U}[0, 1]^D$, prima della normalizzazione. Il risultato chiave (Theis et al. 2016) è che la log-likelihood del modello continuo su $\mathbf{y}$ è un *lower bound* della log-likelihood del modello discreto su $\mathbf{x}$:

$$\mathbb{E}_{\mathbf{u}}[\log p_{\text{model}}(\mathbf{x} + \mathbf{u})] \leq \log P_{\text{model}}(\mathbf{x}).$$

Quindi il numero riportato è onesto e confrontabile fra modelli, e va espresso in *bits per dimension* proprio per rendere il confronto significativo.

La *variational dequantization* (Flow++) migliora il bound sostituendo l'uniforme con una distribuzione $q(\mathbf{u} \mid \mathbf{x})$ appresa, tipicamente essa stessa un flow: il rumore uniforme è una scelta arbitraria e non ottimale, e apprenderlo stringe il bound in modo misurabile.

D7. Confronta normalizing flow, VAE e diffusion sulla capacità di calcolare $p_\theta(\mathbf{x})$ esatta.

	$p_\theta(\mathbf{x})$	note
Flow	esatta	cambio di variabile; costo di un forward più i log-det
VAE	bound (ELBO)	gap = $D_{\text{KL}}(q_\phi \ p_\theta(\mathbf{z} \mid \mathbf{x}))$, non misurabile direttamente
Diffusion	bound (ELBO)	gap tipicamente piccolo; via probability flow ODE diventa esatta
GAN	nessuna	modello implicito

Precisazioni che rendono la risposta completa. Il VAE può stringere il bound con l'*importance weighted* ELBO (IWAE), che con K campioni dà un bound monotonamente più stretto e converge a $\log p(\mathbf{x})$ per $K \rightarrow \infty$; ma resta un bound a costo finito.

La diffusion ha un ELBO gerarchico, e la loss semplificata usata in pratica *non* è l'ELBO — si scartano i pesi λ_t — quindi il modello addestrato non è quello che ottimizza la likelihood. Però la formulazione a probability flow ODE la rende un continuous normalizing flow a tutti gli effetti, e da lì la likelihood *esatta* è calcolabile, al prezzo di integrare l'ODE.

Il punto trasversale da fare: la likelihood esatta ha un costo in espressività, come dice la domanda successiva. Ed è utile ricordare che una likelihood alta non implica buoni campioni — il risultato di Theis, van den Oord e Bethge del 2016 su questa dissociazione è citabile e fa buona impressione.

D8. Cos'è la stima Hutchinson della traccia, e perché è utile per i residual flows?

È uno stimatore stocastico non distorto della traccia di una matrice, che non richiede di costruire la matrice:

$$\text{tr}(\mathbf{A}) = \mathbb{E}_{p(\mathbf{v})}[\mathbf{v}^\top \mathbf{A} \mathbf{v}],$$

per qualunque $\mathbf{v}$ con media nulla e covarianza identità (tipicamente Rademacher o gaussiana). La dimostrazione è immediata: $\mathbb{E}[\mathbf{v}^\top \mathbf{A} \mathbf{v}] = \mathbb{E}[\sum_{ij} v_i A_{ij} v_j] = \sum_{ij} A_{ij} \mathbb{E}[v_i v_j] = \sum_i A_{ii}$.

Perché è decisivo: calcolare $\mathbf{v}^\top \mathbf{J} \mathbf{v}$ richiede un solo prodotto vettore-Jacobiano, che l'autodiff fornisce al costo di *un* backward pass, $\mathcal{O}(d)$. Calcolare la traccia esatta richiederebbe d backward pass, uno per riga, cioè $\mathcal{O}(d^2)$: su immagini con d nell'ordine delle migliaia è impraticabile.

Serve nei *residual flow* $\mathbf{x} = \mathbf{z} + g_\theta(\mathbf{z})$, dove l'invertibilità è garantita imponendo $\text{Lip}(g_\theta) < 1$ (via spectral normalization) e il log-determinante si espande in serie:

$$\log |\det(\mathbf{I} + \mathbf{J}_g)| = \text{tr} \log(\mathbf{I} + \mathbf{J}_g) = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k} \text{tr}(\mathbf{J}_g^k),$$

serie convergente proprio grazie al vincolo di Lipschitz. Ogni traccia si stima con Hutchinson, e la serie si tronca in modo non distorto con l'*estimatore russian roulette*. Stesso ruolo nei CNF (FFJORD), dove rende praticabile l'integrale della traccia.

Il prezzo, da nominare: si guadagna scalabilità e si perde determinismo. La likelihood stimata ha varianza, quindi i confronti fra modelli vanno fatti con molti campioni.

Capitolo 23 — Diffusion Models

D1. *Scrivi il forward process $q(\mathbf{z}_t | \mathbf{z}_{t-1})$ di DDPM. Deriva la forma chiusa $q(\mathbf{z}_t | \mathbf{x}_0)$.*

$$q(\mathbf{z}_t | \mathbf{z}_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} \mathbf{z}_{t-1}, \beta_t \mathbf{I}), \quad \mathbf{z}_0 = \mathbf{x}_0.$$

Con $\alpha_t = 1 - \beta_t$ e $\bar{\alpha}_t = \prod_{s \leq t} \alpha_s$, per induzione. Ipotesi induttiva $\mathbf{z}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \epsilon'$. Allora

$$\mathbf{z}_t = \sqrt{\alpha_t} \mathbf{z}_{t-1} + \sqrt{\beta_t} \epsilon'' = \sqrt{\alpha_t \bar{\alpha}_{t-1}} \mathbf{x}_0 + \underbrace{\sqrt{\alpha_t (1 - \bar{\alpha}_{t-1})} \epsilon' + \sqrt{1 - \alpha_t} \epsilon''}_{\text{somma di due gaussiane indipendenti}}.$$

Due gaussiane indipendenti a media nulla si sommano in una sola con varianza pari alla somma:

$$\alpha_t (1 - \bar{\alpha}_{t-1}) + 1 - \alpha_t = 1 - \alpha_t \bar{\alpha}_{t-1} = 1 - \bar{\alpha}_t,$$

da cui

$$q(\mathbf{z}_t | \mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}).$$

Il passaggio da non saltare è proprio la somma delle varianze: è lì che il prodotto telescopico si semplifica.

Perché conta: si può campionare $\mathbf{z}_t$ per un t qualunque *in un colpo solo*, senza simulare la catena. È ciò che rende il training praticabile — un solo timestep casuale per esempio.

D2. *Perché possiamo assumere che la reverse $p_\theta(\mathbf{z}_{t-1} | \mathbf{z}_t)$ sia gaussiana?*

Per un risultato classico sui processi di diffusione (Feller, 1949): se il passo forward è gaussiano con varianza *piccola*, la transizione inversa è anch'essa approssimativamente gaussiana. L'approssimazione è tanto migliore quanto più β_t è piccolo — ed è la ragione per cui si usano T grandi (1000) con β_t dell'ordine di 10^{-4} – 10^{-2} : non è una scelta arbitraria, è il prezzo per rendere lecita la parametrizzazione.

Il ragionamento intuitivo: se il passo forward sposta pochissimo, la distribuzione dei possibili predecessori di $\mathbf{z}_t$ è concentrata in un intorno piccolo, dove qualunque densità regolare è ben approssimata da una gaussiana (localmente, un'espansione al second'ordine della log-densità).

Attenzione a non confondere due cose. La reverse *vera* $q(\mathbf{z}_{t-1} | \mathbf{z}_t)$ è **intrattabile**, perché per Bayes richiederebbe $q(\mathbf{z}_{t-1})$, cioè la marginale, che dipende da p_{data} . Ma la reverse *condizionata su $\mathbf{x}_0$* , $q(\mathbf{z}_{t-1} | \mathbf{z}_t, \mathbf{x}_0)$, è gaussiana in forma *esatta e chiusa*, con

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{z}_t, \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t.$$

Ed è su questa che si costruisce l'ELBO: in training $\mathbf{x}_0$ c'è, a inferenza no, e per questo serve la rete.

D3. *Deriva l'ELBO per la catena di diffusione. Quali sono i tre tipi di termini?*

Trattando la catena come un VAE gerarchico con encoder fisso q :

$$\log p_\theta(\mathbf{x}_0) \geq \mathbb{E}_q \left[\log \frac{p_\theta(\mathbf{x}_0, \mathbf{z}_{1:T})}{q(\mathbf{z}_{1:T} | \mathbf{x}_0)} \right].$$

Espandendo i prodotti e usando l'identità $q(\mathbf{z}_t | \mathbf{z}_{t-1}) = q(\mathbf{z}_t | \mathbf{z}_{t-1}, \mathbf{x}_0)$ (la catena è markoviana, quindi condizionare in più su $\mathbf{x}_0$ non cambia nulla) e poi Bayes per riscrivere in termini di $q(\mathbf{z}_{t-1} | \mathbf{z}_t, \mathbf{x}_0)$, si ottiene

$$-\text{ELBO} = \mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{z}_T | \mathbf{x}_0) \| p(\mathbf{z}_T))}_{L_T} + \sum_{t=2}^T \underbrace{D_{\text{KL}}(q(\mathbf{z}_{t-1} | \mathbf{z}_t, \mathbf{x}_0) \| p_\theta(\mathbf{z}_{t-1} | \mathbf{z}_t))}_{L_{t-1}} \underbrace{- \log p_\theta(\mathbf{x}_0 | \mathbf{z}_1)}_{L_0} \right].$$

I tre tipi:

- L_T , termine *prior matching*: misura quanto $q(\mathbf{z}_T | \mathbf{x}_0)$ si avvicina a $\mathcal{N}(0, \mathbf{I})$. Non contiene θ , quindi è una **costante** rispetto all'ottimizzazione — e con una schedule ben scelta è praticamente nullo.
- L_{t-1} , termini di *denoising matching*: KL fra due gaussiane, quindi in forma chiusa. Sono il cuore dell'obiettivo.
- L_0 , termine di *ricostruzione* finale, tipicamente una likelihood discretizzata sui valori dei pixel.

La struttura è quella di un VAE con T livelli latenti, con la differenza che l'“encoder” è fissato a priori e non appreso.

D4. *Mostra come si arriva alla loss simplified a partire dall'ELBO. Cosa si trascura?*

Poiché L_{t-1} è una KL fra gaussiane con la stessa varianza, si riduce a una distanza fra le medie:

$$L_{t-1} = \frac{1}{2\sigma_t^2} \|\tilde{\boldsymbol{\mu}}_t(\mathbf{z}_t, \mathbf{x}_0) - \boldsymbol{\mu}_\theta(\mathbf{z}_t, t)\|^2 + \text{cost.}$$

Dal diffusion kernel, $\mathbf{x}_0 = (\mathbf{z}_t - \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon})/\sqrt{\bar{\alpha}_t}$. Sostituendo in $\tilde{\boldsymbol{\mu}}_t$ e semplificando si ottiene

$$\tilde{\boldsymbol{\mu}}_t = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{z}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon} \right),$$

e si sceglie la stessa forma per il modello, con $\boldsymbol{\epsilon}_\theta$ al posto di $\boldsymbol{\epsilon}$. La differenza delle medie diventa allora proporzionale alla differenza dei rumori:

$$L_{t-1} = \frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{z}_t, t)\|^2.$$

La *simplified loss* nasce buttando via il coefficiente:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}} [\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, t)\|^2].$$

Che cosa si trascura, in ordine: i pesi λ_t dipendenti dal timestep; il termine L_0 ; e il termine costante L_T .

Il commento che conta: la riponderazione **non è neutra**. I pesi originali danno molta importanza ai t piccoli (basso rumore, dettagli fini); pesare tutti i t ugualmente sposta l'attenzione verso i t grandi, cioè verso la struttura globale. Empiricamente questo migliora la qualità percettiva e *peggiora* la likelihood — ed è il motivo per cui il modello addestrato non è quello che massimizza l'ELBO. È una decisione di ingegneria, non un'approssimazione innocua, e saperlo dire distingue la risposta.

D5. *Differenza fra classifier guidance e classifier-free guidance. Vantaggi e svantaggi.*

Classifier guidance (Dhariwal e Nichol): si addestra un modello incondizionato più un classificatore *separato* $p_\phi(\mathbf{y} | \mathbf{z}_t)$ che opera su input rumorosi, e a sampling si modifica la media:

$$\boldsymbol{\mu}' = \boldsymbol{\mu}_\theta + \gamma \sigma_t^2 \nabla_{\mathbf{z}_t} \log p_\phi(\mathbf{y} | \mathbf{z}_t),$$

giustificata da $\nabla \log p(\mathbf{z}_t | \mathbf{y}) = \nabla \log p(\mathbf{z}_t) + \nabla \log p(\mathbf{y} | \mathbf{z}_t)$.

Classifier-free guidance (Ho e Salimans): si addestra un *unico* modello condizionale con dropout della condizione (con probabilità ρ si sostituisce $\mathbf{y}$ con un token nullo), così che impari sia il condizionale sia l'incondizionato. A inferenza:

$$\hat{\boldsymbol{\epsilon}} = (1 + \gamma) \boldsymbol{\epsilon}_\theta(\mathbf{z}_t, t, \mathbf{y}) - \gamma \boldsymbol{\epsilon}_\theta(\mathbf{z}_t, t, \emptyset),$$

equivalentemente $(1 - w)\epsilon_\theta + w\epsilon_y$ con $w = 1 + \gamma$ (è la forma usata negli handout del corso).

Confronto. La classifier guidance richiede un modello in più addestrato *su dati rumorosi* — cosa poco naturale, i cui gradienti sono spesso poco informativi o instabili — e mal si presta a condizioni non categoriche come il testo. La classifier-free non richiede modelli aggiuntivi, funziona con qualunque tipo di condizionamento ed è diventata lo standard (Stable Diffusion, Imagen, DALL·E 2). Il suo costo è raddoppiare le valutazioni della rete a sampling.

Il trade-off comune a entrambe, da nominare sempre: aumentando γ i campioni aderiscono di più alla condizione ma diventano meno *diversi*, perché il sampling viene spinto verso i pochi modi più compatibili. Fedeltà contro varietà.

D6. Perché DDIM è più veloce di DDPM senza riaddestrare il modello?

Perché cambia solo la *procedura di sampling*, non il modello. DDIM osserva che si può costruire una famiglia di processi *non markoviani* che condividono le stesse marginali $q(z_t | x_0)$ di DDPM — e poiché la loss dipende solo da quelle marginali, la stessa rete ϵ_θ addestrata per DDPM è ottima anche per tutti i membri della famiglia.

L'aggiornamento generale è

$$z_{t-1} = \underbrace{\sqrt{\bar{\alpha}_{t-1}} \frac{z_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta}{\sqrt{\bar{\alpha}_t}}}_{\hat{x}_0} + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \epsilon_\theta + \sigma_t w,$$

e $\sigma_t = 0$ dà il caso *deterministico*, che è DDIM.

Due conseguenze. Essendo deterministico, il processo è di fatto un'ODE, e un'ODE si può integrare con *passi grandi*: si può usare un sottoinsieme dei timestep (per esempio 50 su 1000) e accettare un errore di discretizzazione modesto. In DDPM lo stocastico va reiniettato a ogni passo e saltare passi degrada molto di più.

Inoltre la mappa $z_T \rightarrow x_0$ diventa *deterministica e invertibile*, il che abilita la latent inversion e quindi l'editing semantico e l'interpolazione fra immagini reali — cose che con DDPM non si fanno.

Da dire per onestà: a parità di passi la qualità DDIM non è sempre superiore; il guadagno è nel rapporto qualità/passi, ed è netto sotto i 100 passi.

D7. Cos'è il latent diffusion e perché è importante computazionalmente?

Si esegue la diffusione non nello spazio dei pixel ma in uno spazio latente compresso, appreso da un autoencoder addestrato a parte:

$$h = E(x) \rightarrow \text{diffusion su } h \rightarrow \tilde{x} = D(h).$$

L'autoencoder (tipicamente un VQ-VAE o un KL-regularized AE, addestrato con perceptual loss e un termine avversariale) comprime per esempio $512 \times 512 \times 3$ in $64 \times 64 \times 4$: un fattore 48 in numero di elementi.

Perché conta computazionalmente: la diffusione richiede *centinaia* di valutazioni della rete, quindi ogni risparmio per valutazione si moltiplica. Lavorare su tensori 48 volte più piccoli riduce di altrettanto memoria e calcolo per passo, e rende la generazione ad alta risoluzione praticabile su una singola GPU consumer — è la ragione tecnica per cui Stable Diffusion è potuto esistere come modello aperto.

C'è anche un argomento qualitativo, non solo di costo: il latente cattura struttura *semantica* scartando la ridondanza percettiva dei pixel, quindi la diffusione lavora su ciò che conta invece di spendere capacità su dettagli ad alta frequenza che l'autoencoder ricostruisce comunque.

Il compromesso da nominare: la qualità finale è limitata dall'autoencoder, che introduce artefatti propri (tipicamente su testo, volti piccoli e dettagli fini). Se il decoder non sa ricostruirlo, nessuna diffusione nel latente potrà recuperarlo.

D8. Confronta VAE, GAN, Flow e Diffusion su: likelihood, qualità campioni, stabilità, velocità.

	VAE	GAN	Flow	Diffusion
Likelihood	bound	nessuna	esatta	bound (esatta via ODE)
Qualità campioni	sfocata	alta	media	molto alta
Stabilità training	alta	bassa	alta	alta
Velocità sampling	1 passo	1 passo	1 passo	lenta , 10^1 – 10^3 passi
Inferenza $\mathbf{x} \rightarrow \mathbf{z}$	sì	no	sì (esatta)	sì (via ODE)
Latente compresso	sì	sì	no	no (sì in latent diffusion)

La sintesi da dire a voce: ciascuno rinuncia a qualcosa. Il flow tiene la likelihood esatta e paga in espressività, perché invertibilità e dimensione preservata vincolano l’architettura. Il GAN tiene la qualità e la velocità e paga con instabilità, assenza di likelihood e mode collapse. Il VAE tiene tutto tranne la qualità. La diffusion tiene qualità, stabilità e copertura dei modi, e paga solo sulla velocità — ed è per questo che ha vinto: la velocità è l’unico difetto *attaccabile con l’ingegneria*, ed è esattamente quello che DDIM, distillazione e flow matching hanno fatto.

D9. In che senso un diffusion model è un denoising autoencoder gerarchico?

Su tre livelli di analogia, sempre più stringenti.

Per passo: la rete riceve $\mathbf{z}_t$ (input corrotto) e deve stimare il rumore aggiunto, cioè ricostruire l’informazione pulita. È letteralmente il compito di un DAE, con la differenza che il livello di corruzione varia.

Gerarchia: invece di un singolo livello di rumore fissato a mano, ce ne sono T ordinati da quasi-pulito a rumore puro. Ogni “strato” della gerarchia risolve lo stesso problema a una scala diversa, e la condivisione dei pesi (una sola rete condizionata su t) fa sì che sia lo stesso denoiser a coprirli tutti.

Formalmente: è un VAE gerarchico in cui l’encoder $q(\mathbf{z}_{1:T} | \mathbf{x}_0)$ è **fisso e non appreso**, tutti i latenti hanno la stessa dimensione dei dati, e l’obiettivo è l’ELBO corrispondente. È la lettura che spiega perché l’ELBO ha esattamente quella forma.

E c’è il collegamento che chiude il cerchio con il Cap. 19: un DAE ottimo stima la score, $\nabla \log p_\sigma(\hat{\mathbf{x}}) = (\mathbb{E}[\mathbf{x} | \hat{\mathbf{x}}] - \hat{\mathbf{x}})/\sigma^2$. Quindi un diffusion model è un DAE gerarchico *che impara una famiglia di score a livelli di rumore crescenti* — ed è precisamente il ponte con il capitolo successivo.

Capitolo 24 — Score matching e flow matching

D1. Cos’è la score function di una distribuzione e perché è utile per modelli energy-based?

$$\mathbf{s}(\mathbf{x}) = \nabla_{\mathbf{x}} \log p(\mathbf{x}).$$

È un campo vettoriale che in ogni punto indica la direzione di massima crescita della log-densità, cioè punta *verso* le regioni dense — verso i dati.

L’utilità per i modelli energy-based è che elimina la partition function. Con $p_\theta(\mathbf{x}) = e^{-f_\theta(\mathbf{x})}/Z_\theta$:

$$\nabla_{\mathbf{x}} \log p_\theta(\mathbf{x}) = -\nabla_{\mathbf{x}} f_\theta(\mathbf{x}) - \underbrace{\nabla_{\mathbf{x}} \log Z_\theta}_{=0},$$

perché Z_θ è un integrale su tutto lo spazio e quindi *non dipende da $\mathbf{x}$* . Il termine intrattabile sparisce derivando. Si trasforma così l’apprendimento di una densità normalizzata nell’apprendimento di un campo vettoriale, che è un problema di regressione.

E la score basta per campionare, tramite *Langevin dynamics*:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{\eta}{2} \mathbf{s}_\theta(\mathbf{x}_k) + \sqrt{\eta} \boldsymbol{\epsilon}_k,$$

che ha p come distribuzione stazionaria per $\eta \rightarrow 0$. La deriva spinge verso l'alto della densità, il rumore impedisce il collasso su un solo modo e preserva la diversità.

Il limite, da citare per completezza: la score è mal stimata dove i dati sono radi, e in alta dimensione quasi tutto lo spazio è rado. È il problema che il *multi-scale noise conditioning* risolve — perturbare i dati a più livelli riempie lo spazio e rende la score stimabile ovunque.

D2. Deriva la loss del Denoising Score Matching per rumore gaussiano. Mostra che è equivalente alla loss simplified di DDPM.

Sia $\tilde{\mathbf{x}} = \mathbf{x} + \sigma \boldsymbol{\epsilon}$, quindi $p(\tilde{\mathbf{x}} | \mathbf{x}) = \mathcal{N}(\mathbf{x}, \sigma^2 \mathbf{I})$. La loss DSM regredisce la rete sulla score del kernel di corruzione, che è nota:

$$\mathcal{L}_{\text{DSM}} = \mathbb{E}_{\mathbf{x}, \tilde{\mathbf{x}}} [\|\mathbf{s}_\theta(\tilde{\mathbf{x}}) - \nabla_{\tilde{\mathbf{x}}} \log p(\tilde{\mathbf{x}} | \mathbf{x})\|^2].$$

Calcolando il target:

$$\log p(\tilde{\mathbf{x}} | \mathbf{x}) = -\frac{\|\tilde{\mathbf{x}} - \mathbf{x}\|^2}{2\sigma^2} + \text{cost} \Rightarrow \nabla_{\tilde{\mathbf{x}}} \log p(\tilde{\mathbf{x}} | \mathbf{x}) = -\frac{\tilde{\mathbf{x}} - \mathbf{x}}{\sigma^2} = -\frac{\boldsymbol{\epsilon}}{\sigma},$$

quindi

$$\mathcal{L}_{\text{DSM}} = \mathbb{E} [\|\mathbf{s}_\theta(\tilde{\mathbf{x}}) + \frac{\boldsymbol{\epsilon}}{\sigma}\|^2].$$

Il risultato non ovvio (Vincent 2011) è che questa loss ha lo *stesso minimizzatore* della score matching sulla densità perturbata p_σ , a meno di una costante indipendente da θ : si sta imparando $\nabla \log p_\sigma$, non solo la score del kernel.

Equivalenza con DDPM. Nel diffusion $\mathbf{z}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}$, e per la formula di Tweedie vale

$$\nabla_{\mathbf{z}_t} \log p_t(\mathbf{z}_t) = -\frac{\boldsymbol{\epsilon}}{\sqrt{1 - \bar{\alpha}_t}} \implies \boldsymbol{\epsilon}_\theta(\mathbf{z}_t, t) = -\sqrt{1 - \bar{\alpha}_t} \mathbf{s}_\theta(\mathbf{z}_t, t).$$

Sostituendo, $\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\|^2 = (1 - \bar{\alpha}_t) \|\mathbf{s} + \boldsymbol{\epsilon} / \sqrt{1 - \bar{\alpha}_t}\|^2$: le due loss coincidono a meno del fattore $(1 - \bar{\alpha}_t)$, cioè della riponderazione già discussa. **La loss di DDPM è denoising score matching riparametrizzato**, a T livelli di rumore.

D3. Scrivi forward e reverse SDE. Cosa rappresenta il termine $g^2(t) \nabla \log p_t$ nella reverse?

$$\text{forward: } d\mathbf{x} = \mathbf{f}(\mathbf{x}, t) dt + g(t) d\mathbf{w},$$

$$\text{reverse (Anderson 1982): } d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g^2(t) \nabla_{\mathbf{x}} \log p_t(\mathbf{x})] dt + g(t) d\bar{\mathbf{w}},$$

con $\bar{\mathbf{w}}$ moto browniano nel tempo invertito. Casi notevoli: *Variance Exploding* (NCSN) con $\mathbf{f} = 0$ e g crescente; *Variance Preserving* (DDPM) con $\mathbf{f} = -\frac{1}{2}\beta(t)\mathbf{x}$ e $g = \sqrt{\beta(t)}$.

Il termine $-g^2(t) \nabla \log p_t$ è il **termine di correzione che inverte la diffusione**. Interpretazione fisica: il processo forward diffonde, cioè sparge probabilità verso l'esterno appiattendolo la densità; per invertirlo serve un drift che riporti la massa *verso le regioni dense*, e la direzione di massima densità è esattamente la score. Il coefficiente g^2 dice che tanto più forte è stata la diffusione, tanto più forte deve essere la contropinta.

È anche l'unico termine *incognito*: $\mathbf{f}$ e g sono scelti da noi nel definire il forward, mentre $\nabla \log p_t$ dipende dalla distribuzione dei dati. Tutta la rete neurale serve a stimare quello, e questa è la risposta compatta alla domanda “che cosa impara un diffusion model?”.

D4. *Cos'è la probability flow ODE e perché collega diffusion e normalizing flows?*

Rimuovendo il termine browniano e dimezzando il coefficiente della score si ottiene un'ODE deterministica:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g^2(t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x}).$$

Il fatto notevole è che questa ODE ha **le stesse marginali** p_t della reverse SDE, per ogni t . Non le stesse traiettorie — quelle sono diverse, una è stocastica e l'altra no — ma la stessa sequenza di distribuzioni. Sono due descrizioni dinamiche dello stesso trasporto di probabilità.

Il collegamento con i normalizing flow è immediato: un'ODE deterministica e invertibile che trasporta una distribuzione semplice in una complessa è un continuous normalizing flow. Quindi un diffusion model, letto attraverso la probability flow ODE, definisce implicitamente una mappa biettiva fra spazio latente e spazio dati.

Le conseguenze pratiche sono tre e vale la pena elencarle: si può calcolare la *likelihood esatta* con la formula dell'instantaneous change of variables; si può *invertire* il flusso integrando all'indietro, ottenendo una codifica $\mathbf{x} \rightarrow \mathbf{z}$ deterministica utile per l'editing; e si possono usare solver ODE di ordine alto per campionare in pochi passi (è l'idea di DPM-Solver e affini, e la giustificazione teorica di DDIM).

È anche il ponte concettuale verso il flow matching: se il trasporto è governato da un campo di velocità, perché non impararlo direttamente?

D5. *Spiega l'idea di Flow Matching: cosa si regredisce e perché si usa un conditional flow matching?*

Si sceglie un cammino di distribuzioni $(p_\tau)_{\tau \in [0,1]}$ da una sorgente semplice p_0 ai dati p_1 , e si impara il *campo di velocità* $\mathbf{v}_\theta(\mathbf{z}, \tau)$ che trasporta lungo quel cammino:

$$\frac{d\mathbf{z}_\tau}{d\tau} = \mathbf{v}_\theta(\mathbf{z}_\tau, \tau).$$

L'obiettivo ideale sarebbe

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{\tau, \mathbf{z}_\tau \sim p_\tau} [\|\mathbf{v}_\theta(\mathbf{z}_\tau, \tau) - \mathbf{u}_\tau(\mathbf{z}_\tau)\|^2],$$

ma la velocità marginale $\mathbf{u}_\tau$ **non è osservabile**: dipende dalla distribuzione dei dati, che è esattamente ciò che non si conosce.

La soluzione è regredire su un target *condizionale*, che invece è noto in forma chiusa una volta scelto il cammino:

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{\tau, \mathbf{x}, \mathbf{z}_\tau \sim q_\tau(\cdot|\mathbf{x})} [\|\mathbf{v}_\theta(\mathbf{z}_\tau, \tau) - \mathbf{u}_\tau(\mathbf{z}_\tau | \mathbf{x})\|^2].$$

Il teorema (Lipman et al. 2023) garantisce che i due obiettivi hanno lo *stesso gradiente* rispetto a θ , perché la velocità marginale è la media condizionale di quelle condizionali:

$$\mathbf{u}_\tau(\mathbf{z}) = \mathbb{E}[\mathbf{u}_\tau(\mathbf{z}_\tau | \mathbf{x}) | \mathbf{z}_\tau = \mathbf{z}].$$

La logica è la stessa del denoising score matching e della loss di DDPM: si regredisce su una quantità condizionale calcolabile, e la media sui condizionamenti restituisce l'oggetto marginale corretto. È un pattern che vale la pena nominare come tale, perché è il filo che unifica tutta la Parte IV.

D6. *Mostra il path rectified flow e calcola la velocità conditional. Perché le traiettorie sono rettilinee?*

Interpolazione lineare fra una sorgente $\mathbf{z}_0 \sim p_0$ e un dato $\mathbf{x} \sim p_{\text{data}}$:

$$\mathbf{z}_\tau = (1 - \tau)\mathbf{z}_0 + \tau\mathbf{x}.$$

Derivando rispetto a τ :

$$\mathbf{u}_\tau(\mathbf{z}_\tau | \mathbf{x}, \mathbf{z}_0) = \frac{d\mathbf{z}_\tau}{d\tau} = \mathbf{x} - \mathbf{z}_0,$$

costante rispetto a τ . Il bersaglio della regressione è quindi semplicemente la differenza fra il dato e la sorgente: nessuna score, nessun kernel reverse, nessun ELBO.

Le traiettorie *condizionali* sono rettilinee perché la velocità è costante: la soluzione dell'ODE con velocità costante è un segmento. Esempio: $z_0 = 0.5$, $x = 2.0$, $u = 1.5$; con quattro passi di Eulero, $\Delta\tau = 0.25$, si ottiene $0.5 \rightarrow 0.875 \rightarrow 1.25 \rightarrow 1.625 \rightarrow 2.0$, cioè l'arrivo *esatto*. Il metodo di Eulero è esatto su traiettorie rettilinee perché l'errore di troncamento dipende dalla derivata seconda, che è nulla.

Da qui il vantaggio sul numero di passi. Ma va detta l'avvertenza: sono rettilinee le traiettorie *condizionali*, cioè per una coppia fissata. Il campo *marginale* appreso è la media delle velocità condizionali che passano per un punto, e quella media in generale curva. Le procedure di *reflow* iterano la costruzione — si rigenerano coppie $(\mathbf{z}_0, \mathbf{x})$ accoppiate dal modello stesso e si riaddestra — proprio per raddrizzare progressivamente il campo marginale, arrivando a generare in 1–4 passi.

D7. *In che senso flow matching e diffusion sono lo stesso modello sotto diverse parametrizzazioni?*

Nel senso preciso che i cammini gaussiani del flow matching *includono* quelli della diffusion come caso particolare. Con

$$\mathbf{z}_\tau = \alpha_\tau \mathbf{x} + \sigma_\tau \boldsymbol{\epsilon}, \quad \alpha_0 = 0, \sigma_0 = 1, \alpha_1 = 1, \sigma_1 = 0,$$

la velocità condizionale è $\mathbf{u}_\tau = \dot{\alpha}_\tau \mathbf{x} + \dot{\sigma}_\tau \boldsymbol{\epsilon}$, e scegliendo $\alpha_\tau = \sqrt{\bar{\alpha}_{t(\tau)}}$, $\sigma_\tau = \sqrt{1 - \bar{\alpha}_{t(\tau)}}$ si recupera esattamente il kernel di diffusione.

Ciò che cambia è *che cosa la rete predice*: $\boldsymbol{\epsilon}$ nella diffusion, la score $\mathbf{s}$ nello score matching, la velocità $\mathbf{v}$ nel flow matching. Ma le tre quantità sono legate da relazioni lineari con coefficienti dipendenti solo da τ — via Tweedie per la coppia $\boldsymbol{\epsilon}/\mathbf{s}$, e via la definizione del cammino per $\mathbf{v}$. Sono la stessa rete con una riscalatura dell'uscita.

E le due ricette di training sono quasi identiche in codice: campiona un dato, campiona un tempo, campiona rumore o sorgente, costruisci il punto interpolato, regredisce il target. Cambia solo il target e chi decide il cammino — nella diffusion lo detta il processo di rumore, nel flow matching lo si sceglie.

La differenza *sostanziale*, quella che sopravvive all'unificazione: nel flow matching il cammino è un grado di libertà progettuale. Si può scegliere rectified per la rettilineità, ottimo trasporto per accoppiare meglio sorgente e target, o cammini adattati alla geometria del problema (per esempio su varietà). La diffusion quella libertà non ce l'ha.

D8. *Vantaggi di flow matching in termini di numero di passi di sampling.*

DDPM richiede $\mathcal{O}(10^2)$ – $\mathcal{O}(10^3)$ passi; il flow matching con cammini rettilinei arriva a $\mathcal{O}(10)$, e con reflow e distillazione a 1–4.

Il motivo tecnico è l'errore di discretizzazione. Integrare $d\mathbf{z}/d\tau = \mathbf{v}_\theta$ con Eulero introduce un errore locale $\mathcal{O}(\Delta\tau^2)$ proporzionale alla *curvatura* della traiettoria. Su un cammino rettilineo la derivata seconda è nulla e l'errore sparisce: si può prendere un passo grande quanto si vuole. Su traiettorie curve, come quelle indotte dal processo di rumore della diffusion, ogni passo grande costa accuratezza.

Ci sono due vantaggi ulteriori da nominare. Il sampling è *deterministico* una volta scelto $\mathbf{z}_0$, quindi si può usare qualunque solver ODE — Eulero, midpoint, Runge-Kutta, o solver adattivi — e beneficiare di metodi di ordine superiore, cosa che con un processo stocastico non si può fare direttamente. E il numero di passi diventa un parametro di qualità regolabile a inferenza, senza riaddestrare.

L'onestà richiede due precisazioni: DDIM e i solver moderni (DPM-Solver) portano anche la diffusion a 10–20 passi, quindi il divario si è ridotto; e la distillazione a pochi passi funziona per entrambe le famiglie. Il vantaggio del flow matching è più netto sul *regime a 1–4 passi* e sulla semplicità concettuale del training.

Capitolo 25 — Causal Representation Learning

D1. Definisci formalmente cosa significa che una rappresentazione è *disentangled*.

Non esiste una definizione universalmente accettata, ed è la prima cosa onesta da dire. La formulazione più usata (Higgins et al. 2018, basata sulla teoria dei gruppi) è: sia $\mathcal{G} = \mathcal{G}_1 \times \dots \times \mathcal{G}_m$ un gruppo di simmetrie che agisce sul mondo e si decompone in sottogruppi indipendenti. Una rappresentazione $\mathbf{z} = g(\mathbf{x})$ è *disentangled* rispetto a quella decomposizione se esiste una partizione di $\mathbf{z}$ in sottospazi $\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}$ tale che l'azione di $\mathcal{G}_i$ modifichi solo $\mathbf{z}^{(i)}$ e lasci invariati gli altri.

La versione operativa e informale, che è quella che si usa in pratica: ogni dimensione latente corrisponde a un *fattore generativo* interpretabile — posizione, colore, dimensione — e cambiarla modifica quel solo fattore nell'immagine ricostruita, lasciando il resto invariato.

Metriche proposte: BetaVAE score, FactorVAE score, MIG (Mutual Information Gap), SAP, DCI. Il fatto che ce ne siano tante, e che non concordino fra loro, è un sintomo del problema definitorio.

Il punto critico da nominare: la definizione presuppone che esista una decomposizione *privilegiata* dei fattori. È esattamente ciò che il risultato di impossibilità mette in discussione.

D2. Cos'è il β -VAE e perché aumentare β dovrebbe promuovere il *disentanglement*?

Si pesa il termine KL dell'ELBO:

$$\mathcal{L}_\beta = \mathbb{E}_{q_\phi}[\log p_\theta(\mathbf{x} | \mathbf{z})] - \beta D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z})), \quad \beta > 1.$$

L'argomento a favore: il prior $p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$ è *fattorizzato*, quindi spingere q_ϕ verso di esso spinge verso un posterior a componenti indipendenti. Inoltre, aumentare β stringe il collo di bottiglia informativo — la KL è un limite superiore all'informazione che $\mathbf{z}$ può portare su $\mathbf{x}$ — costringendo il modello a usare poche dimensioni e a spenderle sui fattori di variazione più salienti, che spesso coincidono con quelli interpretabili.

Il costo è immediato: peggiora la ricostruzione. È un trade-off esplicito fra fedeltà e struttura del latente.

La critica va però fatta, ed è quella che la decomposizione della KL rende precisa: $\mathbb{E}_{p_{\text{data}}}[D_{\text{KL}}(q(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z}))]$ si scompone in *mutual information* $I(\mathbf{x}; \mathbf{z})$, *total correlation* $TC(\mathbf{z})$ e un termine di dimension-wise KL. Solo il *secondo* misura l'entanglement fra dimensioni; aumentando β si penalizzano tutti e tre insieme, quindi si sacrifica anche informazione utile. FactorVAE e β -TCVAE nascono per penalizzare selettivamente la sola total correlation.

E soprattutto: il *disentanglement* ottenuto è un effetto di bias induttivo, non di identificabilità — vedi la domanda seguente.

D3. Enuncia il risultato di impossibilità di Locatello et al. 2019. Quale assunto viene violato?

Enunciato: per qualunque distribuzione generativa con latenti a componenti indipendenti, esistono infinite trasformazioni $\mathbf{z} \mapsto \hat{\mathbf{z}}$ che **preservano la distribuzione marginale** ($\hat{\mathbf{z}}$ ha la stessa distribuzione di $\mathbf{z}$, con componenti ancora indipendenti) ma sono completamente *entangled* rispetto ai fattori originali. Poiché i due modelli inducono la stessa distribuzione sui dati, nessun obiettivo non supervisionato può distinguerli.

La costruzione: se $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$, che è invariante per rotazioni, allora per qualunque matrice ortogonale $\mathbf{R}$ la variabile $\hat{\mathbf{z}} = \mathbf{R}\mathbf{z}$ ha esattamente la stessa distribuzione, e il modello con decoder $\hat{g} = g \circ \mathbf{R}^{-1}$ genera esattamente la stessa $p(\mathbf{x})$. Ma se $\mathbf{z}$ separava colore e posizione, $\hat{\mathbf{z}}$ li mescola in ogni coordinata.

L'assunto violato è che l'*indipendenza marginale delle componenti identifichi i fattori*. Non li identifica: l'indipendenza è invariante per un intero gruppo di trasformazioni, e tutte le soluzioni dentro quel gruppo hanno la stessa likelihood, lo stesso ELBO, la stessa qualsiasi cosa calcolabile dai dati.

Come si esce, ed è la parte costruttiva: bisogna rompere la simmetria dall'esterno, con supervisione debole (coppie, gruppi, etichette parziali), struttura temporale (TCL), variabili ausiliarie (iVAE) o *interventi*. Ed è quest'ultima via che porta al causal representation learning.

Nota empirica dello stesso articolo, da citare: su una valutazione di oltre 12 000 modelli, i risultati di disentanglement dipendono più dal *seed* che dalla scelta del metodo o degli iperparametri.

D4. *Cos'è una variabile ausiliaria $\mathbf{u}$ in iVAE/TCL? Perché permette di superare l'impossibilità?*

$\mathbf{u}$ è una variabile osservata che *condiziona* il prior sui latenti: un indice temporale, un'etichetta di segmento, un indice di ambiente, la classe, il regime sperimentale. Il modello diventa

$$p(\mathbf{x}, \mathbf{z} | \mathbf{u}) = p_\theta(\mathbf{x} | \mathbf{z}) p_\lambda(\mathbf{z} | \mathbf{u}), \quad p_\lambda(\mathbf{z} | \mathbf{u}) = \prod_i p_i(z_i | \mathbf{u}),$$

con il prior condizionale tipicamente in famiglia esponenziale, i cui parametri naturali dipendono da $\mathbf{u}$.

Perché supera l'impossibilità: il prior non è più *isotropo*, quindi non è più invariante per rotazioni. Al variare di $\mathbf{u}$ le componenti cambiano distribuzione in modo *diverso l'una dall'altra*: una rotazione mescolerebbe componenti che rispondono in modo diverso a $\mathbf{u}$, e questo produce una discrepanza osservabile. La simmetria è rotta dai dati, non da un'assunzione arbitraria.

Il teorema di iVAE (Khemakhem et al. 2020) dà identificabilità fino a permutazioni e trasformazioni componente per componente, sotto condizioni di *sufficiente variabilità*: servono almeno $2m + 1$ valori distinti di $\mathbf{u}$ che producano parametri naturali linearmente indipendenti. In TCL il ruolo di $\mathbf{u}$ è giocato dall'indice del segmento temporale, sfruttando la non stazionarietà della serie.

Il messaggio generale: l'identificabilità non è un regalo, si compra con informazione aggiuntiva. $\mathbf{u}$ è la forma più economica di quella informazione.

D5. *Cos'è il problema di Causal Representation Learning? In cosa differisce da disentanglement classico?*

Il CRL vuole recuperare, da osservazioni ad alta dimensione $\mathbf{x} = h(\mathbf{c})$, non solo variabili latenti *indipendenti* ma variabili *causali* $\mathbf{c}$ insieme alla struttura causale che le lega — cioè un grafo o uno SCM su $\mathbf{c}$.

Le differenze rispetto al disentanglement classico sono tre e vanno tenute distinte.

Le variabili possono essere dipendenti. Il disentanglement assume fattori indipendenti; le variabili causali reali sono tipicamente collegate ($C_1 \rightarrow C_2$). Cercare l'indipendenza sarebbe qui l'obiettivo sbagliato.

Si vuole anche la struttura, non solo le variabili: quali causano quali. Il disentanglement non ha nessuna nozione di struttura fra i fattori.

Il criterio di successo è l'invarianza sotto intervento, non l'interpretabilità visiva. Una rappresentazione causale corretta deve permettere di predire l'effetto di $do(c_i = v)$, cosa che una rappresentazione disentangled ma non causale non garantisce.

Il collegamento con la Parte I, da fare esplicitamente: la causalità classica assume le variabili *date* e cerca il grafo; il CRL deve trovare *prima* le variabili e poi il grafo, partendo da pixel. È lo stesso problema spostato di un livello di astrazione, e per questo è più difficile: non basta più la faithfulness, servono interventi.

D6. *Descrivi il setup di CITRIS (input, output, ipotesi su interventi).*

Input: triple temporali $(\mathbf{x}_t, \mathbf{x}_{t+1}, \mathbf{I}_{t+1})$, dove $\mathbf{I}_{t+1}$ è una maschera **binaria** che indica quali variabili causali sono state intervenute al tempo $t + 1$. È il punto centrale: si conoscono i *target* degli interventi, non i *valori* assegnati.

Architettura, quattro componenti: un encoder $\mathbf{z}_t = g_\phi(\mathbf{x}_t)$; una *assignment function* $\psi : \mathcal{Z} \rightarrow \{1, \dots, k\}^m$ che assegna ogni dimensione latente a una variabile causale (permettendo che una variabile causale sia rappresentata da più dimensioni, il che è la generalizzazione utile rispetto a un'assegnazione uno-a-uno); un *transition prior* $p(\mathbf{z}_{t+1} | \mathbf{z}_t, \mathbf{I}_{t+1})$ in cui ogni gruppo $\mathbf{z}^{(i)}$ evolve tramite il proprio modulo, modulato da I_i ; e un decoder $p(\mathbf{x}_t | \mathbf{z}_t)$.

Training: un ELBO temporale, opzionalmente affiancato da un classificatore ausiliario che predice $\mathbf{I}_{t+1}$ dai latenti, per rafforzare l'allineamento fra gruppi latenti e target di intervento.

Output: una partizione dello spazio latente in gruppi allineati alle variabili causali, più il modello di transizione, che è la struttura causale temporale.

Il meccanismo in una frase: un intervento su C_i cambia il comportamento del *solo* gruppo $\mathbf{z}^{(i)}$; osservando quali dimensioni rispondono a quale maschera, il modello scopre l'assegnazione.

D7. *Cosa garantisce il teorema di identifiability di CITRIS?*

Garantisce che, nel limite di dati infiniti, il modello recuperi le variabili causali ground-truth — nel senso di identificare correttamente la *partizione* dello spazio latente in gruppi corrispondenti alle variabili causali — se per ogni variabile C_i valgono due condizioni:

1. esiste un regime in cui C_i è intervenuta e *non sempre congiuntamente* a un'altra C_j ;
2. C_i dipende sempre dalla propria variabile di intervento I_i .

La lettura della prima condizione è la più istruttiva: se due variabili fossero *sempre* intervenute insieme, nessun dato potrebbe distinguerle — sarebbero, dal punto di vista osservazionale, una sola variabile. Serve variabilità negli interventi, esattamente come in iVAE serve sufficiente variabilità in $\mathbf{u}$.

La seconda condizione assicura che l'intervento abbia effettivamente un effetto rilevabile: una variabile che ignorasse il proprio intervento non lascerebbe traccia.

Due precisazioni che rendono la risposta accurata. L'identificabilità è *a meno di trasformazioni interne al gruppo*: si recupera quali dimensioni appartengono a quale variabile causale, non necessariamente la parametrizzazione canonica dentro ciascun gruppo. E le variabili senza alcun intervento finiscono in un gruppo residuo non identificato, che CITRIS tiene esplicitamente da parte invece di fingere di risolverlo.

D8. *Differenza pratica fra CITRIS-VAE e CITRIS-NF.*

CITRIS-VAE addestra tutto congiuntamente: encoder, decoder e transition prior, con l'ELBO temporale. È il modello completo, ma paga il conflitto tipico dei VAE fra ricostruzione e struttura del latente: la capacità dell'encoder è spesa in parte per ricostruire bene i pixel e in parte per organizzare causalmente il latente, e le due cose competono.

CITRIS-NF separa i due problemi. Si parte da una rappresentazione *già addestrata* — tipicamente l'encoder di un autoencoder standard, che non deve preoccuparsi di causalità — e le si applica sopra un *normalizing flow* che la ridispone in coordinate disentangled. Il flow è invertibile e preserva l'informazione, quindi non c'è perdita di ricostruzione: si sta solo cambiando sistema di coordinate.

I vantaggi pratici del secondo: si scala meglio a immagini complesse, perché la parte percettiva difficile è delegata a un modello ottimizzato solo per quella; il training è più stabile perché i due obiettivi non competono; e si può riusare una rappresentazione pre-addestrata invece di ripartire da zero.

Il quadro generale: è la stessa strategia *disentangle-a-posteriori* che ricorre in letteratura, e ha senso perché imparare a rappresentare le immagini e imparare a organizzare causalmente quella rappresentazione sono compiti di natura diversa.

D9. *Confronta causalità classica e CRL: quale è il contributo del deep learning?*

	Causalità classica (Parte I)	CRL
Variabili	date, osservate, semanticamente note	latenti, da scoprire dai pixel
Obiettivo	il grafo fra variabili note	le variabili e il grafo
Dati	tabellari, bassa dimensione	alta dimensione, non strutturati
Identificabilità	classe di equivalenza di Markov	richiede interventi o supervisione debole
Strumenti	test di indipendenza, score	encoder, ELBO, flow

Il contributo del deep learning è duplice, e conviene distinguerlo bene.

Il primo è ovvio: fornisce la mappa $\mathbf{x} \rightarrow \mathbf{z}$, cioè permette di partire da dati percettivi grezzi invece che da variabili già estratte a mano. Senza reti, il CRL sarebbe irrealizzabile perché nessuno saprebbe scrivere h^{-1} .

Il secondo è meno ovvio e più interessante: le reti forniscono anche l'apparato per *imporre struttura* sulla rappresentazione — moduli separati per gruppo, prior condizionati agli interventi, flow che ridispongono le coordinate. La struttura causale non viene letta dai dati come nell'approccio classico, viene *incorporata nell'architettura* e verificata dall'ottimizzazione.

Il limite che il deep learning non risolve, e va detto: l'identificabilità resta un problema *statistico*, non computazionale. Nessuna architettura supera Locatello senza informazione aggiuntiva. Le reti rendono trattabile il problema, non lo rendono ben posto — quello lo fanno gli interventi.

Chapter 5

Parte V — Graph Neural Networks

Capitolo 26 — Fondamenti di deep learning per grafi

D1. *Perché non si può applicare direttamente una CNN a un grafo?*

Perché una CNN presuppone tre proprietà che una griglia ha e un grafo non ha.

Nessun ordine canonico dei vicini. In un'immagine “il pixel a sinistra” è ben definito, quindi il peso w_{-1} ha un significato stabile ovunque. In un grafo i vicini di un nodo sono un *insieme* senza ordinamento naturale: qualunque numerazione è arbitraria, e un modello che dipendesse da essa produrrebbe risultati diversi rinumerando gli stessi nodi.

Grado variabile. Un kernel 3×3 ha esattamente nove ingressi. In un grafo un nodo può averne due e un altro mille, quindi non esiste una matrice di pesi di dimensione fissa che si applichi a tutti.

Assenza di una struttura metrica regolare. Non c'è una nozione di traslazione, quindi non c'è nulla rispetto a cui essere equivarianti nel senso della CNN, e non si può definire lo stride o il pooling per finestre.

Da qui la soluzione: aggregare i vicini con una funzione **invariante per permutazione** (somma, media, max, attenzione) che accetta insiemi di dimensione variabile. È questa la generalizzazione corretta della convoluzione ai grafi.

D2. *Che cosa significa invarianza per permutazione e perché è un requisito? Distingui dalla equivarianza.*

Sia P una matrice di permutazione dei nodi. Una funzione f è

invariante $\iff f(PX, PAP^\top) = f(X, A)$, **equivariante** $\iff f(PX, PAP^\top) = Pf(X, A)$.

Non è una comodità ma un *requisito di correttezza*: la numerazione dei nodi è un artefatto della rappresentazione in memoria, non un'informazione del grafo. Un modello che dipendesse da essa darebbe risposte diverse per lo stesso oggetto, e non potrebbe generalizzare.

Quale serve dove: a livello di **nodo** serve l'*equivarianza* — se permuti i nodi voglio che le loro rappresentazioni si permutino di conseguenza, restando attaccate ai nodi giusti. A livello di **grafo** serve l'*invarianza* — la classificazione di una molecola non deve dipendere da come ho numerato gli atomi.

Ed è esattamente come sono costruite le GNN: i layer di message passing sono equivarianti (ogni nodo produce il proprio embedding), il *readout* finale è invariante (somma, media o max su tutti i nodi). La stessa distinzione della CNN, dove i layer convoluzionali sono equivarianti alla traslazione e il global pooling introduce l'invarianza.

D3. *Descrivi la tassonomia dei task su grafi: node-level, edge-level, graph-level.*

Node-level: si predice una proprietà per ciascun nodo. Esempi: classificazione di documenti

in una rete di citazioni, rilevamento di account fraudolenti in un social network. Il readout è **assente**: si usa direttamente $\mathbf{h}_v^{(L)}$, e la testa di predizione è applicata nodo per nodo.

Edge-level: si predice una proprietà di una coppia, tipicamente l'esistenza dell'arco (link prediction) o il suo tipo. Esempi: raccomandazione, completamento di knowledge graph, predizione di interazioni proteina-proteina. Il readout combina le rappresentazioni dei due estremi — prodotto scalare, concatenazione seguita da MLP, o un decoder bilineare $\mathbf{h}_u^\top \mathbf{R} \mathbf{h}_v$ per grafi con tipi di relazione.

Graph-level: si predice una proprietà dell'intero grafo. Esempi: tossicità o solubilità di una molecola, classificazione di programmi. Qui il readout è **essenziale** e deve essere *invariante per permutazione*: $\mathbf{h}_G = \text{READOUT}(\{\mathbf{h}_v^{(L)}\})$, tipicamente somma, media, max, o un pooling gerarchico appreso.

La scelta del readout non è neutra: la *somma* conserva l'informazione sulla dimensione del grafo ed è più espressiva (è quella richiesta dal risultato su GIN), la *media* la normalizza ed è più stabile quando le dimensioni variano molto.

D4. Che differenza c'è fra setting transduttivo e induttivo? Quali modelli funzionano solo nel primo caso?

Nel setting **transduttivo** si ha un unico grafo fisso, noto interamente al momento del training (struttura e feature), e si predicono etichette per nodi di quel grafo che non le hanno. Il modello non deve mai generalizzare a nodi o grafi mai visti. Esempio tipico: Cora, Citeseer.

Nel setting **induttivo** il modello deve funzionare su nodi o grafi *completamente nuovi*: un nuovo utente che entra nel social network, una molecola mai vista. Esempio: predizione di proprietà molecolari, dove test e training sono insiemi disgiunti di grafi.

Funzionano solo nel caso transduttivo tutti i modelli che apprendono un *embedding per nodo* come parametro libero — DeepWalk, node2vec, LINE — perché un nodo nuovo non ha un embedding e non c'è modo di calcolarlo senza riaddestrare. Stesso problema per i metodi *spettrali* puri: la base di autovettori del Laplaciano dipende dal grafo specifico, quindi non si trasferisce.

Funzionano in entrambi i casi i modelli che apprendono una *funzione* dai vicini alla rappresentazione — GCN (con la trasformazione dei pesi, non con embedding per nodo), GraphSAGE (progettato esplicitamente per l'induttivo, da cui il nome “SAmple and aggreGatE”), GAT, GIN. Il criterio discriminante è sempre lo stesso: si sta imparando una tabella o una funzione?

D5. Confronta i modelli contractive di Scarselli e quelli contextual di Micheli. Che cosa impone la contrattività?

Il *Graph Neural Network* di Scarselli et al. (2009) definisce lo stato dei nodi come il **punto fisso** di un sistema di equazioni:

$$\mathbf{h}_v = \sum_{u \in N(v)} f_w(\mathbf{x}_v, \mathbf{x}_u, \mathbf{h}_u),$$

e per garantire che quel punto fisso esista e sia unico impone che f_w sia una *contrazione* (teorema di Banach). Lo stato si calcola iterando fino a convergenza, e il gradiente si ottiene con un metodo di tipo Almeida-Pineda.

Il vincolo di contrattività costa caro: limita fortemente l'espressività — la mappa non può amplificare differenze — e impone un'iterazione a convergenza a ogni forward pass, quindi il costo è alto e variabile.

Il *Neural Network for Graphs* di Micheli (2009) evita il problema per costruzione: invece di cercare un punto fisso, **impila un numero finito di layer**, ciascuno con i propri pesi, in modo strettamente feed-forward. Ogni layer allarga il contesto di un salto, e la profondità è un iperparametro invece che una conseguenza della convergenza. Nessun vincolo di contrattività è necessario, perché non c'è nessuna iterazione da far convergere.

Il senso storico da dare: sono due risposte contemporanee alla stessa domanda, e ha vinto la seconda. Tutte le GNN moderne — GCN, GraphSAGE, GAT, GIN — sono *contextual* nel senso di Micheli, cioè pile finite di layer con pesi distinti.

D6. *Che cos'è un Deep Graph Network nella sintesi moderna, e in che senso unifica i due filoni storici?*

Un DGN è una rete che mappa un grafo in uno spazio vettoriale tramite un encoding *appreso e componibile gerarchicamente*: una pila di layer, ciascuno dei quali aggiorna la rappresentazione di ogni nodo aggregando quelle dei vicini, seguita da un readout per i task a livello di grafo.

Unifica i due filoni perché ne prende un aspetto ciascuno. Dal filone *contextual* prende la struttura a strati finiti con pesi indipendenti, che elimina il vincolo di contrattività e rende il training standard con backpropagation. Dal filone *contractive* conserva l'idea che l'informazione debba *diffondersi* progressivamente attraverso la struttura: ogni layer aggiuntivo allarga il raggio di un salto, e la profondità è ciò che determina quanto lontano l'informazione può viaggiare.

Nella formulazione moderna tutto si riassume nello schema di message passing (Gilmer et al. 2017), con AGGREGATE invariante per permutazione e UPDATE arbitrario, e le architetture note diventano casi particolari distinti dalla scelta di quelle due funzioni.

Il collegamento da fare per chiudere bene: è la stessa struttura della CNN — un'operazione locale ripetuta e composta in profondità, dove la profondità determina il campo recettivo. Cambia solo che il “vicinato” è definito dagli archi invece che dalla griglia.

Capitolo 27 — Message passing e GNN convoluzionali

D1. *Definisci formalmente il message passing. Quali requisiti deve soddisfare AGGREGATE?*

$$\mathbf{m}_v^{(\ell+1)} = \text{AGGREGATE}^{(\ell)}(\{\mathbf{h}_u^{(\ell)} : u \in N(v)\}), \quad \mathbf{h}_v^{(\ell+1)} = \text{UPDATE}^{(\ell)}(\mathbf{h}_v^{(\ell)}, \mathbf{m}_v^{(\ell+1)}),$$

con $\mathbf{h}_v^{(0)} = \mathbf{x}_v$, e per i task graph-level $\mathbf{h}_G = \text{READOUT}(\{\mathbf{h}_v^{(L)}\})$.

Requisiti su AGGREGATE:

1. **invarianza per permutazione** dei vicini — è il requisito fondante, e senza di esso il modello dipenderebbe dalla numerazione arbitraria dei nodi;
2. **accettare insiemi di cardinalità variabile**, perché i gradi differiscono;
3. **differenziabilità**, per l'addestramento end-to-end.

Che cosa succede se il primo non è soddisfatto: il modello produrrebbe output diversi per lo stesso grafo a seconda dell'ordine in cui i vicini compaiono in memoria, non generalizzerebbe, e in pratica sprecherebbe capacità imparando a essere robusto a un'informazione che non esiste.

Esempi validi: somma, media, max, attention weighted sum. Non valido: concatenazione in ordine, o qualunque funzione che tratti i vicini come una sequenza.

D2. *Scrivi la regola di aggiornamento della GCN. Da quale approssimazione spettrale deriva?*

$$\mathbf{H}^{(\ell+1)} = \sigma(\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{H}^{(\ell)} \mathbf{W}^{(\ell)}), \quad \mathbf{h}_v^{(\ell+1)} = \sigma\left(\sum_{u \in N(v) \cup \{v\}} \frac{1}{\sqrt{\tilde{d}_v \tilde{d}_u}} \mathbf{W}^{(\ell)} \mathbf{h}_u^{(\ell)}\right),$$

con $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ e $\tilde{\mathbf{D}}$ la matrice dei gradi corrispondente.

Derivazione. Si parte dalla convoluzione spettrale $\mathbf{g}_\theta \star \mathbf{x} = \mathbf{U} \mathbf{g}_\theta(\mathbf{\Lambda}) \mathbf{U}^\top \mathbf{x}$ con $\mathbf{U}$ autovettori del Laplaciano normalizzato $\tilde{\mathbf{L}} = \mathbf{I} - \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$. Si approssima $\mathbf{g}_\theta$ con un polinomio di *Chebyshev* di ordine K (ChebNet), il che evita la decomposizione spettrale e rende il filtro locale a K salti. Kipf e Welling troncano a $K = 1$, pongono $\lambda_{\max} \approx 2$ e legano i due coefficienti residui con $\theta = \theta_0 = -\theta_1$, ottenendo

$$\mathbf{g}_\theta \star \mathbf{x} \approx \theta (\mathbf{I} + \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}) \mathbf{x}.$$

Infine applicano il *renormalization trick*: $\mathbf{I} + \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2} \rightarrow \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$, perché il primo operatore ha autovalori in $[0, 2]$ e iterandolo causa instabilità numerica ed esplosione dei gradienti.

Quindi il termine $\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$ viene dal Laplaciano normalizzato più il self-loop più la rinormalizzazione.

D3. *Perché l'approccio spettrale puro non si generalizza a grafi diversi da quello di training?*

Perché la base spettrale è il grafo. Il filtro spettrale è definito come $\mathbf{U} \mathbf{g}_\theta(\mathbf{\Lambda}) \mathbf{U}^\top$, dove $\mathbf{U}$ e $\mathbf{\Lambda}$ vengono dalla decomposizione del Laplaciano di *quel* grafo. Su un grafo diverso, autovettori e autovalori sono diversi, e i coefficienti θ appresi non hanno più lo stesso significato: sono coefficienti relativi a una base che non esiste più.

Ci sono anche due problemi pratici gravi: la decomposizione costa $\mathcal{O}(n^3)$ ed è impraticabile su grafi grandi; e il filtro $\mathbf{g}_\theta(\mathbf{\Lambda})$ non è *localizzato* nello spazio — un filtro spettrale generico coinvolge tutti i nodi contemporaneamente.

L'approssimazione polinomiale risolve tutti e tre i problemi in un colpo. Un polinomio $\sum_k \theta_k \tilde{\mathbf{L}}^k$ è esprimibile senza mai calcolare $\mathbf{U}$, è localizzato entro k salti (perché $\tilde{\mathbf{L}}^k$ ha supporto sui cammini di lunghezza k), ed è **trasferibile**, perché i coefficienti θ_k pesano potenze dell'operatore, non autovettori specifici. È il passaggio da spettrale a spaziale, ed è il motivo per cui la GCN è induttiva mentre i metodi spettrali puri non lo sono.

D4. *A che cosa serve il self-loop in GCN? E la normalizzazione simmetrica?*

Il **self-loop** ($\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$) fa sì che l'aggregazione includa il nodo stesso. Senza, $\mathbf{h}_v^{(\ell+1)}$ dipenderebbe *solo* dai vicini e il nodo dimenticherebbe la propria feature a ogni layer — con conseguenze assurde, per esempio due nodi con vicinati identici ma feature proprie diverse riceverebbero la stessa rappresentazione. In termini di message passing, il self-loop è il modo in cui la GCN implementa la parte UPDATE che usa $\mathbf{h}_v^{(\ell)}$.

La **normalizzazione simmetrica** $1/\sqrt{\tilde{d}_v \tilde{d}_u}$ fa due cose. Impedisce che i nodi ad alto grado dominino: senza normalizzazione la somma su molti vicini produce attivazioni di scala molto maggiore, e le magnitudini divergerebbero fra nodi di grado diverso e fra layer. E mantiene lo spettro dell'operatore in $[-1, 1]$, il che stabilizza numericamente la propagazione attraverso molti layer.

Perché *simmetrica* e non $\mathbf{D}^{-1} \mathbf{A}$ (media semplice): la versione simmetrica preserva la simmetria dell'operatore, quindi ha autovalori reali e una teoria spettrale pulita, ed è quella che deriva naturalmente dal Laplaciano normalizzato. La versione asimmetrica $\mathbf{D}^{-1} \tilde{\mathbf{A}}$ corrisponde a una media ed è quella usata da GraphSAGE-mean.

Il costo, da dichiarare: normalizzando si *perde l'informazione sul grado*, e questo è parte del motivo per cui la GCN è meno espressiva di GIN.

D5. *Che cosa cambia GraphSAGE rispetto a GCN, e quale problema pratico risolve il sampling dei vicini?*

Due cambiamenti.

Aggregatore separato dal self: GraphSAGE concatena invece di sommare tutto insieme,

$$\mathbf{h}_v^{(\ell+1)} = \sigma\left(\mathbf{W} \cdot [\mathbf{h}_v^{(\ell)} \parallel \text{AGG}(\{\mathbf{h}_u^{(\ell)} : u \in N(v)\})]\right),$$

con AGG scelto fra mean, max-pool (un MLP seguito da max elemento per elemento) o LSTM su una permutazione casuale. Mantenere il proprio stato separato da quello aggregato è più espressivo che mescolarli.

Neighbor sampling: a ogni layer si campiona un numero *fisso* S di vicini invece di usarli tutti.

Il problema pratico che risolve è l'**esplosione del vicinato**. Con L layer, il calcolo della rappresentazione di un singolo nodo dipende da tutti i nodi entro L salti; in un grafo con grado medio $\bar{d}$ questo cresce come $\bar{d}^L$, e nei grafi reali — dove la distribuzione dei gradi ha coda pesante e un solo hub trascina dentro milioni di nodi — diventa subito ingestibile. Il sampling limita il costo a $\mathcal{O}(S^L)$, indipendente dalla dimensione del grafo, e rende possibile il *mini-batch training*: si può addestrare su un grafo che non entra in memoria.

Il costo è varianza aggiuntiva nelle stime, mitigata campionando di nuovo a ogni epoca. Ed è anche ciò che rende GraphSAGE nativamente *induttivo*: si impara una funzione di aggregazione, non un embedding per nodo.

D6. *Enuncia il legame fra GNN e il test di Weisfeiler-Lehman a una dimensione. Quale aggregazione raggiunge quel limite?*

Enunciato (Xu et al. 2019, Morris et al. 2019): una GNN a message passing è **al più** potente quanto il test 1-WL nel distinguere grafi non isomorfi; e con AGGREGATE, UPDATE e READOUT *iniettivi* sui multiset raggiunge esattamente quel limite.

Il parallelo è strutturale: 1-WL raffina iterativamente un'etichetta per nodo aggregando le etichette dei vicini con una funzione di hash iniettiva; una GNN fa la stessa cosa con vettori e funzioni apprese. Se le funzioni apprese non sono iniettive, la GNN collassa distinzioni che 1-WL manterrebbe.

L'aggregazione che raggiunge il limite è la **somma**, seguita da un MLP:

$$\mathbf{h}_v^{(\ell+1)} = \text{MLP}^{(\ell)}\left((1 + \epsilon) \mathbf{h}_v^{(\ell)} + \sum_{u \in N(v)} \mathbf{h}_u^{(\ell)}\right).$$

Perché somma e media non sono equivalenti: sui multiset, la media e il max sono invarianti alla *molteplicità*. Il multiset $\{\{1, 1, 1\}\}$ e $\{\{1, 1\}\}$ hanno media 1 e max 1 in entrambi i casi, ma somma 3 e 2. La media perde l'informazione su *quanti* vicini di un dato tipo ci sono, cioè in pratica sul grado; la somma la conserva. Poiché GCN normalizza (media) e GraphSAGE-mean anche, entrambi non distinguono quei due intorni, mentre GIN sì.

D7. *Fai un esempio di due grafi che nessuna GNN a message passing standard riesce a distinguere.*

Il caso classico: un **ciclo a sei nodi** C_6 contro **due triangoli disgiunti** $2 \times C_3$, con feature iniziali tutte uguali.

In entrambi i grafi ogni nodo ha grado 2, e ogni nodo ha esattamente due vicini di grado 2, che a loro volta hanno due vicini di grado 2, e così via a ogni distanza. Il raffinamento 1-WL è quindi *stabile fin dalla prima iterazione*: tutte le etichette restano identiche, in entrambi i grafi. Nessuna quantità di layer aiuta, perché il punto fisso è raggiunto subito.

Poiché una GNN a message passing è limitata da 1-WL, nessuna — GIN incluso — può distinguerli. Eppure sono manifestamente diversi: uno è connesso e ha un ciclo di lunghezza 6, l'altro ha due componenti e cicli di lunghezza 3.

Il caso è più generale: qualunque coppia di grafi *regolari* con lo stesso grado e lo stesso numero di nodi è indistinguibile allo stesso modo.

Che cosa servirebbe per superare la barriera: *feature posizionali o strutturali* da dare in input (autovettori del Laplaciano, random walk encoding, distanze da nodi ancora); *conteggi di sottostrutture* espliciti (numero di triangoli, cicli); GNN di ordine superiore (k -WL, che operano su tuple di nodi ma costano $\mathcal{O}(n^k)$); oppure identificatori casuali per nodo, che rompono la simmetria al prezzo di perdere l'invarianza esatta.

D8. *A che cosa serve il termine ϵ nella formulazione di GIN?*

Serve a rendere **distinguibile il contributo del nodo stesso** da quello dei vicini. Nella formula

$$\mathbf{h}_v^{(\ell+1)} = \text{MLP}\left((1 + \epsilon)\mathbf{h}_v^{(\ell)} + \sum_{u \in N(v)} \mathbf{h}_u^{(\ell)}\right),$$

con $\epsilon = 0$ il termine diventa $\mathbf{h}_v + \sum_u \mathbf{h}_u$, cioè il nodo viene sommato *esattamente come un vicino in più*. Due situazioni diverse — un nodo di tipo a con vicini $\{a, b\}$, e un nodo di tipo b con vicini $\{a, a\}$ — produrrebbero la stessa somma e collasserebbero.

Con $\epsilon \neq 0$ il proprio contributo è pesato diversamente, e la mappa dal multiset $(\mathbf{h}_v, \{\{\mathbf{h}_u\}\})$ resta iniettiva — che è la condizione richiesta dal teorema per raggiungere il limite 1-WL.

Dettagli pratici da nominare: ϵ può essere un iperparametro fissato (spesso 0, e la variante si chiama GIN-0) oppure un parametro appreso (GIN- ϵ). L'articolo riporta che le due varianti si comportano in modo simile, e che GIN-0 generalizza talvolta meglio; quindi la sua importanza è più teorica che empirica — ma è esattamente il genere di dettaglio su cui una domanda di terzo livello può insistere.

D9. *Come si incorporano le feature sugli archi nello schema di message passing?*

Si generalizza la funzione messaggio perché dipenda anche dall'arco:

$$\mathbf{m}_{u \rightarrow v} = \phi(\mathbf{h}_v, \mathbf{h}_u, \mathbf{e}_{uv}), \quad \mathbf{h}_v^{(\ell+1)} = \text{UPDATE}\left(\mathbf{h}_v^{(\ell)}, \bigoplus_{u \in N(v)} \mathbf{m}_{u \rightarrow v}\right),$$

con $\bigoplus$ invariante per permutazione. È la formulazione MPNN generale di Gilmer et al., che ha proprio gli archi come ingrediente di prima classe.

Realizzazioni concrete, in ordine di complessità: *concatenazione* $\phi = \text{MLP}([\mathbf{h}_u \parallel \mathbf{e}_{uv}])$, la più semplice; *edge-conditioned convolution* (ECC), in cui una rete genera la *matrice* di pesi a partire dalla feature dell'arco, $\mathbf{m}_{u \rightarrow v} = \mathbf{W}(\mathbf{e}_{uv})\mathbf{h}_u$, molto espressiva ma costosa; *relational GCN*, che usa una matrice diversa per ogni *tipo* di relazione, adatta ai knowledge graph; e *attenzione condizionata all'arco*, come in GAT con edge features.

Si può anche mantenere uno *stato per arco* aggiornato layer per layer, cioè fare message passing sugli archi oltre che sui nodi — è quello che fa la line-graph formulation, e ciò che serve quando l'informazione da propagare è essenzialmente relazionale (legami chimici, distanze, angoli nelle molecole 3D).

D10. *Confronta message passing nelle GNN e belief propagation nei PGM.*

Che cosa è analogo: la struttura computazionale. In entrambi i casi ogni nodo raccoglie messaggi dai vicini, li combina, aggiorna il proprio stato, e si itera. In entrambi la località è il principio guida e l'informazione si propaga di un salto per iterazione.

Che cosa è diverso, e sono quattro cose:

1. **Che cosa sono i messaggi.** In BP sono *distribuzioni* (o potenziali) su valori di variabili, e hanno un significato probabilistico preciso. In una GNN sono vettori di feature senza semantica probabilistica.
2. **Chi decide la regola.** In BP la regola è *derivata* dalle leggi della probabilità e dai potenziali del modello: $m_{u \rightarrow v}(x_v) = \sum_{x_u} \psi(x_u, x_v) \psi(x_u) \prod_{w \neq v} m_{w \rightarrow u}(x_u)$. In una GNN è *appresa* dai dati.

3. **Il criterio di arresto.** BP itera fino a convergenza (esatta su alberi, approssimata e non garantita su grafi con cicli). Una GNN esegue un numero *fisso* L di layer, con pesi diversi per layer.
4. **Il ruolo del grafo.** In BP il grafo codifica *indipendenze condizionali* fra variabili. In una GNN il grafo è *il dato* da rappresentare.

Il collegamento più profondo, che vale la pena nominare: si può vedere una GNN come una BP in cui potenziali e regole di aggiornamento sono parametrizzati e appresi anziché specificati. Ed esistono architetture che rendono il parallelo esplicito, per esempio le *Neural Belief Propagation* e le GNN usate per l'inferenza approssimata nei modelli grafici.

D11. Confronta GCN, GraphSAGE e GIN su AGGREGATE, espressività e scalabilità.

	GCN	GraphSAGE	GIN
AGGREGATE	media normalizzata	mean / max-pool / LSTM	somma
Self	self-loop (sommato)	concatenato	pesato ($1 + \epsilon$)
Espressività	< 1-WL	< 1-WL (mean/max)	= 1-WL
Scalabilità	full-batch	sampling, mini-batch	full-batch
Setting	induttivo	induttivo (per design)	induttivo

Lettura. GCN è il più semplice e il più usato come baseline; la normalizzazione lo rende stabile ma gli fa perdere l'informazione sul grado. GraphSAGE è la scelta quando il grafo è grande, perché è l'unico dei tre progettato per non caricare l'intero vicinato; con aggregatore max-pool è leggermente più espressivo che con mean, ma resta sotto 1-WL. GIN è il massimo teorico ottenibile con message passing standard, ed è la scelta quando il task richiede di distinguere strutture fini (proprietà molecolari); in compenso la somma non normalizzata può dare problemi di scala su grafi con gradi molto disomogenei.

Il criterio pratico: si sceglie GIN se il task è graph-level e strutturale, GraphSAGE se il vincolo è la dimensione, GCN se serve una baseline solida e il grafo è piccolo.

Capitolo 28 — Graph attention e graph transformer

D1. Scrivi le equazioni del GAT. Differenza concettuale fra attenzione di un GAT e self-attention di un Transformer.

$$e_{vu} = \text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \parallel \mathbf{W}\mathbf{h}_u]), \quad \alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in N(v) \cup \{v\}} \exp(e_{vk})},$$

$$\mathbf{h}'_v = \sigma \left(\sum_{u \in N(v) \cup \{v\}} \alpha_{vu} \mathbf{W}\mathbf{h}_u \right).$$

La differenza concettuale con la self-attention è **su quale insieme si normalizza la softmax**. Nel Transformer la somma è su *tutti* i token: il grafo implicito è completo. Nel GAT è ristretta a $N(v)$: la struttura del grafo agisce come una *maschera* che azzerà l'attenzione fra nodi non adiacenti.

Da qui due conseguenze. Il GAT ha un inductive bias più forte — sa già chi può parlare con chi — e un costo $\mathcal{O}(|E|)$ invece di $\mathcal{O}(n^2)$. Ma non può modellare interazioni a lungo raggio in un solo layer, e resta soggetto a over-squashing e under-reaching.

Differenza tecnica minore ma citabile: il GAT usa un'attenzione *additiva* (concatenazione più vettore $\mathbf{a}$, alla Bahdanau), il Transformer una moltiplicativa scalata. E il GAT non ha proiezioni separate per $\mathbf{Q}, \mathbf{K}, \mathbf{V}$: usa la stessa $\mathbf{W}$.

Un Transformer, in questa luce, è un GAT su grafo completo — ed è il modo migliore di dire il collegamento all'orale.

D2. *Quando preferiresti un GAT a una GCN, e viceversa?*

GAT quando i vicini non sono equivalenti e la loro importanza dipende dal *contenuto*: grafi con nodi eterogenei, reti di citazioni dove alcuni riferimenti contano più di altri, molecole dove certi atomi vicini determinano la reattività. Si guadagna anche interpretabilità, perché i coefficienti α dicono a chi il nodo ha guardato. Ed è nativamente induttivo, perché i pesi dipendono dalle feature e non dalla matrice di adiacenza.

GCN quando il segnale è essenzialmente strutturale e i vicini sono intercambiabili, quando i dati sono pochi (il GAT ha più parametri e si adatta più facilmente al rumore), o quando conta la velocità: la GCN ha coefficienti *precalcolabili*, quindi la propagazione è una moltiplicazione sparsa fissa, mentre il GAT deve ricalcolare l'attenzione a ogni forward.

Che cosa si perde con GAT: costo maggiore, più iperparametri (numero di teste), maggiore rischio di overfitting, e — punto non ovvio — *nessun guadagno di espressività* sul test 1-WL, perché l'attenzione produce comunque una media pesata normalizzata. Il vantaggio è nella qualità delle rappresentazioni su dati reali, non nel potere discriminativo teorico.

D3. *Un GAT è più espressivo di un GIN sul test 1-WL? Motiva.*

No. GIN raggiunge il limite 1-WL, GAT resta sotto.

Il motivo è che l'aggregazione del GAT è una *media pesata con coefficienti che sommano a uno*: $\sum_u \alpha_{vu} \mathbf{W} \mathbf{h}_u$ con $\sum_u \alpha_{vu} = 1$. È una combinazione convessa, quindi eredita il difetto della media — l'invarianza alla molteplicità. Sui due multiset $\{\{1, 1, 1\}\}$ e $\{\{1, 1\}\}$ con feature identiche, i coefficienti di attenzione risultano tutti uguali per simmetria (1/3 e 1/2 rispettivamente), e l'output è lo stesso valore in entrambi i casi.

Il punto concettuale da dire: *attenzione* e *potere discriminativo* sono assi indipendenti. L'attenzione aggiunge la capacità di pesare i vicini in base al contenuto — utile quando le feature sono informative — ma non restituisce l'informazione sulla cardinalità che la normalizzazione ha rimosso.

Corollario: se il task richiede di distinguere strutture (conteggi, sottografi), serve un aggregatore a somma; se richiede di selezionare informazione fra vicini eterogenei, serve l'attenzione. Esistono architetture che fanno entrambe le cose, per esempio usando attenzione *non normalizzata* o combinando i due aggregatori.

D4. *A che cosa serve il multi-head in un GAT, e perché nei layer intermedi si concatena mentre nell'ultimo si media?*

Serve a due cose. *Stabilizzare il training*: una singola attenzione è rumorosa, soprattutto all'inizio quando i coefficienti sono quasi casuali; mediare su più teste riduce la varianza. E *catturare più relazioni contemporaneamente*: ogni testa proietta in un sottospazio diverso e può specializzarsi su un aspetto diverso del vicinato, esattamente come nel Transformer.

Nei layer **intermedi** si *concatena*:

$$\mathbf{h}'_v = \parallel_{k=1}^K \sigma \left(\sum_{u \in N(v)} \alpha_{vu}^k \mathbf{W}^k \mathbf{h}_u \right),$$

perché si vuole *preservare* tutte le informazioni raccolte dalle diverse teste e passarle al layer successivo, che deciderà come combinarle. La dimensione dell'output è $K \cdot F'$.

Nel layer **finale** si *media*:

$$\mathbf{h}'_v = \sigma \left(\frac{1}{K} \sum_{k=1}^K \sum_{u \in N(v)} \alpha_{vu}^k \mathbf{W}^k \mathbf{h}_u \right),$$

perché lì l'output deve avere la dimensionalità del task (per esempio il numero di classi), e concatenare produrrebbe K copie di un vettore di logit — che andrebbero comunque ridotte. Mediare è il modo naturale di farlo, e mantiene la nonlinearità finale (softmax o sigmoide) applicata una sola volta.

D5. *Che cos'è un Graph Transformer? Perché serve un positional encoding strutturale?*

È un Transformer applicato ai nodi di un grafo trattati come token, quindi con attenzione *globale*: ogni nodo attende su tutti gli altri, non solo sui vicini.

Il positional encoding strutturale serve perché, rimuovendo la restrizione ai vicini, si **butta via la struttura del grafo**. Senza reinserirla, il modello vedrebbe un insieme di nodi senza archi, e due grafi con gli stessi nodi ma topologie diverse sarebbero indistinguibili. Nel Transformer testuale il problema analogo è l'ordine dei token, risolto con le sinusoidi; qui la “posizione” è la posizione *nel grafo*, che è un oggetto molto più ricco.

Forme che può prendere, in ordine di diffusione:

- *Laplacian eigenvectors*: i primi k autovettori del Laplaciano come feature per nodo. Sono l'analogo diretto delle sinusoidi (che sono gli autovettori del Laplaciano di un cammino), ma hanno segno e ordine arbitrari, il che richiede accorgimenti;
- *random walk structural encoding*: le probabilità di ritorno $[\mathbf{R}^k]_{vv}$ per vari k , che codificano informazione locale sui cicli;
- *distanze sul grafo* usate come bias nell'attenzione (Graphormer), spesso insieme a un bias di centralità e a un edge encoding lungo il cammino minimo.

L'architettura *GPS* (Rampášek et al. 2022) è la sintesi pratica: a ogni layer combina un ramo di message passing locale con un ramo di attenzione globale, prendendo il bias induttivo dal primo e la capacità a lungo raggio dal secondo.

D6. *Come si fa pooling gerarchico in una GNN? Spiega DiffPool.*

Il readout globale (somma o media su tutti i nodi) è “piatto”: collassa in un colpo solo e perde la struttura gerarchica, che nelle molecole o nei programmi è informativa. Il pooling gerarchico costruisce invece una sequenza di grafi progressivamente più piccoli, come il pooling in una CNN.

DiffPool (Ying et al. 2018) impara una matrice di assegnazione *soft* a ogni livello:

$$\mathbf{S}^{(\ell)} = \text{softmax}(\text{GNN}_{\text{pool}}^{(\ell)}(\mathbf{A}^{(\ell)}, \mathbf{X}^{(\ell)})) \in \mathbb{R}^{n_\ell \times n_{\ell+1}},$$

dove la softmax è per riga, quindi ogni nodo è distribuito fra i cluster del livello successivo. Poi si costruiscono il nuovo grafo e le nuove feature:

$$\mathbf{X}^{(\ell+1)} = \mathbf{S}^{(\ell)\top} \mathbf{Z}^{(\ell)}, \quad \mathbf{A}^{(\ell+1)} = \mathbf{S}^{(\ell)\top} \mathbf{A}^{(\ell)} \mathbf{S}^{(\ell)},$$

con $\mathbf{Z}^{(\ell)}$ prodotto da una seconda GNN di embedding. Tutto è differenziabile, quindi il clustering è appreso end-to-end insieme al task.

Due termini di regolarizzazione sono necessari perché funzioni: una *link prediction loss* $\|\mathbf{A}^{(\ell)} - \mathbf{S}\mathbf{S}^\top\|_F$, che incoraggia a raggruppare nodi effettivamente connessi, e un termine di *entropia* sulle righe di $\mathbf{S}$, che spinge le assegnazioni verso l'hard e impedisce che tutti i nodi finiscano spalmati su tutti i cluster.

Il limite pratico da nominare: $\mathbf{A}^{(\ell+1)}$ è *densa*, quindi la memoria è $\mathcal{O}(n^2)$ e DiffPool non scala a grafi grandi. Alternative più leggere: Top- k pooling (gPool), che seleziona i k nodi con punteggio più alto, e SAGPool, che usa l'attenzione per il punteggio.

D7. *Che cos'è l'over-smoothing? Da che cosa è causato a livello spettrale e come si mitiga?*

È il fenomeno per cui, impilando molti layer di message passing, le rappresentazioni dei nodi diventano progressivamente indistinguibili fra loro.

La causa è **spettrale**. L'operatore di propagazione $\mathbf{S} = \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$ ha autovalore dominante 1, con autovettore $\propto \sqrt{\tilde{\mathbf{d}}}$. Iterare $\mathbf{S}^L$ proietta qualunque segnale su quell'autovettore, perché tutte le altre componenti sono moltiplicate per $|\lambda_i| < 1$ e decadono esponenzialmente. Il risultato limite dipende quindi solo dai *gradi*, e tutta l'informazione contenuta nelle feature iniziali viene cancellata.

Il conto sul grafo a catena di quattro nodi del Cap. 28 lo mostra: l'energia di Dirichlet passa da 1.736 a 4.2×10^{-6} in venti layer, e due input completamente diversi — (1, 2, 3, 4) e (10, -5, 0, 3) — convergono alla *stessa* direzione (0.447, 0.548, 0.548, 0.447).

Mitigazioni, e tutte agiscono impedendo alla dinamica di raggiungere quel punto fisso: *residual/skip connection* ($\mathbf{h}^{(\ell+1)} = \mathbf{h}^{(\ell)} + \text{GNN}(\cdot)$, che aggiunge un $\mathbf{I}$ all'operatore); *jumping knowledge*, che concatena o combina le rappresentazioni di tutti i layer invece di usare solo l'ultima; *PairNorm* e normalizzazioni analoghe, che riscavano per mantenere costante la distanza media a coppie; *DropEdge*, che rimuove archi a caso rallentando la diffusione; e più semplicemente usare poche layer, che è ciò che si fa quasi sempre in pratica (2–3).

D8. *Che cos'è l'over-squashing? Distinguiamo dall'over-smoothing.*

È il fenomeno per cui l'informazione proveniente da un vicinato che cresce *esponenzialmente* con il numero di layer deve essere compressa in un vettore di dimensione *fissa*, e viene inevitabilmente distorta o persa. Con L layer un nodo dipende da $\mathcal{O}(\bar{d}^L)$ nodi, ma $\mathbf{h}_v$ ha sempre d componenti.

La causa è **topologica**, non spettrale: dipende dai *colli di bottiglia* del grafo. Se due regioni sono collegate da pochi archi, tutta l'informazione fra loro deve passare per quel canale stretto, e si può quantificare con la curvatura di Ricci degli archi o con la sensibilità $\partial \mathbf{h}_v^{(L)} / \partial \mathbf{x}_u$, che decade esponenzialmente con la distanza per i nodi separati da bottleneck.

La distinzione precisa, che è ciò che la domanda chiede: over-smoothing è *omogeneizzazione* — le rappresentazioni convergono allo stesso valore, e il sintomo è che nodi diversi diventano indistinguibili. Over-squashing è *perdita di banda* — l'informazione lontana non arriva, e il sintomo è che le dipendenze a lungo raggio non vengono apprese anche con abbastanza layer. La verifica rapida: se aggiungere layer *peggiora*, è over-smoothing; se aggiungere layer non basta mai, è over-squashing.

Come l'attenzione globale lo attenua: un Graph Transformer collega direttamente ogni coppia di nodi, quindi l'informazione non deve più attraversare il bottleneck — il cammino diventa di lunghezza 1 indipendentemente dalla topologia. Alternative meno drastiche: *rewiring* del grafo (aggiungere archi dove la curvatura è più negativa), *virtual node* collegato a tutti, che crea una scorciatoia globale a costo lineare.

D9. *Che cos'è l'under-reaching? Perché non si risolve semplicemente aggiungendo layer?*

Con L layer di message passing, la rappresentazione di un nodo dipende *solo* dai nodi entro distanza L . Se il task richiede informazione da un nodo a distanza maggiore, quell'informazione non è semplicemente attenuata: non è **proprio arrivata**. È un limite di raggio, non di qualità.

È il più banale dei tre problemi e, formalmente, l'unico che *si risolverebbe* aggiungendo layer. Il motivo per cui in pratica non si risolve così è che le altre due patologie intervengono prima: aumentando L si peggiora l'over-smoothing (le rappresentazioni collassano) e si aggrava l'over-squashing (il vicinato da comprimere cresce esponenzialmente). Le tre cose sono in tensione, e il numero di layer è un compromesso fra un raggio troppo corto e una dinamica degenerare.

Ed è questa tensione, più di ogni altra cosa, a motivare le architetture con attenzione globale o con virtual node: permettono di raggiungere nodi lontani *senza* aumentare la profondità, quindi disaccoppiano il raggio dalle patologie della profondità.

Vale la pena aggiungere la diagnosi pratica: se il diametro del grafo è molto maggiore del numero di layer che ci si può permettere, l'under-reaching è probabilmente il collo di bottiglia, e la soluzione è architetturale, non di tuning.

D10. *Che cosa sono i modelli reservoir per grafi? Che cosa si addestra e che cosa resta fisso?*

Sono l'estensione ai grafi del reservoir computing (le Echo State Network della Parte III). Lo stato dei nodi è calcolato da una dinamica di message passing con pesi **fissi e casuali**:

$$\mathbf{h}(v) = \tanh\left(\mathbf{V}\mathbf{x}_v + \sum_{u \in N(v)} \mathbf{W}\mathbf{h}(u)\right),$$

iterata fino a convergenza o per un numero fissato di passi.

Che cosa resta fisso: $\mathbf{W}$ (i pesi ricorrenti di message passing) e $\mathbf{V}$ (i pesi di input), generati casualmente e mai aggiornati. *Che cosa si addestra:* solo il **readout lineare** dallo stato finale all'output, tipicamente con una ridge regression, quindi *in forma chiusa* e senza discesa del gradiente.

La condizione su $\mathbf{W}$ è la contrattività, garantita imponendo $\rho(\mathbf{W}) < 1$ sul raggio spettrale (in pratica si genera $\mathbf{W}$ casuale sparsa e la si riscalda). Serve a garantire la stabilità della dinamica e l'analogo grafo dell'Echo State Property: lo stato dipende dalla struttura e dalle feature, non dall'inizializzazione arbitraria.

Vantaggi: training in secondi invece che ore, nessun problema di vanishing gradient, pochissimi iperparametri, ottimo come baseline forte e per confrontare quanto del guadagno di una GNN venga davvero dall'apprendimento della propagazione. Limite: meno espressivo di una GNN addestrata end-to-end, perché la parte che estrae le feature strutturali è casuale.

Bacciu è uno degli autori di riferimento su questa linea, quindi è ragionevole aspettarsi che la domanda arrivi.

D11. *Il denoiser di GrIDDD è un graph transformer: le patologie di questo capitolo valgono anche lì?*

In parte, e la risposta va articolata perché è precisamente il tipo di domanda-ponte che chiude un orale.

Over-squashing e under-reaching: sono largamente **attenuati**, perché il denoiser è un graph transformer con attenzione globale. Ogni coppia di atomi comunica direttamente, quindi non c'è bottleneck topologico né limite di raggio — ed è esattamente la motivazione dichiarata nel paper per quella scelta architetturale: i DDPM su grafi devono raffinare iterativamente interazioni a lungo raggio come le chiusure di anelli, che un decoder one-shot dovrebbe imparare in una sola trasformazione.

Over-smoothing: resta un rischio, perché è una proprietà della profondità e dell'aggregazione, non della connettività. Con attenzione globale l'operatore è diverso da $\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$, ma la tendenza dei layer di attenzione a omogeneizzare le rappresentazioni è documentata anche nei Transformer (*token uniformity*), e si contrasta con gli stessi rimedi: connessioni residue e normalizzazioni, che infatti il blocco Transformer ha per costruzione.

Il nodo appena inserito. È il caso interessante. Un nodo attivato tardi nel processo di diffusione ha **pochissimi archi**, quindi in un message passing locale riceverebbe pochi messaggi e resterebbe scarsamente informato: sarebbe un caso estremo di under-reaching, in cui il problema non è la distanza ma l'assenza stessa di vicini. Con l'attenzione globale il nodo può comunque attendere su tutti gli altri, il che mitiga il problema ma non lo elimina — le sue *key* restano poco informative, perché la sua rappresentazione locale è ancora quasi vuota.

Ed è coerente con il secondo limite dichiarato dagli autori: GrIDDD tende a produrre molecole **spezzate**, con più componenti connesse, proprio perché i nodi inseriti tardi faticano a connettersi prima della fine del denoising. Il rimedio che suggeriscono — dare in input il numero di componenti connesse come feature — è esattamente un *feature strutturale globale* nello spirito dei positional encoding di questo capitolo.

Chapter 6

Parte VI — Reinforcement Learning

Capitolo 29 — Fondamenti di RL

D1. *Definisci un MDP. Che cosa significa esattamente “Markov” qui, e su che cosa è un’assunzione forte?*

Un Markov Decision Process è la quintupla $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$: insieme degli stati, insieme delle azioni, dinamica di transizione $P(s' | s, a)$, funzione di ricompensa $R(s, a, s')$ e fattore di sconto $\gamma \in [0, 1)$.

“Markov” significa che la dinamica dipende *solo* dallo stato corrente e dall’azione, non dalla storia:

$$P(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0) = P(s_{t+1} | s_t, a_t).$$

Dove l’assunzione è forte: dice che lo stato è una **statistica sufficiente** di tutto il passato. È un’assunzione sulla *rappresentazione* scelta, non sul mondo — e infatti la si può quasi sempre soddisfare arricchendo lo stato, al prezzo di farlo esplodere. Il caso da citare è quello dei giochi Atari: un singolo frame non è markoviano, perché non contiene la velocità della pallina; impilando quattro frame consecutivi lo si rende (approssimativamente) tale.

Quando non si può soddisfare si esce dal formalismo e si entra nei *POMDP*, dove l’agente osserva o_t correlato ma non identico a s_t , e la statistica sufficiente diventa la *belief* $b_t = P(s_t | o_{1:t}, a_{1:t-1})$ — cioè, di nuovo, un problema di inferenza in un modello a variabili latenti, e il ponte con la Parte II.

D2. *Che cosa fa γ ? Dai le due giustificazioni, quella economica e quella matematica.*

γ sconta le ricompense future nel ritorno $G_t = \sum_{k \geq 0} \gamma^k r_{t+k+1}$: con $\gamma \rightarrow 0$ l’agente è miope e ottimizza solo la ricompensa immediata, con $\gamma \rightarrow 1$ pesa quasi ugualmente tutto il futuro.

Giustificazione economica: una ricompensa oggi vale più della stessa domani, per preferenza temporale o per incertezza sulla continuazione dell’episodio. Anzi, c’è un’equivalenza esatta e citabile: massimizzare un ritorno scontato con fattore γ equivale a massimizzare un ritorno *non* scontato in un processo che a ogni passo termina con probabilità $1 - \gamma$.

Giustificazione matematica, più importante: garantisce che la serie **converga** su orizzonte infinito. Se le ricompense sono limitate da $R_{\max}$, allora $|G_t| \leq R_{\max}/(1 - \gamma)$. E soprattutto rende l’operatore di Bellman una *contrazione* di modulo γ in norma infinito, il che dà esistenza e unicità del punto fisso e la convergenza geometrica di value iteration. Senza $\gamma < 1$ (e senza altre ipotesi, come stati terminali raggiunti con probabilità 1) nulla di tutto questo vale.

L’effetto pratico da nominare: γ definisce un orizzonte effettivo di circa $1/(1 - \gamma)$ passi. Con $\gamma = 0.99$ sono cento passi, ed è il motivo per cui valori vicini a 1 rendono l’apprendimento molto più lento e instabile.

D3. Definisci il ritorno, la state-value e la action-value. Che relazione le lega?

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \quad v_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid s_t = s], \quad q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid s_t = s, a_t = a].$$

La differenza fra le due value function: v_{π} valuta uno stato *seguendo* π *da subito*; q_{π} valuta uno stato *avendo forzato la prima azione* ad essere a , e seguendo π da lì in poi.

Le due relazioni che le legano, entrambe da saper scrivere:

$$v_{\pi}(s) = \sum_a \pi(a \mid s) q_{\pi}(s, a), \quad q_{\pi}(s, a) = \sum_{s'} P(s' \mid s, a) [R(s, a, s') + \gamma v_{\pi}(s')].$$

La prima media su ciò che farebbe la politica; la seconda media su ciò che farebbe l'ambiente. Sostituendo l'una nell'altra si ottengono le equazioni di Bellman.

Nel caso ottimo: $v^*(s) = \max_a q^*(s, a)$ — il max prende il posto della media sulla politica, perché la politica ottima è greedy.

D4. Deriva la Bellman expectation equation per $v_{\pi}(s)$ partendo dalla definizione di ritorno.

Si separa il primo termine dalla coda e si riconosce che la coda è il ritorno al passo successivo:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = r_{t+1} + \gamma G_{t+1}.$$

Prendendo l'attesa condizionata a $s_t = s$ sotto π :

$$v_{\pi}(s) = \mathbb{E}_{\pi}[r_{t+1} + \gamma G_{t+1} \mid s_t = s] = \mathbb{E}_{\pi}[r_{t+1} \mid s_t = s] + \gamma \mathbb{E}_{\pi}[G_{t+1} \mid s_t = s].$$

Esplicitando le attese su azione e stato successivo, e usando la proprietà di Markov per riconoscere $\mathbb{E}[G_{t+1} \mid s_{t+1} = s'] = v_{\pi}(s')$:

$$v_{\pi}(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) [R(s, a, s') + \gamma v_{\pi}(s')].$$

Il passaggio decisivo è $G_t = r_{t+1} + \gamma G_{t+1}$: è lì che una quantità definita su traiettorie *infinite* viene espressa in termini di sé stessa a un passo di distanza. È la stessa mossa della programmazione dinamica di Viterbi, e trasforma un'enumerazione impossibile in una ricorsione.

In forma matriciale, $\mathbf{v}_{\pi} = \mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} \mathbf{v}_{\pi}$, che è un sistema lineare con soluzione $\mathbf{v}_{\pi} = (\mathbf{I} - \gamma \mathbf{P}_{\pi})^{-1} \mathbf{r}_{\pi}$ — risolubile direttamente per $|\mathcal{S}|$ piccolo.

D5. Differenza fra Bellman expectation e Bellman optimality. Perché la seconda non è lineare?

$$\text{expectation: } v_{\pi}(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) [R + \gamma v_{\pi}(s')],$$

$$\text{optimality: } v^*(s) = \max_a \sum_{s'} P(s' \mid s, a) [R + \gamma v^*(s')].$$

La prima valuta una politica *data*: la media su a è pesata da π , che è fissata. La seconda caratterizza la politica *ottima*: al posto della media compare un max sulle azioni.

La non linearità viene esattamente da lì: max non è un operatore lineare. La expectation è un sistema di $|\mathcal{S}|$ equazioni lineari e si può risolvere in forma chiusa invertendo $(\mathbf{I} - \gamma \mathbf{P}_{\pi})$; la optimality no, e va risolta iterativamente.

Ma l'operatore di Bellman ottimale conserva la proprietà che conta: è una **contrazione** di modulo γ in norma infinito, perché $|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$. Quindi il punto fisso esiste, è unico, e l'iterazione vi converge geometricamente — che è precisamente value iteration.

Il legame con gli algoritmi: dalla expectation nasce *policy evaluation* (e quindi policy iteration), dalla optimality nasce *value iteration*.

D6. Perché l'operatore di Bellman è una contrazione, e che cosa se ne deduce?

Sia $(\mathcal{T}v)(s) = \max_a \sum_{s'} P(s' | s, a) [R + \gamma v(s')]$. Per due funzioni valore u, v :

$$\|\mathcal{T}u - \mathcal{T}v\|_\infty \leq \gamma \|u - v\|_\infty.$$

Dimostrazione in due passi: si usa $|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$ per portare il max fuori dalla differenza, poi si osserva che $\sum_{s'} P(s' | s, a) |u(s') - v(s')| \leq \|u - v\|_\infty$ perché P è una distribuzione di probabilità e somma a uno. Resta il fattore γ .

Per il teorema del punto fisso di Banach in uno spazio metrico completo — e $(\mathbb{R}^{|S|}, \|\cdot\|_\infty)$ lo è:

1. esiste un **unico** punto fisso v^* ;
2. l'iterazione $v_{k+1} = \mathcal{T}v_k$ converge a v^* da *qualunque* inizializzazione;
3. la convergenza è **geometrica**: $\|v_k - v^*\|_\infty \leq \gamma^k \|v_0 - v^*\|_\infty$.

Il punto (3) dà anche un criterio di arresto utilizzabile: se $\|v_{k+1} - v_k\|_\infty < \varepsilon$, allora $\|v_{k+1} - v^*\|_\infty < \varepsilon\gamma/(1 - \gamma)$.

Ed è anche il punto in cui si vede il costo di γ vicino a 1: la convergenza è geometrica di ragione γ , quindi con $\gamma = 0.999$ servono migliaia di iterazioni per lo stesso errore che con $\gamma = 0.9$ si ottiene in decine.

D7. Perché quasi tutti gli algoritmi model-free lavorano su Q e non su V ?

Perché da V non si può estrarre una politica senza conoscere il modello. La politica greedy rispetto a V è

$$\pi(s) = \arg \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma v(s')],$$

e quella somma richiede P e R , cioè richiede di **sapere che cosa succede** se si prende l'azione a . In un contesto model-free non lo si sa.

Da Q invece la politica si estrae direttamente:

$$\pi(s) = \arg \max_a q(s, a),$$

senza nessuna conoscenza della dinamica. È tutta lì la ragione, ed è la risposta compatta alla domanda.

Il prezzo è dimensionale: Q è una tabella $|S| \times |A|$ invece che $|S|$, quindi serve più memoria e più esperienza per stimarla — ogni coppia stato-azione va visitata, non ogni stato.

Da aggiungere per completezza: esiste una terza via, cioè imparare V e una politica separata, ed è quello che fanno i metodi actor-critic. Il critico stima V (o l'advantage), l'attore mantiene la politica esplicitamente, e nessuno dei due ha bisogno del modello. È il motivo per cui actor-critic è la scelta standard per le azioni continue, dove l' $\arg \max$ su $\mathcal{A}$ sarebbe comunque un problema di ottimizzazione a sé.

D8. Che cos'è una politica ottima, e perché ne esiste sempre almeno una deterministica in un MDP finito?

Una politica π^* è ottima se $v_{\pi^*}(s) \geq v_\pi(s)$ per *ogni* stato s e ogni altra politica π . Non è ovvio che una politica possa essere simultaneamente migliore ovunque — potrebbero esserci

politiche migliori in stati diversi — e il teorema di ottimalità per gli MDP afferma che invece esiste sempre, in un MDP finito.

L'esistenza di una politica ottima *deterministica* segue dalla forma dell'equazione di ottimalità. Una volta noto q^* , si definisce

$$\pi^*(s) = \arg \max_a q^*(s, a),$$

e si verifica che questa politica raggiunge v^* . L'argomento intuitivo: randomizzare significa mettere probabilità positiva su almeno un'azione con q^* non massimo, il che può solo abbassare (o pareggiare) il valore atteso. Nessun guadagno può venire dal randomizzare, perché la ricompensa attesa è *lineare* nella distribuzione sulle azioni, e il massimo di una funzione lineare su un semplice è raggiunto in un vertice.

Le eccezioni sono istruttive e vale la pena nominarle: la politica ottima *non* è deterministica nei giochi a somma zero con avversario (dove essere prevedibili è fatale — sasso-carta-forbici), nei POMDP (dove la parziale osservabilità può rendere ottimo randomizzare), e nei problemi con vincoli. Ed è uno dei motivi per cui i metodi policy-based, che rappresentano politiche stocastiche, non sono ridondanti.

Capitolo 30 — Algoritmi RL

D1. *Confronta policy iteration e value iteration: che cosa itera ciascuno, quale converge in meno iterazioni?*

Policy iteration alterna due fasi. **Policy evaluation:** si calcola v_π risolvendo il sistema lineare $\mathbf{v}_\pi = \mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v}_\pi$, esattamente o iterando fino a convergenza. **Policy improvement:** si aggiorna $\pi(s) = \arg \max_a \sum_{s'} P[R + \gamma v_\pi(s')]$. Si ripete finché la politica non cambia più.

Value iteration non mantiene una politica esplicita: itera direttamente l'operatore di ottimalità,

$$v_{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) [R + \gamma v_k(s')],$$

ed estrae la politica greedy solo alla fine. È equivalente a fare policy iteration con *un solo* passo di evaluation.

Il confronto: policy iteration converge in **meno iterazioni** — tipicamente pochissime, e il numero è limitato dal numero di politiche deterministiche distinte, quindi finito — ma ogni iterazione è *costosa*, perché richiede una valutazione completa ($\mathcal{O}(|\mathcal{S}|^3)$ se risolta esattamente). Value iteration ha iterazioni economiche ($\mathcal{O}(|\mathcal{S}|^2|\mathcal{A}|)$) ma ne serve un numero maggiore, governato dalla contrazione γ^k .

Il compromesso pratico è la *modified policy iteration*, che fa k passi di evaluation invece di uno o di infiniti: con $k = 1$ è value iteration, con $k = \infty$ è policy iteration.

Da dire per chiudere: entrambe sono **model-based**, cioè richiedono P e R noti. Sono algoritmi di programmazione dinamica, non di apprendimento.

D2. *Che differenza c'è fra metodi Monte Carlo e Temporal Difference? Che cos'è il bootstrapping?*

Monte Carlo aspetta la fine dell'episodio e usa il ritorno effettivamente osservato:

$$V(s_t) \leftarrow V(s_t) + \alpha [G_t - V(s_t)].$$

Temporal Difference aggiorna dopo un solo passo, usando la propria stima corrente al posto della coda del ritorno:

$$V(s_t) \leftarrow V(s_t) + \alpha \underbrace{[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]}_{\text{target TD}}.$$

Il **bootstrapping** è precisamente questo: usare una stima per aggiornare un'altra stima. MC non fa bootstrapping (il target G_t è un campione della quantità vera), TD sì.

Il compromesso bias-varianza. Il target MC è *non distorto* — $\mathbb{E}[G_t] = v_\pi(s_t)$ per definizione — ma ha *varianza alta*, perché G_t dipende da tutta la traiettoria, cioè da molte transizioni e molte scelte casuali. Il target TD è *distorto*, perché $V(s_{t+1})$ è sbagliato all'inizio, ma ha *varianza bassa*, perché dipende da una sola transizione.

In pratica il TD vince quasi sempre, per tre ragioni: converge più rapidamente, funziona *online* e in problemi continui senza episodi (MC richiede episodi terminanti), e la sua distorsione svanisce man mano che le stime migliorano. $TD(\lambda)$ con eligibility trace interpola fra i due estremi, con $\lambda = 0$ che dà $TD(0)$ e $\lambda = 1$ che dà MC.

D3. *Scrivi l'aggiornamento di Q-learning. Perché è off-policy?*

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

È **off-policy** perché il target usa $\max_{a'} Q(s', a')$, cioè il valore dell'azione che prenderebbe la politica *greedy*, indipendentemente da quale azione la politica di comportamento sceglierà effettivamente in s' .

Si stanno quindi usando due politiche diverse: una *di comportamento* (behaviour), tipicamente ε -greedy, che genera l'esperienza; e una *target*, quella greedy, di cui si sta imparando il valore. Q-learning converge a q^* — il valore della politica ottima — anche seguendo una politica di comportamento sub-ottima, purché tutte le coppie (s, a) continuino a essere visitate.

Le conseguenze pratiche sono importanti e vanno nominate: si può imparare da esperienza *vecchia* (experience replay), da esperienza generata da un'altra politica o da un altro agente, e da dimostrazioni umane. È ciò che rende possibile DQN.

Il rovescio: il max su stime rumorose produce *sovrastima sistematica* (maximization bias), perché $\mathbb{E}[\max] \geq \max \mathbb{E}$. È il problema che Double Q-learning risolve separando la selezione dell'azione dalla valutazione del suo valore, usando due stimatori distinti.

D4. *Differenza fra Q-learning e SARSA. Quando preferiresti uno o l'altro?*

$$\text{Q-learning: } r + \gamma \max_{a'} Q(s', a'), \quad \text{SARSA: } r + \gamma Q(s', a'),$$

dove in SARSA a' è l'azione *effettivamente scelta* dalla politica di comportamento in s' . Da qui il nome, dalla quintupla (s, a, r, s', a') . SARSA è quindi **on-policy**: impara il valore della politica che sta seguendo, esplorazione inclusa.

Con numeri: $Q(s, a) = 2.0$, $r = 1$, $\gamma = 0.9$, $\alpha = 0.1$, con $\max_{a'} Q(s', a') = 3.0$ e l'azione esplorativa scelta che vale 1.0. Q-learning dà $2.0 + 0.1(1 + 2.7 - 2.0) = \mathbf{2.17}$; SARSA dà $2.0 + 0.1(1 + 0.9 - 2.0) = \mathbf{1.99}$. Stessa transizione, aggiornamenti di segno opposto.

Quando preferire l'uno o l'altro. *SARSA* quando l'esplorazione ha un costo reale durante l'apprendimento — robotica, sistemi fisici, qualunque contesto in cui un errore fa danno. L'esempio canonico è il *cliff walking*: Q-learning impara il cammino ottimo rasente il burrone, perché nel calcolo assume che poi si comporterà greedy; ma se durante l'esecuzione si continua a esplorare con ε -greedy, ogni tanto si cade. SARSA impara un cammino più lungo e prudente, e ottiene un ritorno effettivo migliore.

Q-learning quando si vuole la politica ottima indipendentemente da come si esplora, quando si vuole riusare esperienza off-policy, o quando l'esplorazione è simulata e gli errori non costano nulla.

Nota: se $\varepsilon \rightarrow 0$ le due convergono allo stesso limite. La differenza vive nel regime esplorativo.

D5. Cos'è il *deadly triad*? Come DQN mitiga?

È la combinazione di tre ingredienti che, presi insieme, possono far divergere l'apprendimento anche in problemi semplici:

1. **function approximation** (una rete al posto di una tabella);
2. **bootstrapping** (il target usa la stima corrente);
3. **off-policy** (si impara su una distribuzione di dati diversa da quella della politica target).

Presi a due a due sono innocui; tutti e tre insieme rompono le garanzie di convergenza — l'esempio classico di divergenza è quello di Baird. Il meccanismo: con l'approssimazione, aggiornare $Q(s, a)$ modifica anche $Q(s', a')$, che compare nel target; l'errore si retroalimenta, e la distribuzione off-policy può amplificarlo invece di correggerlo.

DQN non elimina il triade — ne ha tutti e tre gli ingredienti — ma la stabilizza con tre accorgimenti:

- **experience replay**: si campiona da un buffer di transizioni passate invece di usare quella corrente. Rompe la correlazione temporale dei campioni e li rende più simili a un campionamento i.i.d., riducendo la varianza degli aggiornamenti;
- **target network**: il target usa una copia $\hat{Q}$ dei pesi aggiornata solo ogni C passi. Congela il bersaglio, spezzando il ciclo di retroazione che è la causa della divergenza;
- **reward clipping** a $[-1, 1]$: rende la scala del gradiente confrontabile fra giochi diversi, a costo di perdere l'informazione sulla magnitudine delle ricompense.

Le estensioni successive raccolte in *Rainbow* affrontano gli altri difetti: Double DQN per il maximization bias, prioritized replay per campionare le transizioni più informative, dueling architecture per separare valore e advantage.

D6. Enuncia il *policy gradient theorem* e ricavalalo via *log-derivative trick*.

Con $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) Q^{\pi_\theta}(s_t, a_t) \right].$$

Derivazione. Il problema è che la distribuzione rispetto a cui si media dipende da θ . Il *log-derivative trick* lo risolve:

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)} = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau),$$

quindi

$$\nabla_\theta J = \nabla_\theta \int p_\theta(\tau) R(\tau) d\tau = \int p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) R(\tau) d\tau = \mathbb{E}[\nabla_\theta \log p_\theta(\tau) R(\tau)].$$

Il passo decisivo è il successivo. La probabilità di una traiettoria è

$$p_\theta(\tau) = p(s_0) \prod_t \pi_\theta(a_t | s_t) P(s_{t+1} | s_t, a_t),$$

e prendendo il logaritmo la somma si spezza; ma $p(s_0)$ e $P(s_{t+1} | s_t, a_t)$ **non dipendono da θ** , quindi il loro gradiente è nullo:

$$\nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t).$$

La dinamica dell'ambiente sparisce. È questo che rende il metodo *model-free*: non serve conoscere P , e questo va detto esplicitamente perché è il punto del teorema.

D7. *Cos'è REINFORCE? Perché ha alta varianza e come si riduce?*

È l'implementazione Monte Carlo del policy gradient: si esegue un episodio completo, si calcolano i ritorni G_t e si aggiorna

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t.$$

È non distorto, perché $\mathbb{E}[G_t] = Q^{\pi}(s_t, a_t)$.

Perché ha alta varianza, tre ragioni cumulative. G_t è un campione singolo di una somma su tutta la traiettoria, quindi accumula la casualità di ogni transizione e di ogni scelta. Serve un episodio intero per un solo aggiornamento, quindi i dati sono pochi. E — punto sottile ma decisivo — se tutti i ritorni sono positivi, *tutte* le azioni vengono rese più probabili, e la distinzione fra buone e cattive si gioca solo sulle differenze relative dei moduli, sepolte in un segnale grande.

Come si riduce:

- **baseline:** si sottrae $b(s)$ dal ritorno. Non introduce bias, perché $\mathbb{E}[\nabla \log \pi \cdot b(s)] = b(s) \nabla \sum_a \pi(a | s) = b(s) \nabla 1 = 0$. Con ritorni (10, 12, 11) e $b = 11$, la scala che governa la varianza passa da $\mathbb{E}[R^2] = 121.7$ a $\mathbb{E}[A^2] = 0.67$: un fattore 182;
- **reward-to-go:** si usa $\sum_{k \geq t} \gamma^{k-t} r_k$ invece del ritorno totale, perché le ricompense *precedenti* all'azione non ne dipendono e aggiungono solo rumore;
- **bootstrapping con un critico:** si sostituisce G_t con una stima a bassa varianza, ed è actor-critic;
- normalizzazione dei vantaggi nel batch, GAE, e più traiettorie per aggiornamento.

D8. *Architettura actor-critic: ruolo di actor e critic. Vantaggio rispetto a REINFORCE puro.*

L'**actor** è la politica parametrizzata $\pi_{\theta}(a | s)$: decide che cosa fare e viene aggiornata con il policy gradient. Il **critic** è una value function parametrizzata $V_w(s)$ (o Q_w): non decide nulla, ma *valuta* quanto era buono ciò che l'attore ha fatto.

L'aggiornamento tipico usa il TD error come stima dell'advantage:

$$\delta_t = r_{t+1} + \gamma V_w(s_{t+1}) - V_w(s_t), \quad \theta \leftarrow \theta + \alpha_{\theta} \delta_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t), \quad w \leftarrow w + \alpha_w \delta_t \nabla_w V_w(s_t).$$

Il vantaggio rispetto a REINFORCE è duplice. *Varianza:* si sostituisce il ritorno campionato G_t , che dipende da tutta la traiettoria, con una stima bootstrap che dipende da una sola transizione. *Aggiornamenti online:* non serve più attendere la fine dell'episodio, quindi si può apprendere in problemi continui e con molta più frequenza.

Il costo, che va detto: si introduce **bias**, perché V_w è approssimata e il target è bootstrappato. Si è passati da uno stimatore non distorto ad alta varianza a uno distorto a bassa varianza — lo stesso compromesso di MC contro TD, applicato al gradiente della politica. GAE (Generalized Advantage Estimation) parametrizza esplicitamente questo compromesso con λ .

Il collegamento da fare: l'actor risolve il problema delle azioni continue e delle politiche stocastiche, il critic recupera parte della stabilità dei metodi value-based. È la sintesi delle due famiglie.

D9. *PPO: cos'è il clipped objective e perché stabilizza il training?*

Il problema che risolve: nel policy gradient, un passo troppo grande può portare la politica in una regione pessima, e da lì i dati raccolti sono cattivi e non c'è modo di tornare indietro. A

differenza dell'apprendimento supervisionato, la distribuzione dei dati *dipende dai parametri*, quindi un errore si autoalimenta.

PPO limita quanto la politica può cambiare a ogni aggiornamento. Con il rapporto di probabilità $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t) \right],$$

con ε tipicamente 0.1–0.2.

Come leggere il min, che è la parte non ovvia. Se $\hat{A}_t > 0$ (azione buona) l'obiettivo cresce con r_t , ma il clipping lo appiattisce oltre $1 + \varepsilon$: aumentare ulteriormente la probabilità non porta più guadagno, quindi non c'è incentivo a un passo grande. Se $\hat{A}_t < 0$ (azione cattiva) il clipping agisce sotto $1 - \varepsilon$, limitando quanto si può abbassare. Il min garantisce che l'obiettivo sia un *lower bound pessimistico* dell'obiettivo non clippato: il clipping non può mai far apparire un passo migliore di quanto sia.

Perché stabilizza: si possono fare **più epoche di ottimizzazione sugli stessi dati** senza che la politica si allontani troppo da quella che li ha generati, il che rende PPO molto più efficiente in campioni del policy gradient vanilla, senza il costo computazionale del vincolo KL esatto di TRPO (che richiede un prodotto con la matrice di Fisher). È l'algoritmo di riferimento per il controllo continuo e per l'RLHF.

D10. Confronta value-based, policy-based e actor-critic: quando serve l'uno e quando l'altro.

	Value-based	Policy-based	Actor-critic
Che cosa impara	$Q(s, a)$	$\pi_\theta(a s)$	entrambi
Azioni continue	no (arg max difficile)	sì	sì
Politiche stocastiche	no (greedy)	sì	sì
Varianza	bassa	alta	media
Bias	medio (bootstrap)	nullo (MC)	medio
Convergenza	può oscillare	ottimo locale, regolare	compromesso
Efficienza in campioni	alta (off-policy)	bassa (on-policy)	media

Value-based (DQN e varianti) quando le azioni sono **discrete e poche**, e si vuole efficienza in campioni grazie al replay off-policy. Atari è il caso paradigmatico.

Policy-based puro quando serve una politica stocastica per ragioni strutturali — giochi con avversario, ambienti parzialmente osservabili — o come componente di metodi più complessi. Raramente si usa da solo per via della varianza.

Actor-critic (A3C, PPO, SAC, DDPG) è la scelta di default per il **controllo continuo**: robotica, locomozione, guida. Copre entrambi i requisiti — azioni continue e varianza gestibile — ed è ciò che si usa nella pratica moderna.

Il criterio in una riga: se l'arg max su $\mathcal{A}$ è banale, si può restare value-based; appena non lo è, serve un attore.

D11. Perché nel VAE si può usare il reparametrization trick e nel policy gradient no?

Perché la variabile casuale rispetto a cui si deriva ha ruoli diversi rispetto alla funzione da ottimizzare.

Nel VAE si campiona $\mathbf{z} \sim q_\phi(\mathbf{z} | \mathbf{x})$, una *gaussiana continua* i cui parametri sono prodotti dalla rete, e la funzione da mediare $f(\mathbf{z}) = \log p_\theta(\mathbf{x} | \mathbf{z})$ è *differenziabile in $\mathbf{z}$* . Si può quindi scrivere $\mathbf{z} = \boldsymbol{\mu}_\phi + \boldsymbol{\sigma}_\phi \odot \boldsymbol{\epsilon}$ e derivare *attraverso* il campione: è lo stimatore **pathwise**, che usa $\nabla_{\mathbf{z}} f$ e ha varianza bassa.

Nel policy gradient si campiona un'azione $a \sim \pi_\theta(a | s)$ e la funzione da mediare è il ritorno, che dipende da a *attraverso l'ambiente*. Due ostacoli, ciascuno sufficiente: le azioni

sono spesso **discrete**, e per una categorica non esiste una riparametrizzazione location-scale; e anche con azioni continue, il ritorno non è una funzione differenziabile nota di a — l'ambiente è una scatola nera, di cui non si ha $\nabla_a R$.

Resta quindi lo stimatore **score function** (REINFORCE), che usa solo il *valore* scalare del ritorno moltiplicato per $\nabla_{\theta} \log \pi_{\theta}$, e non richiede alcuna derivata rispetto all'azione. Il prezzo è la varianza molto più alta — da cui baseline, critico e GAE.

Il quadro generale, che è la risposta di terzo livello: sono due stimatori dello stesso oggetto $\nabla_{\theta} \mathbb{E}_{p_{\theta}}[f]$, con requisiti diversi. Il pathwise richiede una distribuzione riparametrizzabile e una f differenziabile; lo score function non richiede nulla di tutto ciò e paga in varianza. Dove entrambi sono applicabili, il pathwise vince nettamente — ed è il motivo per cui i metodi che rendono differenziabili le variabili discrete (Gumbel-Softmax) o l'ambiente (model-based, differentiable simulators) sono un'area attiva.

Chapter 7

Parte VII — GrIDDD

Le risposte di questo capitolo seguono i tre livelli del capitolo 31 della dispensa. Poiché è il punto da cui l'orale più probabilmente parte, le risposte di livello 1 vanno sapute a memoria come racconto fluido, non come elenco.

Livello 1 — sai raccontarlo

D1. *Racconta GrIDDD in tre minuti: problema, idea, risultato principale.*

Problema. I diffusion model discreti su grafi molecolari — DiGress, MiDi, FreeGress — hanno taglia fissa: il numero di atomi si decide prima di iniziare la diffusione e non cambia più. Per le molecole è una limitazione seria, perché molte proprietà chimiche correlano direttamente col numero di atomi.

Idea. Rendere la taglia una variabile del processo: si permettono inserimenti e cancellazioni **monotone** di nodi sia nel forward sia nel reverse, incorporandoli *durante il training* e non come post-processing. I timestep degli edit sono campionati da una distribuzione logistica $\zeta'(t)$ concentrata a metà del processo. Le cancellazioni sono modellate con un tipo di atomo aggiuntivo DEL, assorbente per garantire la monotonia, più uno stato transiente DEL* che permette al reverse di re-inserire. Il reverse usa un *posterior generalizzato* che marginalizza sul tempo di attivazione di ciascun nodo, perché i nodi inseriti non esistono in G^0 .

Risultato. Su property optimization di QED il success rate è 45,1% contro il 9,4% di GCPN, quasi cinque volte. Su property targeting il caso che racconta meglio il paper è il peso molecolare: MAE 4,89 contro 8,96 di FreeGress, ottenuto *senza* conoscere a priori la distribuzione delle taglie.

D2. *Qual è il problema concreto che risolve?*

I DDPM su grafi agiscono sui *tipi* di atomo e di legame, non sulla cardinalità del grafo: il numero di nodi è un parametro dato in ingresso al processo di generazione. GrIDDD rende quel numero una quantità che il modello può modificare mentre denoizza, quindi **disaccoppia la distribuzione delle taglie dalla distribuzione delle proprietà** — cosa che fino a quel momento i metodi DDPM su grafi non sapevano fare.

D3. *Perché la taglia fissa è un problema per le molecole e non, per esempio, per le immagini?*

Perché per un'immagine la risoluzione è una proprietà del *formato*, scelta a monte, e nessuno chiede a un modello di generare “un'immagine con il numero ottimale di pixel”. Per una molecola il numero di atomi è una proprietà dell'*oggetto*, e molte grandezze chimiche vi correlano direttamente — il peso molecolare in modo quasi definitorio, ma anche LogP e QED in modo forte. Se il target è “peso molecolare 250”, la taglia non è un iperparametro: è parte della risposta.

D4. *Come lo risolvevano prima DiGress, MiDi e FreeGress?*

DiGress e *MiDi* campionano il numero di nodi dalla distribuzione empirica delle taglie del training set, quindi in modo indipendente dal condizionamento. *FreeGress* — lavoro precedente dello stesso gruppo — fa un passo in più e *predice* la taglia con un classificatore ausiliario a partire dall'informazione di condizionamento.

In entrambi i casi la decisione è presa **prima** e non viene mai rivista durante la generazione.

D5. *Perché quelle soluzioni bastano per il property targeting ma non per la property optimization?*

Nel *property targeting* si genera da zero una molecola con una proprietà voluta: si può ancora indovinare la taglia con un modello ausiliario e poi generare a taglia fissa, perché non c'è nessun vincolo su come debba essere fatta la molecola.

Nella *property optimization* si parte da una molecola esistente e la si modifica per migliorarne una proprietà, mantenendo una similarità minima con l'originale. Qui la taglia è **legata alla struttura che si sta modificando**: fissarla dall'esterno significa vietarsi metà dello spazio delle soluzioni, perché molte modifiche utili richiedono di aggiungere o togliere atomi.

D6. *Qual è il claim in una frase?*

Inserimenti e cancellazioni **monotone** di nodi durante sia il forward sia il reverse process, incorporate **durante il training**.

D7. *Su quali dataset è valutato, e qual è il numero più forte?*

QM9 (133k molecole fino a 9 atomi non idrogeno) e *ZINC-250k* (250k molecole drug-like), con $T = 500$ timestep, addestramento su NVIDIA A100.

Il numero più forte è il **success rate del 45,1% su QED** in property optimization, contro il 9,4% di GCPN e il 4,8% di CG-VAE. È anche il task in cui la flessibilità di taglia conta di più, quindi il risultato è coerente con la tesi del paper e non un artefatto.

Livello 2 — sai come funziona**D1.** *Che cosa vuol dire che gli edit sono monotoni?*

Che una volta che un nodo è stato inserito o cancellato, non cambia più direzione: un nodo cancellato non torna in vita durante il forward, e un nodo inserito non viene poi rimosso. Serve a rendere il processo ben definito e a permettere di contare esattamente quanti edit servono, $|\Delta^T| = n^T - n^0$. È anche la proprietà che costringe a usare uno stato *assorbente* per la cancellazione — e da lì nasce tutto il meccanismo DEL/DEL*.

D2. *Quanti edit servono e come si distribuiscono nel tempo? Che ruolo hanno D e w ?*

Il numero è $|\Delta^T| = n^T - n^0$, differenza fra la taglia del latente e quella del dato. I timestep in cui avvengono sono campionati da

$$\zeta'(t) = \left| \frac{e^{(t-D)/w}}{w(e^{(t-D)/w} + 1)^2} \right|,$$

il valore assoluto della densità di una distribuzione logistica, normalizzata così che $\sum_{t=0}^T \zeta'(t) = 1$ e $\zeta'(0) = \zeta'(T) = 0$.

D indica il timestep in cui gli edit sono *massimizzati*, w la *ripidità* della curva. Nel paper $T = 500$, $D = T/2 = 250$, $w = 0.05$: la maggior parte degli edit avviene a metà del processo.

Il perché di quella scelta va saputo: inserire un nodo quando il grafo è quasi pulito significa piazzare un atomo estraneo in una struttura già formata, che il denoising residuo non ha

tempo di riassorbire; agire quando è già rumore puro è inutile, perché non c'è struttura da rispettare. Il punto utile è in mezzo. L'integrale $\zeta(t)$ è la cumulativa e compare dentro le matrici di transizione modificate.

D3. Spiega il trucco DEL/DEL*. Perché servono due stati e non uno?

La cancellazione è modellata come un tipo di atomo aggiuntivo DEL. Perché la monotonia sia garantita, DEL deve essere **assorbente**: $p(x^t \neq \text{DEL} \mid x^{t-1} = \text{DEL}) = 0$. Se non lo fosse, un nodo cancellato potrebbe tornare in vita durante il forward e la monotonia salterebbe.

Ma un assorbente puro **rompe il reverse**. Il reverse percorre la catena al contrario e deve poter re-inserire un nodo che il forward aveva cancellato; se da DEL non si esce mai, quel nodo resta morto per sempre e il modello non può generare nulla che il forward avesse tolto.

La soluzione è spezzare la cancellazione in due stati. DEL* è **transiente**: nel forward, da DEL* si scivola deterministicamente in DEL; nel reverse, da DEL* si può uscire verso un tipo di atomo proprio non-DEL. DEL resta l'assorbente vero.

L'immagine da usare: DEL* è la *soglia*, DEL è la stanza chiusa. La monotonia la garantisce la stanza chiusa, la reversibilità la soglia. Un solo stato non può fare entrambe le cose, perché le due proprietà sono in contraddizione diretta.

D4. Come si garantisce che esattamente $|\Delta^T|$ nodi siano in DEL al tempo T ?

Con uno **schema ibrido**: solo $|\Delta^T|$ nodi seguono la matrice di transizione speciale

$$Q^{*t} = \zeta(t)(\alpha^t A^* + (1 - \alpha^t)B^*) + (1 - \zeta(t))C^*,$$

mentre tutti gli altri seguono il forward standard. Poiché ζ è una cumulativa, a $t = T$ si ha $Q^{*T} = C^*$, che è la matrice che porta deterministicamente nello stato di cancellazione. Le matrici A^*, B^*, C^*, D^* codificano le transizioni verso DEL e DEL*.

D5. Scrivi il forward standard ereditato da DiGress/FreeGress. Che cos'è il “rumore” nel caso discreto?

$$q(G^t \mid G^{t-1}) = (X^{t-1}Q_X^t, E^{t-1}Q_E^t), \quad Q_X^t = \alpha^t A_X + (1 - \alpha^t)B_X,$$

con $A_X = \mathbf{I}$ e $B_X = \mathbf{1} \mathbf{m}_X^\top$, dove $\mathbf{m}_X$ è la *distribuzione marginale* dei tipi di atomo nel training set. Le multi-step si compongono come $Q_X^{t|s} = \alpha^{t|s} A_X + (1 - \alpha^{t|s})B_X$, con schedule coseno alla MiDi ($\nu = 1$ per i nodi, $\nu = 1.5$ per gli archi).

Il “rumore” nel caso discreto è la **marginale del dataset**, non una gaussiana. Con $\alpha^t \approx 1$ non cambia nulla; con $\alpha^t \approx 0$ ogni nodo viene rimpiazzato da un tipo campionato da $\mathbf{m}_X$. Quindi lo stato terminale non è rumore bianco: è una molecola con composizione atomica *plausibile* — più carbonio e azoto che atomi rari, nelle proporzioni giuste — ma struttura completamente casuale. È la scelta corretta per lo stesso motivo per cui nel continuo si sceglie $\mathcal{N}(0, \mathbf{I})$: serve una distribuzione terminale semplice, nota e facile da cui ripartire.

D6. Come funzionano gli inserimenti, e perché sono più semplici delle cancellazioni?

Un nodo viene *attivato* al timestep $s + 1$ se il modello ha campionato s come tempo di inserimento. La categoria iniziale del nodo si campiona dalla marginale $\mathbf{m}_X$, gli archi incidenti dalla marginale $\mathbf{m}_E$.

Sono più semplici perché non richiedono nessuno stato assorbente. La monotonia dell'inserimento è garantita banalmente dal fatto che un nodo attivato resta attivo, e non c'è la tensione fra assorbenza (necessaria nel forward) e reversibilità (necessaria nel reverse) che rende delicata la cancellazione. Il costo si sposta invece sul *posterior*, perché un nodo inserito non esiste in G^0 — ed è la domanda successiva.

D7. Qual è il problema del reverse process, e come lo risolve il posterior generalizzato?

Il problema: nella diffusion classica l'ELBO si costruisce su $q(z_{t-1} | z_t, x^0)$, che è trattabile perché condizionata sul dato originale. Ma in GrIDDD un nodo *inserito* durante il forward **non esiste in G^0** : per quel nodo x^0 non è definito, e il termine $p_\theta(x^{t-1} | x^t, x^0)$ è mal posto.

La soluzione è sostituire il condizionamento su x^0 con una *marginalizzazione sul tempo di attivazione $\hat{s}$* :

$$p_\theta(x^{t-1} | x^t, y) = \sum_{x \in \mathcal{X}} \frac{q(x^t | x^{t-1}) q(x^{t-1} | x^{\hat{s}} = x)}{q(x^t | x^{\hat{s}} = x)} p_\theta(x^{\hat{s}} = x | x^t, y).$$

Per un nodo esistente fin dall'inizio si ha $\hat{s} = 0$ e la formula collassa nel posterior classico: la generalizzazione è *conservativa*, non sostituisce ciò che già funzionava.

Il costo è che la rete deve predire un'uscita in più: non solo il tipo originale di nodi e archi, ma anche *quando* ciascun nodo è stato attivato. Da lì il termine dedicato nella loss.

D8. A che cosa serve la rete ausiliaria $g_\phi(G^t)$, e quando viene addestrata?

Predice **quanti DEL* aggiungere** al passo t , prima di passare il grafo al denoiser principale. Serve perché nel reverse i nodi vanno re-inseriti, e qualcuno deve decidere quanti: il denoiser principale lavora su un grafo di taglia data, non può crearla.

Viene addestrata in parallelo al modello principale e **solo nei timestep in cui $\zeta'(t) > 0$** , cioè dove gli inserimenti sono effettivamente possibili. Fuori da quell'intervallo non ci sarebbe nulla da predire, e addestrarla sarebbe rumore.

È anche la fonte del primo limite dichiarato: due reti che decidono cose correlate possono contraddirsi.

D9. Da che cosa è composta la loss?

Una somma di **quattro cross-entropy**, pesate da iperparametri:

$$\mathcal{L} = \lambda_X \text{CE}(X^{*0}) + \lambda_E \text{CE}(E^{*0}) + \lambda_S \text{CE}(S^*) + \lambda_{\text{DEL}} \text{CE}(n_{\text{DEL}*}),$$

cioè sui tipi dei nodi, sui tipi degli archi, sui *tempi di attivazione*, e sul numero di DEL* predetti dalla rete ausiliaria. I primi due sono ereditati da DiGress/FreeGress; il terzo e il quarto sono la novità, e corrispondono esattamente ai due meccanismi nuovi (posterior generalizzato e rete ausiliaria).

D10. Come si fa generazione condizionata? È un meccanismo nuovo o ereditato?

Ereditato da FreeGress, e va detto subito perché è la risposta onesta: GrIDDD non innova sul condizionamento, innova sugli edit di taglia.

Si usa la *classifier-free guidance* di Ho e Salimans con *conditional dropout*: durante il training, con probabilità ρ il vettore di condizionamento y viene sostituito da un placeholder parametrizzato $\bar{y}$, così che lo stesso modello impari sia il condizionale sia l'incondizionato. A inferenza si combinano le due predizioni pesando per λ .

È lo stesso meccanismo del Cap. 23 della dispensa, con lo stesso trade-off: aumentando il peso della guidance si guadagna aderenza alla condizione e si perde diversità.

Livello 3 — sai criticarlo e collocarlo

D1. Colloca GrIDDD nella tassonomia generativa del corso.

Ramo *explicit* $\rightarrow$ *latent* $\rightarrow$ *variational*: lo stesso ramo dei VAE, perché entrambi massimizzano un ELBO della log-verosimiglianza anziché la verosimiglianza esatta.

La caratteristica che distingue i diffusion dentro quel ramo è che lo spazio latente ha **la stessa dimensionalità dei dati** — non c'è compressione, i latenti z_t sono grafi come G^0 . GrIDDD eredita la proprietà con una torsione: quella cardinalità ora *può cambiare durante il processo*, invece di essere un invariante.

Sulle cinque lenti del corso: lente 1 (famiglia generativa) ramo diffusion esteso a taglia variabile; lente 2 (ELBO) ELBO gerarchico con posterior generalizzato; lente 3 (message passing) il denoiser è un graph transformer; lente 4 (attention) eredita la classifier-free guidance, non l'attention in sé; lente 5 (causalità) nessun legame diretto — e saperlo dire è meglio che forzare un collegamento inesistente.

D2. *Se un diffusion model è un VAE gerarchico con encoder fisso, che cosa cambia in GrIDDD?*

Che l'encoder **non è più del tutto fisso**. Nella diffusion classica il forward process è interamente specificato a priori — schedule del rumore, matrici di transizione — e non contiene parametri appresi: è questo che si intende per “encoder fisso”.

In GrIDDD la parte di edit di taglia è *appresa*: il tempo di attivazione dei nodi è predetto dal modello, e il numero di DEL* da re-inserire è predetto dalla rete ausiliaria. Quindi una componente del processo di corruzione/ricostruzione è parametrizzata.

E cambia una seconda cosa, più strutturale: la dimensione dello spazio latente non è più un invariante del processo. In un VAE gerarchico classico ogni livello ha una dimensione fissata dall'architettura; qui la cardinalità del grafo latente varia lungo la catena.

D3. *Perché il caso MW è quello che racconta meglio il paper?*

Perché il peso molecolare correla in modo quasi *definitorio* con il numero di atomi: è precisamente la proprietà per cui la flessibilità di taglia è progettata. I numeri: MAE $4,89 \pm 0,57$ contro $8,96 \pm 1,93$ di FreeGress e $20,92$ di DiGress, con validità $84,2\%$ contro $79,7\%$ e $40,4\%$.

Ma il punto decisivo non è il numero, è *come* ci si arriva. FreeGress dimezza il proprio MAE su MW **grazie** a una rete ausiliaria che predice la taglia ottimale dal target: cioè ottiene il risultato con un espediente specifico per quel problema. GrIDDD ottiene un errore inferiore **senza conoscere a priori la distribuzione delle taglie**, perché la taglia emerge dal processo generativo stesso. È la differenza fra risolvere il sintomo e risolvere la causa, ed è esattamente la tesi del paper.

D4. *Discuti il trade-off MAE/validità su QM9: è un miglioramento uniforme?*

No, e va detto. Su QM9 il quadro è misto:

- momento di dipolo μ : MAE $0,66$ contro $0,74$ di FreeGress — meglio — ma validità $79,0\%$ contro $83,7\%$ — peggio;
- HOMO: MAE $0,37$ contro $0,32$ — peggio — ma validità 94% contro $90,1\%$ — meglio.

Quindi su μ si guadagna in accuratezza pagando in validità, su HOMO il contrario. Non c'è dominanza.

L'interpretazione ragionevole: QM9 contiene molecole piccole (fino a 9 atomi non idrogeno), quindi il margine di manovra sulla taglia è ridotto e il vantaggio strutturale di GrIDDD si vede poco. Su ZINC-250k, con molecole drug-like più grandi e proprietà più legate alla taglia, il quadro è nettamente più favorevole. Dire questo, invece di presentare il paper come un miglioramento uniforme, è ciò che distingue una lettura critica da un riassunto.

D5. *Che cosa dimostra l'esperimento out-of-distribution?*

Si addestrano DiGress e GrIDDD su QM9 (molecole fino a 9 atomi pesanti), forzando GrIDDD a inserire fino a 14 nodi, e si chiede poi a entrambi di generare molecole fino a 15 atomi — cioè *oltre* la capacità di training di GrIDDD.

Risultato: fino a 12 atomi i due sono comparabili; oltre, la validità di DiGress crolla mentre **GrIDDD mantiene il 35% di validità a 15 nodi**.

Che cosa dimostra: che il modello ha imparato la *flessibilità di taglia* come meccanismo, non semplicemente a restare nel range visto in training. È un'evidenza di generalizzazione strutturale, non di interpolazione — e siccome $15 > 14$, cioè oltre la capacità di inserimento vista in addestramento, è un test genuinamente out-of-distribution.

D6. *Che cosa dice l'ablation, e perché è importante per l'attribuzione del risultato?*

Disabilitando insert e delete, il success rate su QED scende da 45,1% a **33,8%**, mentre resta pressoché invariato su LogP e DRD2.

È importante perché stabilisce l'*attribuzione causale* del risultato: il guadagno viene dalla novità del paper, non da altro — non da un'architettura più grande, non da un tuning migliore, non dalla pipeline ereditata da FreeGress. Senza ablation, il 45,1% sarebbe un numero impressionante di provenienza ambigua.

Il fatto che il calo si veda *su QED e non sugli altri task* è coerente e rafforza la tesi: QED è la proprietà più legata alla taglia fra le tre, quindi è lì che ci si aspetta che gli edit contino.

D7. *Perché il baseline naive con classe PAD non basta?*

Il baseline simula gli edit usando una classe PAD per nodi e archi in eccesso, cioè mantiene un grafo di taglia massima fissa e marca come “non esistenti” i nodi in più — l'approccio ovvio, e va detto che gli autori l'hanno provato.

Risultato: genera circa il 3% di molecole valide in più, ma è **drammaticamente peggiore su tutto il resto**: numero medio di componenti connesse, cross-entropy, e in generale la qualità strutturale.

La ragione concettuale: con PAD il grafo ha sempre la stessa taglia, e i nodi “assenti” partecipano comunque al message passing come token speciali. Il modello deve imparare a ignorarli, sprecando capacità, e la struttura reale è diluita in un grafo più grande del necessario. Soprattutto, l'informazione “questo nodo non c'è” è trattata come un valore di feature invece che come una proprietà del processo generativo — quindi non c'è nulla che garantisca monotonia o coerenza temporale degli edit. È la differenza fra simulare un meccanismo e modellarlo.

D8. *Quali sono i due limiti dichiarati dagli autori?*

Sono due e precisi, e non vanno improvvisati.

Primo: le due reti possono **entrare in conflitto**. Occasionalmente il modello principale predice un'attivazione e la rete ausiliaria una cancellazione nello stesso timestep, il che è *formalmente illegale* nel framework — le due operazioni sono mutuamente esclusive per costruzione. È una conseguenza diretta di avere due predittori indipendenti su decisioni correlate.

Secondo: il modello tende a produrre più molecole **spezzate**, cioè con un numero maggiore di componenti connesse, rispetto a FreeGress e DiGress. La causa è strutturale: i nodi inseriti tardi nel processo hanno pochi archi e faticano a connettersi prima della fine del denoising. Gli autori suggeriscono di dare in input il numero di componenti connesse come feature.

Il secondo limite è anche un ottimo ponte verso la Parte V: un nodo appena inserito ha grado quasi nullo, quindi nell'aggregazione riceve pochissimi messaggi — è un caso estremo di under-reaching, dove manca non la profondità ma proprio il vicinato.

D9. *Quanto costa in training e in sampling?*

In **training** è circa il 30% più lento di FreeGress (nell'appendice il numero riportato è $\approx 27\%$), per l'overhead delle due reti: il denoiser principale deve predire anche i tempi di attivazione, e la rete ausiliaria va addestrata in parallelo.

In **sampling** è spesso *più veloce*, perché può adattare il numero di step al task invece di eseguirne sempre un numero fisso.

Il bilancio è quindi favorevole nel caso d'uso reale, dove un modello si addestra una volta e si campiona molte: si paga una volta il 30% e si guadagna a ogni generazione.

D10. *Come cambieresti il paper?*

La modifica più diretta riguarda il primo limite. Il conflitto fra le due reti nasce dall'avere *due predittori indipendenti* su decisioni mutuamente esclusive. Si potrebbe sostituirli con un'**unica testa** che predice l'edit come variabile categorica a tre valori — inserisci, cancella, nessuna operazione — per ciascun nodo e timestep. Così l'esclusività è garantita per costruzione dalla softmax, invece di essere sperata, e sparisce anche il termine di loss separato λ_{DEL} .

Sul secondo limite, l'idea degli autori (numero di componenti connesse come feature) è ragionevole ma *reattiva*: informa il modello del problema senza prevenirlo. Un'alternativa *proattiva* sarebbe modificare $\zeta'(t)$ per anticipare gli inserimenti, dando ai nodi nuovi più passi di denoising per collegarsi — al costo di inserirli quando la struttura è ancora più rumorosa, quindi c'è un compromesso da esplorare empiricamente. Oppure un termine di loss che penalizzi esplicitamente le componenti multiple.

Il tono giusto per questa domanda: proporre qualcosa di concreto e riconoscerne il costo, non elencare desideri.

D11. *Se dovessi fare la stessa cosa con un flow matching su grafi invece che con un DDPM, che cosa cambierebbe?*

Cambierebbe soprattutto in peggio, e la ragione è precisa: il flow matching definisce un *cammino deterministico* governato da un'ODE, e la cardinalità variabile è molto più scomoda in quel quadro.

Nel DDPM discreto si ha una **catena di stati categorici**, e su un insieme discreto di stati si può aggiungere uno stato speciale con proprietà scelte — assorbente, transiente — e definire matrici di transizione che lo trattino come si vuole. È esattamente ciò che permette il trucco DEL/DEL*.

In un flow matching su spazio continuo non esiste una catena di stati discreti su cui appoggiare uno stato assorbente: c'è un campo di velocità su una varietà, e “il numero di nodi” non è una coordinata su cui si possa integrare un'ODE. Servirebbe qualcosa di diverso — un flow matching su spazi discreti (che esiste, ed è un'area attiva), oppure trattare la cardinalità come una variabile separata con una dinamica propria, il che romperebbe l'eleganza dell'approccio unificato.

Un vantaggio ci sarebbe: il sampling in pochi passi. Ma il costo concettuale è alto, e vale la pena dire che questa è una domanda aperta, non un'estensione ovvia.

D12. *Il denoiser fa message passing: valgono oversmoothing e oversquashing? Che cosa succede a un nodo appena inserito?*

Il denoiser è un *graph transformer* sulla scia di DiGress/MiDi, quindi con attenzione globale. Da qui la risposta articolata.

Over-squashing e *under-reaching* sono largamente **attenuati**, perché ogni coppia di nodi comunica direttamente e non c'è bottleneck topologico. È la motivazione dichiarata nel paper per quella scelta: i DDPM su grafi devono raffinare iterativamente interazioni a lungo raggio come le chiusure di anelli.

Over-smoothing resta un rischio, perché dipende dalla profondità e dall'aggregazione, non dalla connettività; nei Transformer si manifesta come *token uniformity* e si contrasta con connessioni residue e normalizzazioni, che il blocco Transformer ha per costruzione.

Il nodo appena inserito è il caso interessante. Ha pochissimi archi, quindi in un message passing locale riceverebbe pochi messaggi: sarebbe under-reaching estremo, dove manca il

vicinato stesso. L’attenzione globale mitiga — il nodo può comunque attendere su tutti — ma non elimina il problema, perché le sue *key* restano poco informative: la sua rappresentazione locale è ancora quasi vuota, quindi gli altri nodi hanno poca ragione di attendere su di lui.

Ed è coerente con il secondo limite dichiarato: le molecole spezzate nascono proprio da nodi inseriti tardi che non riescono a collegarsi. Il rimedio suggerito dagli autori — il numero di componenti connesse come feature di input — è esattamente un *feature strutturale globale*, nello spirito dei positional encoding dei graph transformer.